



UNIVERSIDAD
COMPLUTENSE
MADRID

Apache Spark

Dr. Pablo J. Villacorta

CONTENTS

1 Capítulo 1: Introducción a las Tecnologías Big Data **3**
- 1.1 Origen de las Tecnologías Big Data . . . 3
- 1.2 La Transformación Digital . . . 6
- 1.3 Definición de *Tecnologías Big Data* . . . 7
- 1.4 Origen de las tecnologías de procesamiento distribuido . . . 9
- 1.5 El ecosistema Hadoop . . . 11
- 1.6 Bibliografía recomendada . . . 14

2 Capítulo 2: Apache Spark I (Core y SQL) **15**
- 2.1 Introducción . . . 15
- 2.2 Arquitectura de Spark . . . 19
- 2.3 RDD: Resilient Distributed Datasets . . . 23
- 2.4 DataFrames y Spark SQL . . . 25
- 2.5 Bibliografía recomendada . . . 48

3 Capítulo 3: Catálogo de metadatos de Hive **49**
- 3.1 Antecedentes: Apache Hive . . . 49
- 3.2 Unidades de información en el metastore de Hive . . . 51
- 3.3 Databricks y el catálogo de Hive . . . 54

4 Capítulo 4: Spark MLlib **57**
- 4.1 Despliegue de modelos en producción con Spark . . . 58
- 4.2 Estimadores y transformadores . . . 59

5 Capítulo 5: Structured Streaming y GraphFrames **79**
- 5.1 Structured Streaming . . . 79
- 5.2 Procesamiento de grafos con GraphFrames . . . 84

¡Bienvenidos al curso de Spark! En él vamos a tratar con varias herramientas Big Data que se ejecutan sobre un cluster de ordenadores. La principal que vamos a ver en detalle es Apache Spark, un framework open-source de procesamiento distribuido que permite hacer cualquier tipo de operación con datos sobre un cluster, para poder manipular y transformar datasets realmente grandes - aunque nosotros vamos a usar un cluster de juguete gratuito. Todo el trabajo lo vamos a hacer utilizando notebooks de la plataforma de la empresa Databricks (similares a los de Jupyter pero con algunas "vitaminas") que estarán enlazados con un cluster real de Spark, gracias a la infraestructura proporcionada por Databricks y Microsoft Azure. Todo se hará en la nube, por lo que no tendréis que instalar nada en vuestro ordenador, y por supuesto, es accesible desde cualquier sitio donde tengáis conexión a internet.

Para poder llevar a cabo nuestras actividades, es necesario leer detenidamente y seguir las instrucciones de la guía de la siguiente sección, para crear una cuenta en Azure, una cuenta de almacenamiento en Azure y a continuación levantar la plataforma de Databricks, y dentro de ella, un cluster sencillito (Personal compute). Estas herramientas son las mismas que habíais configurado en el curso de Tecnologías del Big Data, por lo que quienes ya hicisteis esta configuración, no es necesario que la volváis a crear, a pesar de que hayáis escogido nombres diferentes para los servicios (no supone ningún problema). El cluster de Databricks está programado por defecto para desmantelarse tras permanecer inactivo cierto tiempo, lo cual es beneficioso, aunque eso también lo podéis configurar.

El curso está estructurado de la siguiente manera:

- *Capítulo 1: Introducción a las Tecnologías Big Data*
- *Capítulo 2: Apache Spark I (Core y SQL)*
  - *Introducción*
  - *Arquitectura de Spark*
  - *RDD: Resilient Distributed Datasets*
  - *DataFrames y Spark SQL*
- *Capítulo 3: Catálogo de metadatos de Hive*
- *Capítulo 4: Spark MLlib*
- *Capítulo 5: Structured Streaming y GraphFrames*

La correspondencia entre capítulos y semanas podría ser:

- Semana 1: Introducción, Spark Core y SQL
- Semana 2: Catálogo de metadatos de Hive y Spark MLlib
- Semana 3: Spark Structured Streaming y GraphFrames

CHAPTER

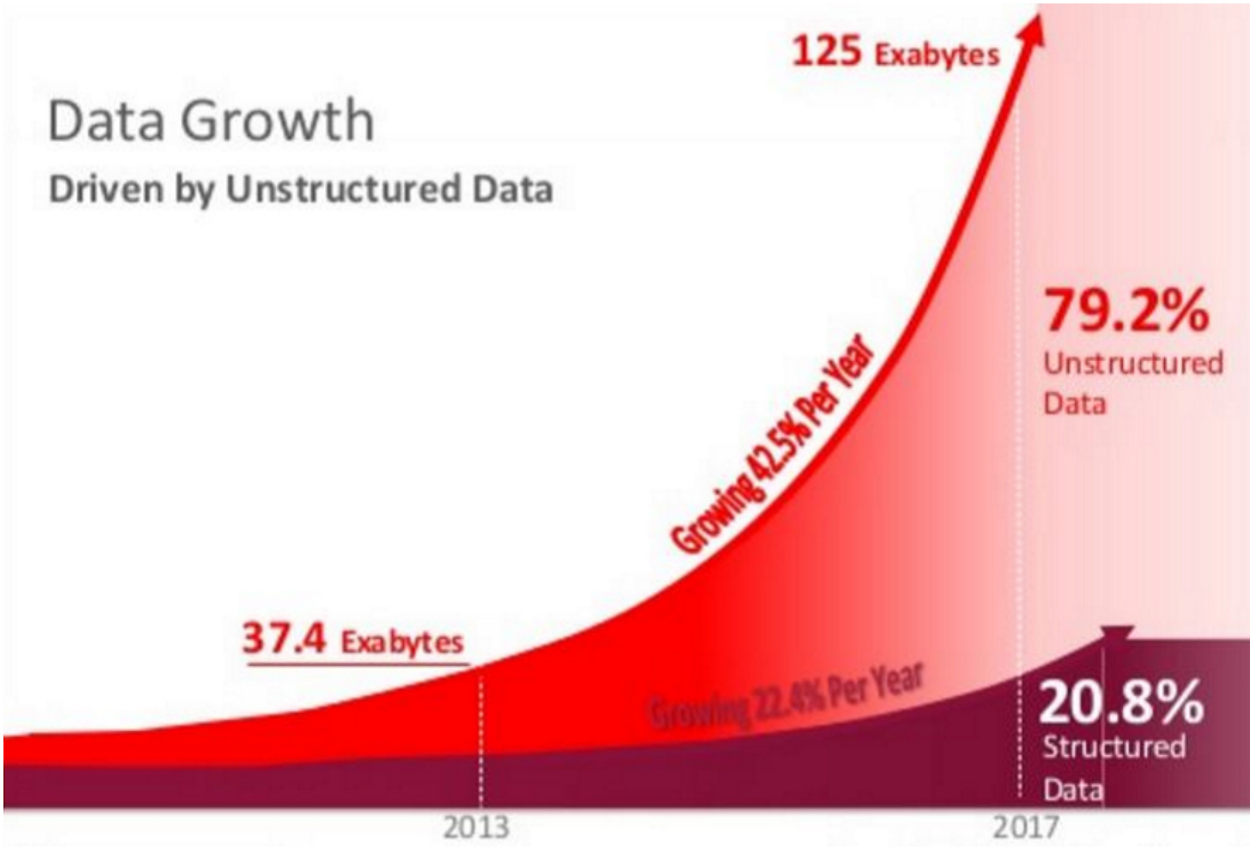
ONE

CAPÍTULO 1: INTRODUCCIÓN A LAS TECNOLOGÍAS BIG DATA

Este capítulo explica la motivación del resto de contenidos que estudiaremos en el curso, desde el punto de vista de las necesidades que resuelven, y en qué suponen un cambio con respecto a tecnologías anteriores.

1.1 Origen de las Tecnologías Big Data

Las tecnologías Big Data surgen para dar respuesta a las nuevas necesidades de la sociedad actual. Vivimos en un mundo interconectado, en el que el 90 % de la información existente, persistida en medios de cualquier tipo, se ha creado en los últimos 2 años. El crecimiento de la información producida en el mundo, por fuentes de todo tipo tanto físicas como electrónicas, es exponencial desde hace unos años. Aunque las estimaciones acerca del volumen divergen, la siguiente gráfica muestra de manera orientativa este fenómeno:



Source: ISD - 2014, Structured Data vs. Unstructured Data: The Balance of Power Continues to Shift

Fig. 1. Tasa de crecimiento estimado de datos en todo el mundo según Oracle

Casi el 80 % de los datos que se crean son generados por personas, y por ello suelen ser datos no estructurados. El 20 % restantes son datos estructurados generados por máquinas (datos de logs, sensores, IOT -Internet de las Cosas- en general) con el fin de ser procesados generalmente por otras máquinas.

1.1.1 Fuentes de datos en la actualidad

Existen principalmente tres tipos de situaciones que generan datos en la actualidad:

- La interacción entre humanos a través de un sistema informático que registra información mientras se produce la interacción. Ejemplos claros son el correo electrónico, los foros de Internet o las redes sociales, donde los datos los generamos los humanos al interactuar entre nosotros utilizando dichos medios, y los datos son almacenados y posteriormente procesados por máquinas.
- La interacción entre un humano y una máquina. El ejemplo más claro es la navegación en Internet: los servidores web generan logs con información sobre el proceso de navegación. Lo mismo ocurre al efectuar compras en alguna plataforma web de comercio electrónico o en banca online, donde cada una de nuestras transacciones queda registrada y será procesada después con el objetivo de estudiar nuestro comportamiento y ofrecernos productos mejores y más personalizados.
- La interacción entre máquinas. Varias máquinas intercambian información y la almacenan, con el objetivo de ser procesados por otras máquinas. Algunos ejemplos son sistemas de monitorización, en los que un sistema de sensores proporcionan la información que reciben a otras máquinas para que realicen algún procesado sobre ella.

Algunas cifras que dan idea de la cantidad de datos generados gracias a Internet se resumen en la siguiente imagen.


THE INTERNET IN 2023 EVERY MINUTE
22,831
visits to ChatGPT
241.2M
emails sent
18.8M
text messages sent
2.4M
Google searches
694,000
video hours viewed
347,222
tweets
3.47M
snaps created
10.4M
viewing minutes
11,035
fake accounts removed
6.3M
total Zoom meeting minutes
34,247
Slack messages
11,834
chats on Microsoft Teams
6.94M
emoji sent
3.02M
photos created with smartphones
271,309
IOS & Android app downloads
60
SECONDS
Created by: eDiscovery Today & LTMG

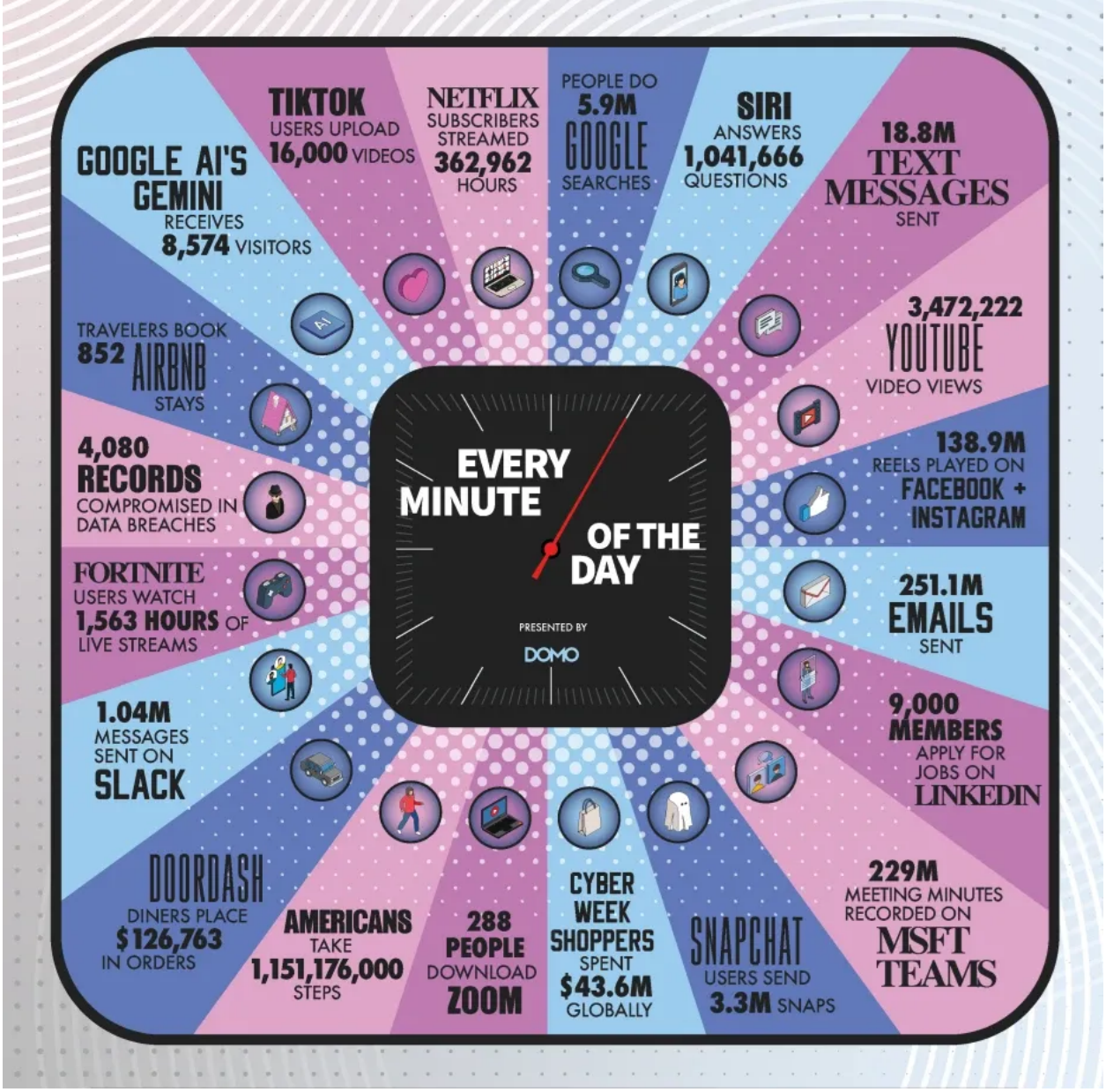


Fig 2. Eventos que suceden en Internet durante un minuto en 2023 y 2024

1.2 La Transformación Digital

La conclusión global a la que podemos llegar **no** es que el mundo esté cambiando: **el mundo ya ha cambiado**, y lo podemos confirmar si examinamos hechos como los siguientes:

- La empresa que transporta a más personas en el mundo es Uber, que tiene 0 coches físicos.
- La empresa que más habitaciones reserva es Airbnb, que tiene 0 hoteles físicos.
- La empresa que más música vende es Spotify, que tiene 0 estudios de grabación.
- La empresa que vende más películas es Netflix, que tiene 0 estudios.

Estamos viendo que se producen *más interacciones digitales que físicas* entre las personas y las compañías que les dan servicio. Estas interacciones están generando datos masivos muy valiosos que hablan del comportamiento de los clientes, y permiten anticipar qué es lo que estos necesitan y van a demandar. De hecho, estamos evolucionando más rápido que las propias compañías, hasta el punto que se ha abierto una brecha entre las empresas físicas y las empresas digitales. Con el objetivo de llenar este espacio, surge la Transformación Digital.

Estamos en la Era del Cliente, marcada por una sociedad interconectada en la que hemos reinventado cómo vivimos y cómo nos relacionamos. Esta conectividad digital hace que las noticias vuelen más que nunca, para bien y para mal. Un cliente insatisfecho tiende a compartir su experiencia mucho más que un cliente satisfecho, con el consiguiente perjuicio a la imagen de una marca. Además, los clientes que cambian de proveedor en un sector suelen hacerlo por una mala experiencia como clientes, más que estrictamente por un mal servicio, y lo más frecuente es que abandonen a su proveedor sin avisar ni quejarse explícitamente.

La Transformación Digital persigue tres objetivos:

- Centrarse en el cliente: pensar continuamente en lo que necesita y en mejorar (personalizar) su experiencia y sus interacciones con la compañía. Esto requiere *ingestar y analizar grandes cantidades de datos sobre su comportamiento*.
- Centrarse en canales digitales, especialmente dispositivos móviles, puesto que las interacciones digitales son las que generan mayor cantidad de datos analizables.
- Decisiones dirigidas por los datos (*data-driven*), para lo cual es necesaria la Ciencia de (Grandes) Datos (*Big Data Science*).

1.3 Definición de *Tecnologías Big Data*

Para acometer estos objetivos, la mayor parte de las tecnologías existentes hasta hace pocos años (principios de los años 2000) no eran suficientes. El motivo es la necesidad de procesar, almacenar y analizar datos con ciertas características especiales (las denominadas tres V del Big Data).

Note: Según la definición clásica, un proyecto es Big Data cuando implica alguna de las tres V's: volumen, velocidad o variedad.

Estos conceptos pueden definirse como:

- Volumen: cantidades de datos lo suficientemente grandes como para no poderse procesar con tecnologías tradicionales.
- Velocidad: flujos de datos que van llegando en tiempo real y tienen que procesarse de manera continua según se van recibiendo.
- Variedad: datos de fuentes diversas, estructuradas y no estructuradas (sean BBDD relacionales, no relacionales, datos de imágenes, sonido, etc) que tienen que ser manejados y cruzados de manera conjunta.

Una definición más ajustada y realista de un proyecto con tecnologías Big Data sería:

Note: Un proyecto es Big Data cuando la mejor manera de resolverlo (más rápida, eficiente, sencilla) implica utilizar tecnologías Big Data

---

Podemos definir *Big Data* como

Note: Conjunto de **tecnologías** y **arquitecturas** para almacenar, mover, acceder y procesar (incluido analizar) datos que eran muy difíciles o imposibles de manejar con tecnologías tradicionales

Las causas de esta imposibilidad pueden ser:

- Cantidades ingentes de datos inimaginables hace unos años
- Datos de fuentes diversas, heterogéneas, poco estructuradas como documentos o imágenes/sonido, que aun así necesitamos almacenar y consultar (NoSQL)
- Datos dinámicos recibidos y procesados según llegan (flujos de datos o *streams*)

Nótese que en la definición anterior se omiten de manera deliberada palabras como *algoritmo, inteligencia, Ciencia de Datos* o cualquier referencia a *qué hacer o cómo analizar y explotar esos datos*. Las tecnologías Big Data son tecnologías que permiten aplicar a datos masivos técnicas que ya existían, pero son tecnologías y no técnicas en sí mismas. Las técnicas de análisis pertenecen al ámbito de la Estadística, las Matemáticas, las Ciencias de la Computación y la Inteligencia Artificial. La mayoría de estas técnicas y algoritmos han existido desde mucho antes, algunas desde mediados del siglo XX. La diferencia es que ahora pueden aplicarse a cantidades de datos mucho mayores, de naturaleza heterogénea y se pueden obtener resultados en menos tiempo.

Por supuesto, las tecnologías Big Data no tienen nada que ver con mitos como:







Fig. 3. Todo lo que NO es Big Data. Falsos mitos extendidos entre la población.

1.4 Origen de las tecnologías de procesamiento distribuido

La primera empresa que fue consciente del aumento de los datos que se estaban generando en Internet fue Google, debido a que su buscador debe ser capaz de indexar las webs nuevas para que puedan ser encontradas. En 2003 publicaron un artículo de investigación que se hizo mundialmente famoso. En él explicaban el sistema de archivos distribuido *Google File System* (GFS), donde presentaban por primera vez la idea de utilizar ordenadores convencionales conectados entre sí (formando un *cluster*) para poder almacenar archivos que ocupaban más que un solo disco duro. A esto se le denomina ***commodity hardware***: máquinas no especialmente potentes, similares a las que tienen los usuarios domésticos, pueden conectarse entre sí para trabajar conjuntamente para resolver tareas de mayor envergadura. GFS fue la base del sistema de archivos distribuido HDFS que veremos en temas posteriores.

En 2004, Google publicó otro artículo que también se popularizó rápidamente, donde explicaban un modelo de programación (MapReduce) aplicable a un cluster de ordenadores para procesar en paralelo archivos almacenados en su sistema de archivos GFS. Google también publicó una biblioteca de programación de código abierto implementando dicho paradigma. Su principal punto fuerte era la abstracción de todos los detalles de hardware, redes y comunicación entre nodos para que el usuario pudiera centrarse en el desarrollo de la lógica de la aplicación distribuida de manera sencilla. Durante muchos años, MapReduce ha sido el estándar de desarrollo de software Big Data a nivel comercial.

En 2002, el ingeniero Doug Cutting desarrolló una conocida herramienta de búsqueda llamada Nutch. Cuando fue contratado por Yahoo en 2006, lideró el proyecto Hadoop, que se independizó de Nutch e incorporó las ideas de los artículos de Google, creando así un ecosistema de herramientas y bibliotecas de programación para ejecutar MapReduce en clusters de ordenadores (principalmente en lenguaje Java). Yahoo donó Hadoop a la Apache Software Foundation en 2008 para convertirlo en un proyecto open-source. La siguiente línea temporal resume los acontecimientos de este período.

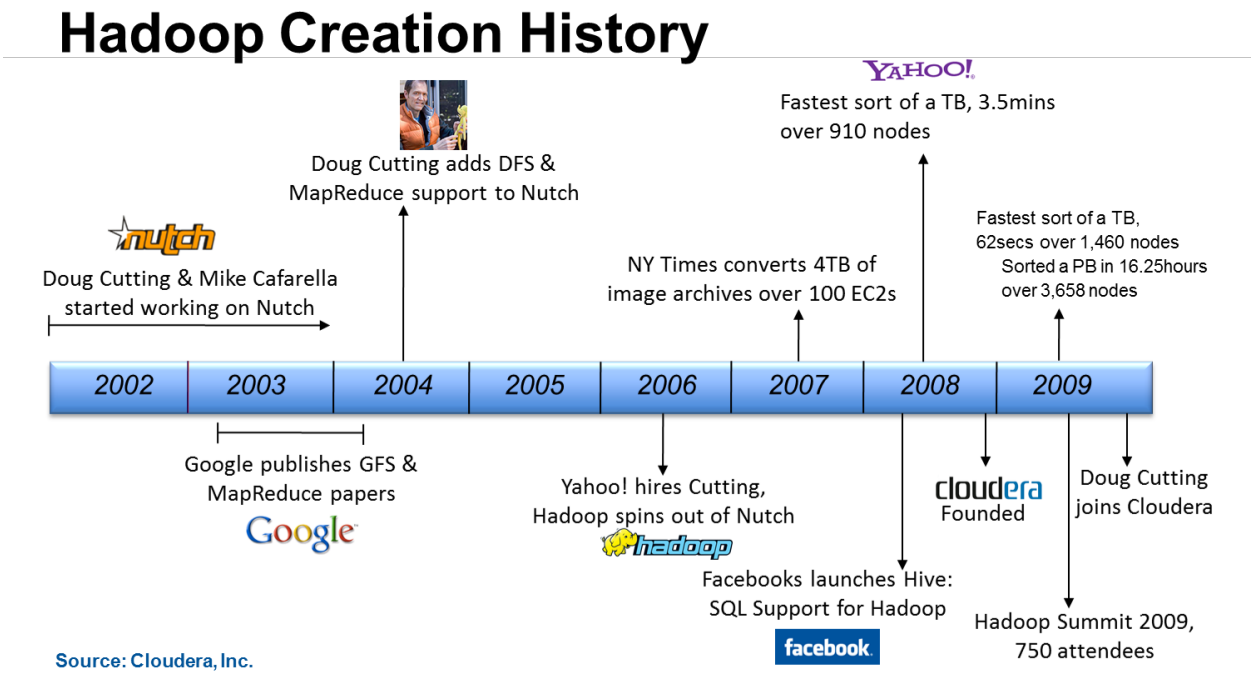


Fig. 4. Origen y evolución de Hadoop en los primeros años



El logotipo de Hadoop, un elefante amarillo, hace referencia a un elefante de peluche llamado Hadoop de una de las hijas de Doug Cutting.

Años después, y motivado por las deficiencias de Hadoop en ciertas tareas, un investigador llamado Matei Zaharia (fundador y actualmente Chief Technology Officer -CTO- de la empresa Databricks) creó una nueva tecnología open-source de procesamiento distribuido llamada Apache Spark durante su tesis doctoral en Berkeley. Estudiaremos Spark en pro-

fundidad en temas posteriores. Como adelanto, MapReduce ha sido reemplazado por Spark casi en su totalidad. Las herramientas que lo utilizaban como motor de ejecución han sobrevivido gracias a que el motor de ejecución es una pieza intercambiable en muchas de ellas, y han sabido adaptarlo a Spark.

1.5 El ecosistema Hadoop

La idea básica que hay tras las tecnologías de procesamiento distribuido es la siguiente:

Note: Es posible procesar grandes cantidades de datos de forma distribuida entre varias máquinas interconectadas (cluster), cada una no necesariamente muy potente (commodity hardware). Si se necesita más potencia de cálculo o más capacidad de almacenamiento, basta con añadir más máquinas al cluster

---

Siguiendo esta filosofía, y teniendo como punto de partida el sistema de archivos distribuido GFS (que en Hadoop se transformó en HDFS – Hadoop Distributed File System) y el paradigma MapReduce, se crearon un conjunto de herramientas distribuidas, cada una con un propósito específico pero todas interoperables entre sí. Muchas de ellas están en desuso porque MapReduce ha sido reemplazado por Spark.





Fig. 5. El cluster Mare Nostrum 4, en el Barcelona Supercomputing Center (CSIC). Cada armario se denomina rack, y cada bandeja es un ordenador (nodo).

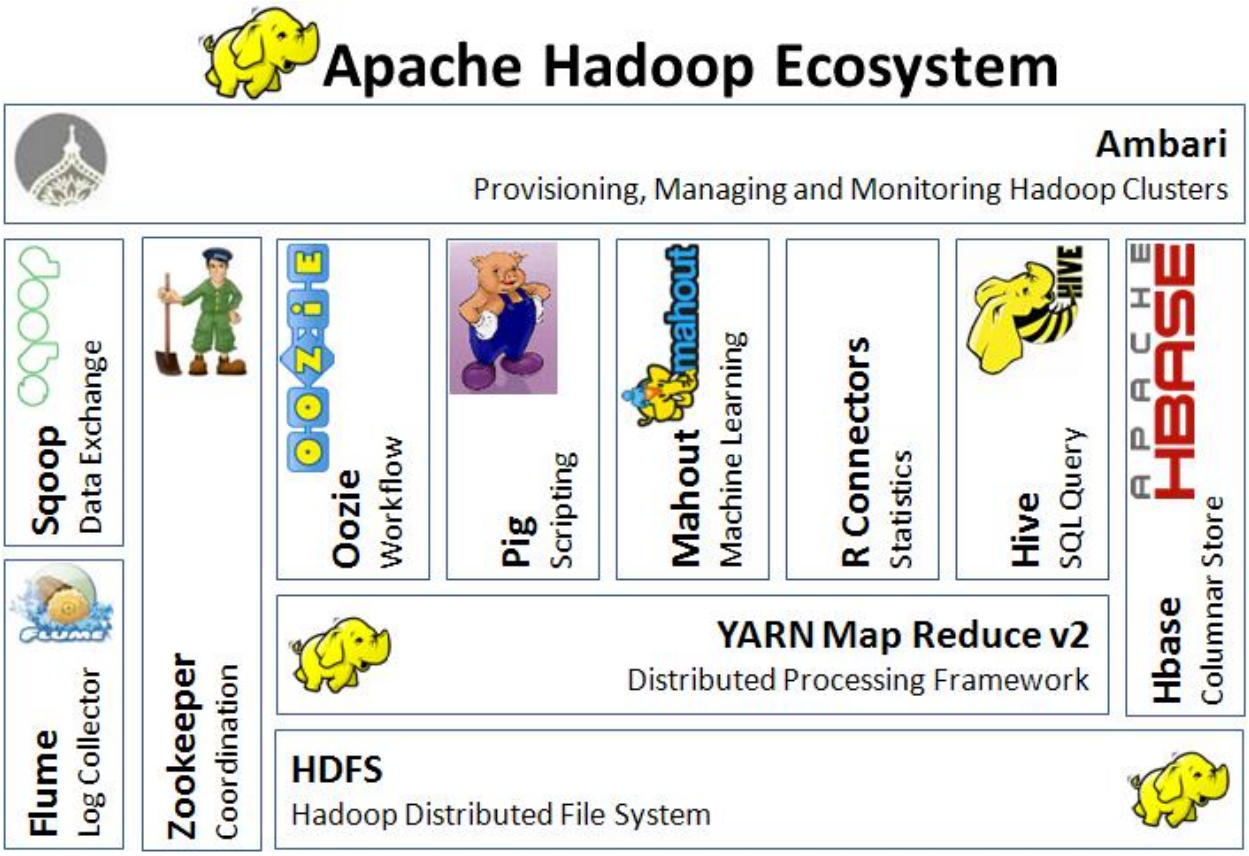


Fig. 6. El ecosistema open-source Hadoop para procesamiento distribuido

Sin ánimo de ser exhaustivos, damos una breve descripción de cada una:

- HDFS: Sistema de archivos distribuido que estudiaremos en el tema siguiente
- MapReduce: paradigma de programación para un cluster de ordenadores (forma de estructurar programas y también biblioteca de programación que se ejecuta sobre el cluster).
- Flume: herramienta para tratamiento de logs.
- Sqoop: herramienta para migración de grandes cantidades de datos desde BBDD convencionales a HDFS.
- Zookeeper: coordinador.
- Oozie: herramienta para planificación y ejecución de flujos de datos.
- Pig: herramienta para programar flujos de datos con sintaxis similar a SQL pero de mayor nivel de granularidad.
- Mahout: biblioteca de algoritmos de Machine Learning. Originalmente estaba programada con MapReduce, por lo que tenía un rendimiento pobre, pero actualmente soporta otros backend como Spark.
- R Connectors: herramientas para conectar MapReduce con el lenguaje de programación R. En desuso al igual que
- Hive: herramienta para manejar datos almacenados en HDFS utilizando lenguaje SQL. Originalmente utilizaba MapReduce como motor de ejecución. Actual-mente soporta Spark y Apache Tez.
- HBase: base de datos NoSQL de tipo columnar, que permite entre otras cosas tener registros (filas) de longitud y número de campos variable.

La herramienta que estudiaremos en el curso es Apache Spark, un motor de procesamiento distribuido y bibliotecas de programación distribuida de propósito general, que opera siempre en la memoria principal (RAM) de los nodos del

cluster. Abstrae todos los detalles de bajo nivel relativos a comunicación entre máquinas y redes. Desde hace varios años ha reemplazado totalmente a MapReduce al ser mucho más rápido.

Mencionamos también como antecedente y solo de forma anecdótica la tecnología MapReduce, que se define como una manera de programar y también, un motor de ejecución de tareas que corren de forma distribuida en los diferentes nodos del clúster Hadoop. Al igual que en Spark, la forma en la que los datos se distribuyen en diferentes sub-tareas y cómo estas se asignan a cada máquina resulta transparente para el desarrollador, y de hecho fue la primera tecnología que logró esto. Precedió a Spark y no la trataremos durante las clases al estar obsoleta, pero conviene saber de dónde venimos para entender mejor las ventajas de Spark.

1.5.1 Distribuciones de Hadoop

Al estar constituido Hadoop por un conjunto de herramientas diferentes, cada una requiere su propia instalación y configuración en el cluster para poder operar con otras ya instaladas. Este proceso era tedioso y requería bastantes conocimientos. Con el fin de simplificar esta tarea, surgieron las *distribuciones de Hadoop*, que eran conjuntos de herramientas del ecosistema Hadoop empaquetadas juntas en versiones compatibles y perfectamente interoperables entre ellas, distribuidas con un único software donde no hay necesidad de instalarlas por separado.

Empresas como Cloudera, Hortonworks (estas dos fueron competencia mutua durante mucho tiempo hasta que acabaron por fusionarse en 2018) o MapR nacieron para crear distribuciones de Hadoop y añadirles, en algunos casos, herramientas propietarias totalmente nuevas que no pertenecían al ecosistema Hadoop, o incluso modificaciones propias del código fuente original de las herramientas de Hadoop para solucionar fallos o añadir características avanzadas. Todas las distribuciones de Hadoop de estas empresas tienen versiones open-source y de pago. La siguiente tabla compara sus características:

	Cloudera	**Hortonworks**	**MapR**
Componentes	Apache modificados y añadidos	Sólo Apache oficiales	Apache y añadidos
Versiones	Open-source (CDH) y de pago	Sólo 100 % open-source	Open-source y de pago
Sistema operativo	Linux (Windows: VM Ware)	Linux y Windows	Linux (Windows: VM Ware)
Año creación	2008	2011	2009
Observaciones	Es la más extendida.	Única para Windows	La más rápida y fácil de instalar
	Certificación muy popular	Única 100 % open source	

Tabla 1. Comparación entre las principales distribuciones de Hadoop en la actualidad

Cloudera, por ejemplo, incluye versiones de los componentes que son anteriores a la última versión oficial disponible, además de modificaciones del código oficial que solucionan incidencias reportadas pero aún no resueltas en el código oficial de las Apache. Además, incluye en su distribución ciertas herramientas propias como por ejemplo Clouder Data Science Workbench (CDSW).

1.6 Bibliografía recomendada

1. Tom White: *Hadoop, The Definitive Guide*. O'Reilly, 2015. Capítulo 1.
2. http://static.googleusercontent.com/media/research.google.com/es//archive/gfs-sosp2003.pdf
3. http://static.googleusercontent.com/media/research.google.com/es//archive/mapreduce-osdi04.pdf

CHAPTER

TWO

CAPÍTULO 2: APACHE SPARK I (CORE Y SQL)

En el tema anterior vimos cómo MapReduce permite procesar los datos almacenados de manera distribuida en el sistema de archivos (por ejemplo HDFS) de un cluster de ordenadores. MapReduce permite resolver todo tipo de problemas, pese a que no siempre es sencillo pensar la solución en términos de operaciones *map* y *reduce* encadenadas. Por ello se dice que es de propósito general, a diferencia de, por ejemplo, el lenguaje SQL que está orientado específicamente a realizar consultas sobre los datos. Implementar en SQL algoritmos tales como un método de ordenación, o el algoritmo de Dijkstra para encontrar caminos mínimos en un grafo no sería posible.

Los siguientes enlaces nos llevan a cada sección.

- *Introducción*
- *Arquitectura de Spark*
- *RDD: Resilient Distributed Datasets*
- *DataFrames y Spark SQL*

2.1 Introducción

Antes de empezar con la tecnología de Apache Spark, que es el tema principal del curso, es necesario hacer un comentario sobre cómo nacieron las tecnologías Big Data. Repasaremos el problema del procesamiento masivo, que dio origen a Spark.

2.1.1 Antecedentes de Spark: programación Distribuida y MapReduce

Una vez que Google tenía resuelto el problema del almacenamiento de ficheros masivos en el sistema de ficheros distribuido Google File System (GFS), en 2004 publicaron un nuevo artículo sobre cómo aprovechar los datanodes para procesar estos ficheros que estaban almacenados de forma distribuida particionados en bloques. El artículo describe un paradigma de programación llamado MapReduce, que consiste en una manera abstracta de abordar los problemas para que puedan ser implementados y ejecutados sobre un cluster de ordenadores. Junto con el artículo, también liberaron una biblioteca de programación en lenguaje Java que implementaba este paradigma. Podemos definir MapReduce como

> Modelo abstracto de programación general, e implementación del modelo como bibliotecas de programación para procesamiento paralelo y distribuido de grandes datasets, inspirado en la técnica divide y vencerás.

Al mismo tiempo, en el año 2002, el ingeniero Doug Cutting desarrolló una conocida herramienta de búsqueda llamada Nutch. Cuando fue contratado por Yahoo en 2006, Doug Cutting lideró el proyecto Hadoop, que se independizó de Nutch e incorporó las ideas de los artículos de Google, creando así un ecosistema de herramientas y bibliotecas de programación para ejecutar MapReduce en clusters de ordenadores (principalmente en lenguaje Java). Yahoo donó Hadoop a la Apache Software Foundation en 2008 para convertirlo en un proyecto open-source. El logotipo de Hadoop, un elefante amarillo, hace referencia a un elefante de peluche llamado Hadoop de una de las hijas de Doug Cutting.

La gran ventaja que proporciona tanto el modelo MapReduce como la implementación en forma de biblioteca, es que abstrae al programador de todos los detalles relativos al hardware, redes y comunicación entre los nodos, para que pueda centrarse solamente en el desarrollo de software para solucionar su problema.

El punto de partida son archivos muy grandes almacenados (por bloques) en HDFS (en aquel momento aún era GFS). ¿Podríamos aprovechar las CPUs de los datanodes del cluster para procesar en paralelo (simultáneamente) los bloques de un archivo? No queremos mover datos (el tráfico de red es muy lento): llevamos el cómputo (el código fuente de nuestro programa) donde están almacenados los datos (es decir, a los datanodes, en lugar de al revés, como ocurría en el modelo tradicional de aplicación). Las CPUs de los datanodes van a procesar (preferentemente) los datos que hay en ese datanode.

En el paradigma MapReduce, el usuario sólo necesita escribir dos funciones (enfoque divide y vencerás):

- Mapper: función escrita por el usuario e invocada por el framework en paralelo (simultáneamente) sobre cada bloque de datos de entrada (que generalmente coinciden con un bloque de HDFS), generando así resultados intermedios que están siempre en forma de (clave, valor) y son escritos también al disco local.
- Reducer: función del usuario invocada por el framework en paralelo para cada clave distinta, pasándole todos los valores que comparten esa clave.

Inconvenientes de MapReduce

MapReduce permite procesar los datos almacenados de manera distribuida en el sistema de archivos (por ejemplo HDFS) de un cluster de ordenadores. Permite resolver todo tipo de problemas, pese a que no siempre es sencillo pensar la solución en términos de operaciones map y reduce encadenadas. Por ello se dice que es de propósito general, a diferencia de, por ejemplo, el lenguaje SQL que está orientado específicamente a realizar consultas sobre los datos. Implementar en SQL algoritmos tales como un método de ordenación, o el algoritmo de Dijkstra para encontrar caminos mínimos en un grafo no sería posible. Sin embargo, MapReduce presenta varios inconvenientes, algunos muy serios:

- El resultado de la fase map (tuplas) se escribe en el disco duro de cada nodo, como resultado intermedio. Los accesos a un disco duro son aproximadamente un orden de magnitud (es decir, diez veces) más lentos que los accesos a la memoria principal (RAM) de cada nodo, por lo que estamos penalizando el rendimiento debido a cómo está estructurado el propio framework.
- Después de la fase map, hay tráfico de red obligatoriamente (movimiento de datos, conocido como shuffle). De hecho, el movimiento de datos forma parte como una etapa obligada en el framework de MapReduce. En cualquier aplicación distribuida, el movimiento de datos de un nodo a otro constituye un cuello de botella y es una operación que debe evitarse si es posible.
- Por otro lado y relacionado con el punto anterior, para mover datos se necesita, en primer lugar, escribirlos temporalmente en el disco duro de la máquina origen, enviarlos por la red y escribirlos temporalmente en el disco duro de la máquina destino (el shuffle siempre va de disco duro a disco duro) para, finalmente, pasarlos a la memoria principal de dicho nodo.
- Estos dos inconvenientes se acentúan especialmente cuando el algoritmo que queremos implementar sobre un cluster es de tipo iterativo, es decir, requiere varias pasadas sobre los mismos datos para ir convergiendo. Este tipo de procesamiento es típico en algoritmos de Machine Learning, que es uno de los que más se puede beneficiar del procesamiento de grandes conjuntos de datos para extraer conocimiento. Se comprobó que sus implementaciones con MapReduce no eran eficientes debido a la propia concepción del framework.
- Finalmente, enfocar cualquier problema en términos de operaciones map y reduce encadenadas no siempre es fácil ni intuitivo para un desarrollador, y la solución resultante puede ser difícil de mantener si no está bien documentada.

En el año 2009 surgió en el grupo de investigación AMPLab, en la Univ. California en Berkeley, EEUU, una nueva propuesta para paliar estas deficiencias, que poco a poco ha ido imponiéndose hasta reemplazar completamente a Hadoop (más concretamente, a MapReduce).

Note: Apache Spark es un motor unificado de cálculo en memoria y un conjunto de bibliotecas para procesamiento paralelo y distribuido de datos en clusters de ordenadores

Analizando la definición:

- Es un *motor* de cálculo, esto es, un framework de propósito general orientado a resolver cualquier tipo de problema y con el que se puede implementar cualquier algoritmo.
- *Unificado*: el motor de cálculo es único e independiente de cómo utilicemos Spark:
  - Desde la API de DataFrames (para los lenguajes R, python, Java y Scala)
  - Desde herramientas externas que lanzan consultas SQL contra Spark (Hive, herramientas de Business Intelligence conectadas a Spark como Tableau, PowerBI, etc)
  - Desde una instrucción de la API de programación que recibe una consulta SQL como string, tan compleja como sea necesario
  - …todo se traduce a un grafo de tareas al que Spark aplica optimizaciones de código automáticamente, y por tanto todos (API, aplicaciones BI, SQL, etc) se benefician de ellas
- *En memoria*: todos los cálculos se llevan a cabo en memoria, y solo se escriben resultados a disco (parciales o finales) cuando el usuario lo indica explícitamente (operación de guardado) o cuando la operación indicada por el usuario requiere forzosamente realizar movimiento de datos entre nodos (*shuffle,* el cual se lleva a cabo del disco duro del emisor al disco duro del nodo receptor). Además, el movimiento de datos solo se produce cuando es irremediable, y no de manera obligatoria. Esto consigue un rendimiento hasta 100 veces superior que MapReduce en tareas iterativas (varias pasadas sobre los mismos datos, como tareas de machine learning). La medición de tiempos en una regresión logística arrojó un factor de mejora de x100.

2.1.2 Breve historia de Spark

La Fig. 1 muestra un gráfico con la evolución temporal de Spark. Fue concebido como parte de la tesis doctoral de Matei Zaharia, un investigador en el grupo AMPLab de UC Berkeley. Tras la publicación en 2010 del primer artículo sobre Spark, y la liberación desde el principio del código fuente, se vio su potencia y la ganancia que se podría llegar a conseguir sustituyendo el procesamiento anterior en MapReduce por el procesamiento con este nuevo framework.

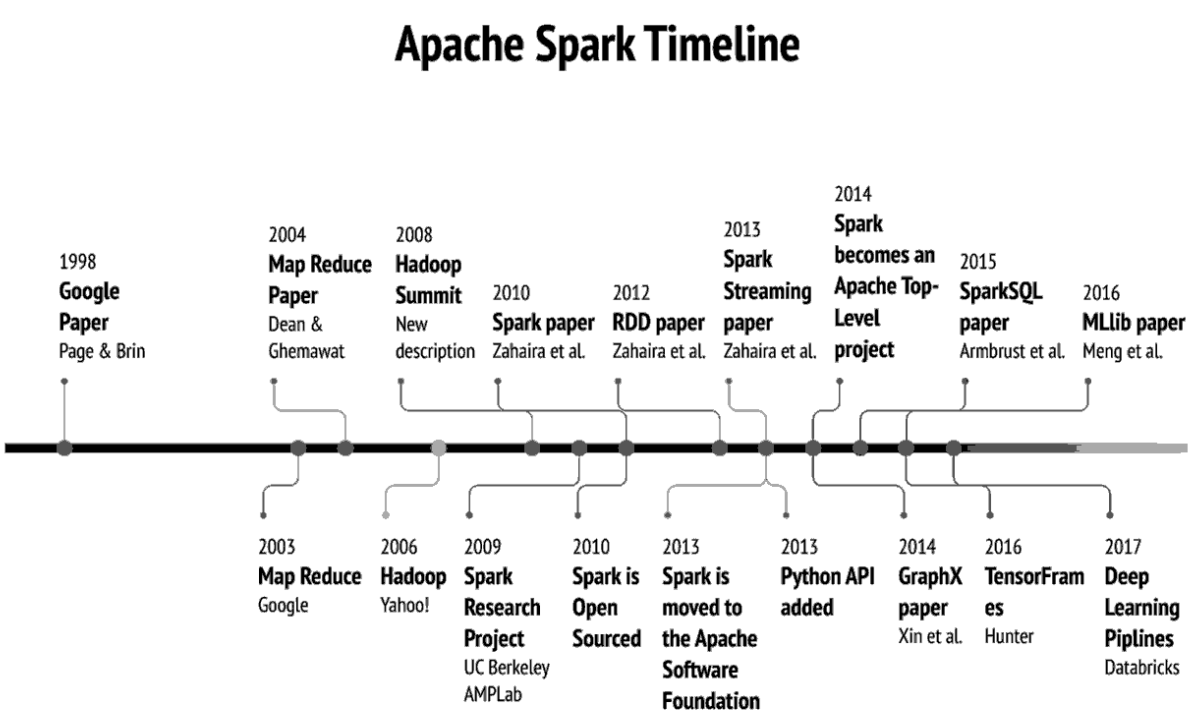


Fig. 1. Evolución de Apache Spark a lo largo del tiempo

Poco después del nacimiento de Spark, su creador Matehi Zaharia fundó la empresa Databricks, dedicada a dar servicios en torno a Spark, tales como como su propia plataforma Big Data y servicios cloud. Poco a poco se fue completando Spark con módulos específicos que aprovechaban esta tecnología para ofrecer API de mayor nivel de abstracción para tareas concretas, como procesamiento de datos en *streaming*, o la utilización de estructuras de datos distribuidas de más alto nivel y más similares a las tablas de una base de datos relacional, además de un módulo específico que implementa modelos de Machine Learning que pueden entrenarse con datos distribuidos.

2.1.3 Apéndice: el formato de fichero Parquet

Esta sección no está relacionada con Spark como tal, pero explica un formato de fichero muy frecuente para almacenar grandes volúmenes de datos, y que se utiliza muchísimo como fuente para leer datos masivos que luego se procesan utilizando algún framework distribuido, como por ejemplo Spark.

Apache Parquet es un formato de archivo nacido en 2013, de tipo columnar y comprimido (binario). Es el formato más utilizado para almacenar datos estructurados en HDFS (tablas o similar, incluyendo tipos complejos y anidados en las columnas). Es muy frecuente su uso en conjunción con Apache Hive (tema siguiente). Parquet utiliza la técnica conocida como record-shredding para almacenar tipos de datos complejos y anidados (que un CSV no es capaz de representar). Sus principales características son:

- Almacena el esquema junto a los datos. También máx/mín por columna (resúmenes)
- Los valores de cada columna se guardan físicamente juntos (almacenamiento columnar). Beneficios:
  - Compresión de datos por cada columna para ahorrar espacio

  - Se pueden aplicar técnicas de compresión diferentes, específicas según el tipo de dato que almacena cada columna
  - Las consultas con filtrados sobre columnas concretas no necesitan leer toda la fila, mejorando el rendimiento
- Preparado para añadir más esquemas de codificación que se desarrollen en el futuro
- No requiere especificar separador de columnas, ni si se incluyen o no nombres de columna (siempre se incluyen por defecto)

Es con diferencia el formato más ampliamente utilizado para procesamiento en batch, tanto para datos de entrada como para guardar resultados, con datos estructurados, ya que puede ocupar entre 7 y 10 veces menos que un fichero CSV que almacene la misma información.

2.2 Arquitectura de Spark

2.2.1 Componentes de Spark

Apache Spark consiste en una API (bibliotecas) orientada al programador, **mucho** más intuitiva que MapReduce. Al igual que éste, abstrae todos los detalles de comunicación de red y hardware, pero además, opera de manera similar a las consultas SQL tradicionales como si los datos fuesen tablas (distribuidas). Esto la hace fácil de usar y de aprender.

Spark ofrece APIs para cuatro lenguajes:

- Java: muy tediosa de usar (sintaxis muy verbosa) pero útil para aplicaciones legacy existentes.
- Scala: es la API más utilizada para aplicaciones en producción. Scala es un lenguaje funcional que se compila sobre la JVM (la compilación genera bytecode de Java), muy compacto y cómodo de utilizar con Spark. La mayor parte del código del propio Spark es lenguaje Scala.
- Python (paquete de Python *pyspark*)*:* API casi idéntica a la de Scala, salvo peculiaridades del lenguaje. En realidad es un wrapper (capa adicional) sobre la implementación en Scala, que es la verdaderamente distribuida. No obstante, Spark también permite ejecutar código python en los nodos del cluster.
- R (paquete de R *SparkR*): es una API muy diferente al resto, más cercana a los comandos típicos de R. Últimamente, el paquete *sparklyr* (creado por la empresa RStudio) es más útil, ofrece una mejor integración y más avanzada, y su sintaxis es más intuitiva.

La Fig. 2, tomada de [1], muestra los principales componentes de Spark.

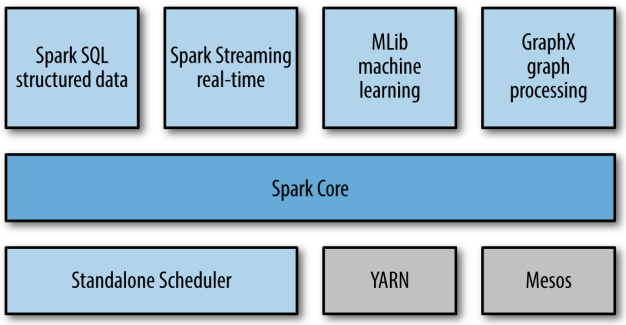


Fig 2. Módulos que componen Apache Spark

De estos, el principal es el módulo *Spark core*. En él se encuentran las abstracciones fundamentales de Spark, tales como el RDD (Resilient Distributed Dataset), del que hablaremos más adelante, y las operaciones que se puede llevar a cabo con él. En última instancia, todas las operaciones que se pueden hacer con Spark, independientemente del módulo utilizado o la API donde se encuentren, se traducen a operaciones sobre RDDs, que son las únicas que Spark es capaz de ejecutar.

Los tres módulos inferiores representan tres gestores de recursos de un cluster sobre los que puede ejecutarse Spark. Un gestor de recursos se encarga de asignar máquinas, CPUs y memoria principal a Spark, de forma que disponga de un determinado número de nodos y en cada uno existan suficientes recursos para ejecutar la aplicación que hemos implementado con Spark. YARN y Mesos no forman parte de Spark pero son compatibles con Spark. El Standalone Scheduler es el gestor de recursos más sencillo, y viene incluido con Spark. No permite un control de grano muy fino de los recursos pero es suficiente para casos de uso simples.

El resto de módulos cumplen las siguientes funciones:

- **Spark SQL** es una API con una serie de funciones para manejar tablas de datos distribuidas, estructuradas en columnas con nombre y tipo, denominadas DataFrames. Un DataFrame proporcionan un nivel de abstracción adicional sobre un RDD, al cual recubre. Este módulo incluye también una función para ejecutar sentencias SQL sobre las mismas estructuras de datos distribuidas. Estas sentencias son analizadas y convertidas en un plan lógico que en última instancia se traduce a operaciones sobre RDDs al igual que cualquier otra operación de Spark.
- **Spark Streaming** es el módulo para operar de manera distribuida sobre datos en tiempo real según los vamos recibiendo (*stream* de datos). Actualmente (a partir de la versión Spark 2.0) este módulo ha sido reemplazado por *Spark Structured Streaming*, que simplifica las operaciones con flujos de datos y será el que estudiaremos en este curso.
- **Spark MLlib** contiene implementaciones distribuidas de algoritmos de Machine Learning. Aunque el módulo conserva este nombre, el paquete actual de la API se llama Spark ML y simplifica el uso de estos algoritmos puesto que actúan sobre DataFrames.
- **Spark GraphX** es el módulo de procesamiento de grafos, representados mediante RDDs. Contiene algunos algoritmos de camino mínimo y similares. Actualmente este módulo ha quedado obsoleto, y ha sido reemplazado *de facto* por el paquete GraphFrames, que representa un grafo como una pareja de DataFrames con los nodos y los arcos respectivamente. GraphFrames se desarrolló como una biblioteca externa a Spark y será la biblioteca que estudiaremos en el curso.

2.2.2 Arquitectura de una aplicación con Spark

La siguiente figura muestra la arquitectura de una aplicación que utiliza Spark al ejecutarse en un cluster.

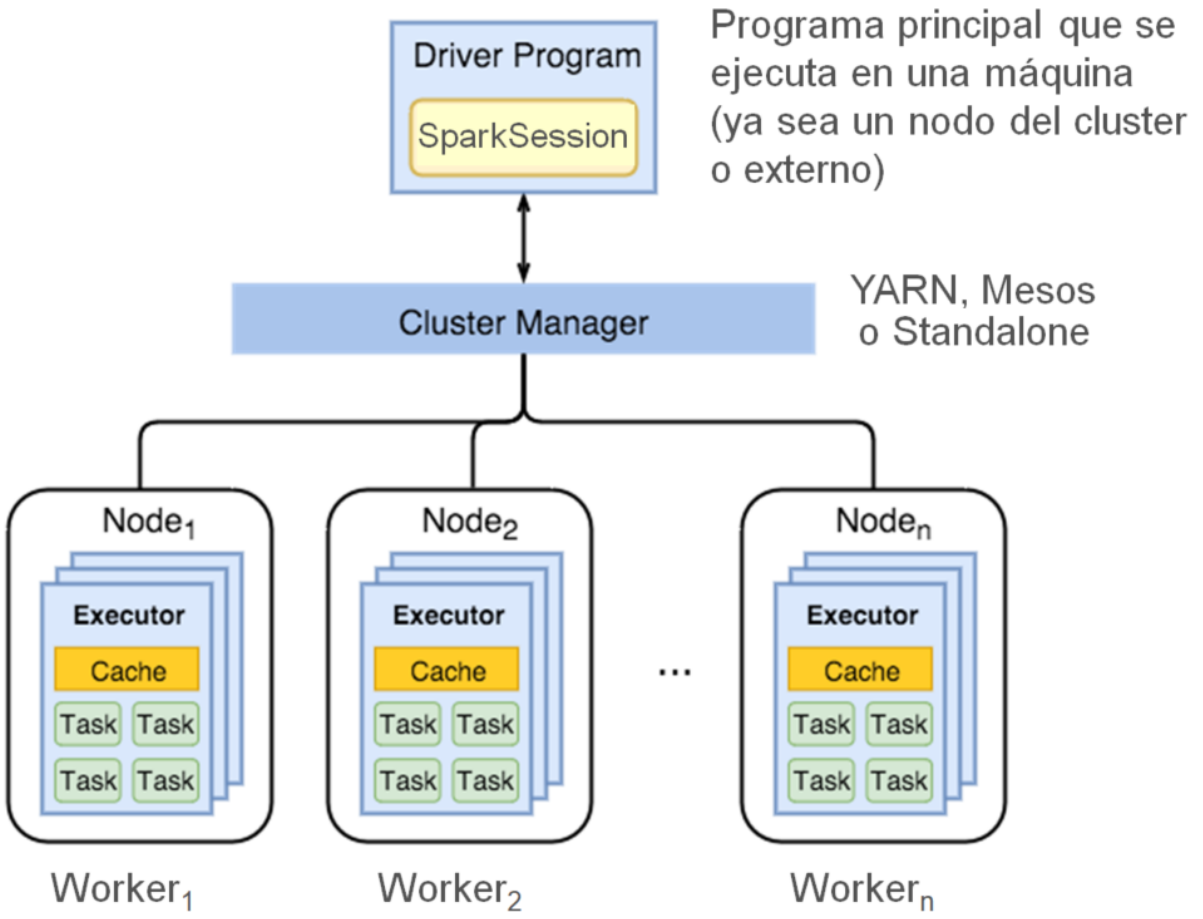


Fig. 3. Arquitectura de una aplicación en Spark

Cuando escribimos una aplicación en Spark, en realidad estamos escribiendo una aplicación **secuencial** (no paralela) utilizando la biblioteca de Spark para el lenguaje en el que estemos programando (Java, Scala, Python o R). Cuando ejecutamos el código de este programa, el proceso en el cual se ejecuta se denomina *Driver*, y se ejecuta sobre una máquina concreta, que puede incluso ser externa al cluster. Por ejemplo, podríamos escribir un programa en Spark y ejecutarlo en nuestro ordenador portátil, y desde aquí, el programa Driver podría conectarse a un cluster de Spark remoto, lo cual se denomina ejecución en modo *client* (cliente). También existe la opción de que el programa Driver sea enviado al cluster y se ejecute dentro de uno de los nodos del cluster, lo cual se denomina ejecución en modo cluster.

En el transcurso de la ejecución, normalmente necesitaremos crear un objeto que solemos guardar en una variable llamada *spark*, de la clase SparkSession de Spark. En el momento de crear este objeto, hay que indicar dónde (en qué dirección IP y en qué puerto) existe un cluster de Spark configurado o, más concretamente, dónde existe un gestor de cluster que nos pueda dar acceso a los recursos (nodos, memoria, CPUs) de un cluster. Ciertas aplicaciones, como por ejemplo ciertos *kernel* especiales para Jupyter Notebook, o la línea de comandos de Spark tanto en Scala (spark-shell) como en python (pyspark-shell) ya crean un objeto sparkSession por nosotros al arrancar la aplicación.

De esta manera, la sparkSession establece comunicación con el gestor de cluster. A partir de este momento, podemos utilizar dichos recursos *para las sentencias que requieran ejecución distribuida*.

Es importante recalcar esto último: la ejecución de un programa en Spark es secuencial, en una sola máquina, tal como sería cualquier otro programa en el lenguaje elegido, **excepto cuando el flujo de programa llega a ciertas funciones específicas de Spark que desencadenen ejecución distribuida**, que son la mayoría (pero no todas) las que forman la API de Spark. Muchas de estas funciones se aplican sobre el objeto sparkSession, y otras se aplican sobre estructuras de datos distribuidas que hayamos ido creando en el programa, como por ejemplo RDDs o DataFrames de Spark.

Configuración del objeto SparkSession

En el momento de crear el objeto sparkSession, hemos indicado una configuración con el número de nodos (en realidad, número de *executors*), la memoria RAM y número de cores físicos que necesitamos reservar en cada nodo. Los recursos concedidos por el gestor de cluster específico no siempre coinciden exactamente con los solicitados (depende de las políticas y del número de recursos que ya estuvieran ocupados en ese momento). En cualquier caso, la solicitud los deja reservados para que, llegado el momento en el que se vaya a ejecutar una operación distribuida de nuestro programa, puedan ser utilizados.

Estos recursos en un nodo constituyen lo que se denomina un *Executor*: un proceso de la JVM (Máquina Virtual de Java) que se ejecuta en el nodo y que ocupa los recursos indicados (cores, RAM, disco duro). El proceso ejecutor es creado por el gestor de cluster cuando arranca nuestra aplicación de Spark, y muere cuando la aplicación finaliza (ya sea con éxito o por alguna condición imprevista que provoca que toda la aplicación termine abruptamente). Cada ejecutor queda preparado para ejecutar *tareas* de Spark, que es la unidad mínima de ejecución de trabajos. Cada tarea require un core libre para ejecutarse, por lo que si un Executor tiene reservados 4 cores, podrá ejecutar 4 tareas en paralelo. Detallaremos esto más adelante. Cada uno de los nodos del cluster en los que se crean ejecutores se denomina *Worker*.

Para crear la sesión de Spark con estas indicaciones, se utiliza código similar a:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder\
                    .config("spark.executor.instances", "3")\
                    .config("spark.executor.cores", "4")\
                    .getOrCreate()
```

Este código creará 3 executors con 4 cores cada uno, siempre que existan recursos suficientes para hacerlo.

Algunas notas importantes:

- Podrían crearse varios executors en un mismo worker, si existen suficientes recursos en esa máquina worker, y si el gestor de recursos del cluster lo permite.
- Varias aplicaciones podrían compartir un mismo worker si existen suficientes recursos en esa máquina worker. Cada aplicación creará sus propios executors.
- En versiones sucesivas de Spark se introdujo la característica llamada ***dynamic allocation*** que permite crear y destruir **executors** en tiempo de ejecución en función de las necesidades de cómputo (por ejemplo si Spark detecta pocos datos al crear en DAG de un procesamiento o detecta un procesamiento muy pesado computacionalmente)
- Además, el proveedor Databricks tiene un servicio específico de *auto-escalado de clusters*, que unido a un proveedor cloud, permite añadir o quitar **workers** (y por tanto, executors dentro de ellos) en función de la carga de trabajo que detecta.
- En los **notebooks** proporcionados **por proveedores cloud concretos**, ciertos kernel configurables en esos notebooks ya traen una Spark Session configurada y lista para usar.
  - Un kernel simple de Jupyter como el que podemos desplegar en nuestro portátil no ofrece esta funcionalidad, que es propia de cada proveedor
  - En el caso particular de Databricks, se crea **un executor en cada máquina del cluster al cual asociamos el notebook**, y en principio ocupa todo el worker. Más detalles en la documentación oficial: https://spark.apache.org/docs/latest/job-scheduling.html
- Habitualmente, las ejecuciones batch de flujos de procesamientos con Spark en producción, tales como procesos de ingesta, consolidación/fusión de datos de varias fuentes o preprocesado de datos no suelen realizarse en un notebook, con lo que la creación de la sesión de Spark es un primer paso imprescindible.

Cuando creamos un cluster en una única máquina, que es la misma en la que se está ejecutando el propio Driver, se suele utilizar la opción `.master("local[4]")` siendo 4 (o cualquier número) el número de cores que deseamos utilizar

para el único executor que se creará. Para usar todos los disponibles, se utiliza `local[*]`.

2.3 RDD: Resilient Distributed Datasets

Los RDD constituyen la **abstracción fundamental de Spark** (más detalles en http://spark.apache.org/docs/latest/rdd-programming-guide.html).

Note: Un RDD es una colección no ordenada (bag) de objetos distribuida en la memoria RAM de los nodos del cluster.

La colección está dividida en *particiones*, y cada una está en la memoria RAM de un nodo distinto del clúster. Desgranando el nombre, tenemos que

- Resilient (resistente, adaptable): es posible reconstruir un RDD que estaba en memoria a pesar de que una de las máquinas falle, gracias al DAG de ejecución (Directed Acyclic Graph, el grafo de ejecución). Comentaremos esto en detalle más adelante.
- Distributed (distribuido): los objetos de la colección están divididos en *particiones* que están distribuidas en la memoria principal de los nodos del cluster. La colección no está ordenada, por lo que no se puede acceder mediante una posición a objetos individuales.
- Dataset: la colección representa un conjunto de datos que estamos procesando de forma paralela y distribuida, transformándola, calculando agregaciones, etc.

La Fig. 4 representa tres RDDs diferentes distribuidos en la memoria RAM de un cluster de 4 nodos. No todos los RDD tienen el mismo número de particiones. En la figura, uno de los RDD tiene solo 2 particiones, otro tiene 3 y otro tiene 4. La idea es similar al almacenamiento de los ficheros en HDFS, donde están distribuidos entre los discos duros de los nodos, con la diferencia de que en este caso los RDD están distribuidos en la memoria RAM de los nodos, y además, **no hay replicación de cada partición**. Si un nodo falla, es posible reconstruir las particiones que estuvieran en ese momento en la memoria principal de ese nodo gracias al DAG, que mantiene la traza de cómo se construyeron. El DAG es otro mecanismo que proporciona robustez sin necesidad de replicar las particiones de un RDD. Nótese también que, a pesar de que la Fig. 4 muestra una sola partición de cada RDD en cada nodo, lo habitual es que en la memoria de un mismo nodo haya numerosas particiones de un mismo RDD (de hecho es normal que los RDD que se van calculando tengan decenas o cientos de particiones).

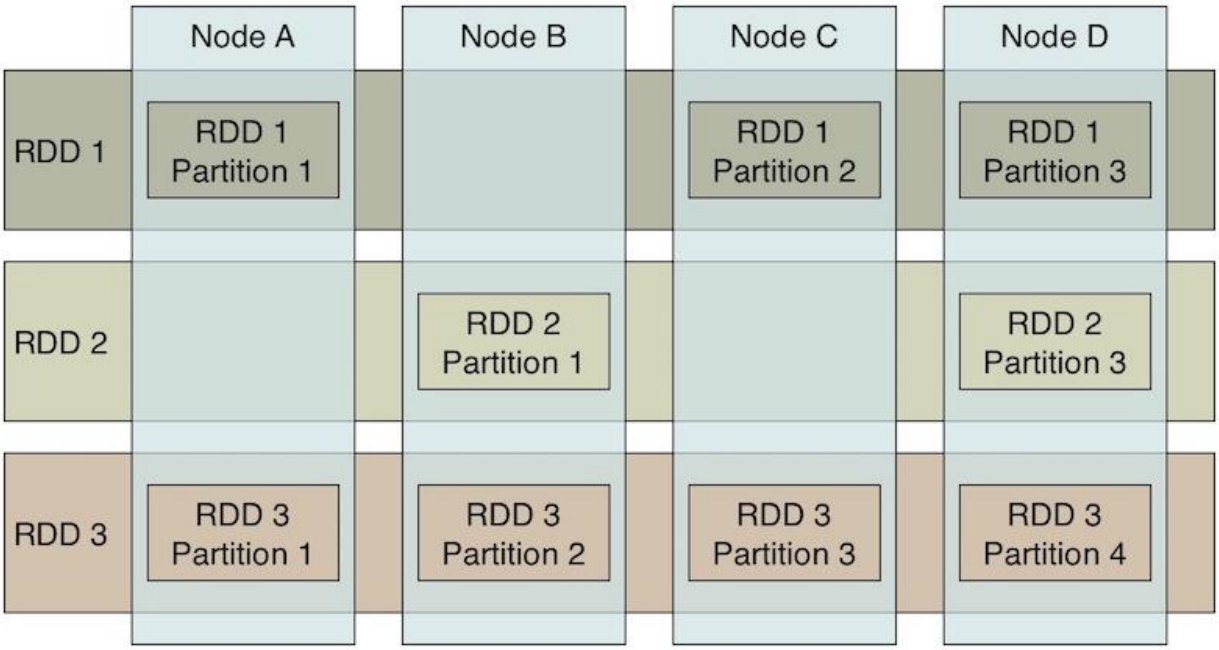


Fig. 4. Representación de tres RDDs en un cluster de Spark con cuatro workers.

Inmutabilidad de un RDD: el contenido de un RDD no puede modificarse una vez creado. Lo que hacemos es aplicar transformaciones a los RDD para obtener otros nuevos, pero los datos del RDD existente no se alteran.

La idea de la ejecución es que, cuando aplicamos una transformación, se ejecuta en paralelo sobre todas las particiones del RDD, de manera transparente al programador, para dar lugar a un nuevo RDD como resultado, cuyas particiones son el resultado de aplicar la transformación sobre cada una de las particiones del original.

- Ejemplo: dado un RDD de números reales, para multiplicar cada elemento por 2, aplicamos una transformación que actúa en cada elemento y lo multiplica por 2. Spark *lleva nuestro código de la transformación* (lo serializa y lo envía por la red) a cada uno de los nodos del cluster donde haya particiones de ese RDD, y lo ejecuta en ese nodo para que actúe en cada elemento de esa partición. Todo de manera transparente al programador.
  - Una vez más, *los datos son el centro, lo más importante; no se mueven salvo que sea imprescindible*. Ciertos tipos de transformaciones no requieren movimiento de datos pero otros sí, como veremos después.
- *Partición*: subconjunto de los objetos de un RDD que están presentes en un mismo nodo. **Es la unidad de datos mínima sobre la que se ejecuta una tarea de transformación de manera independiente al resto de particiones**. Idealmente, tendría que haber al menos tantas particiones como cores físicos (procesadores) disponibles en nuestro cluster. De esta manera nos aseguramos de que todos los cores estarán ocupados incluso en el caso de que todos estuviesen libres y nadie más que nuestra aplicación estuviese ocupando ese cluster.

Originalmente, los programadores de Spark trabajaban a nivel de RDD. En Spark 1.6 se introdujeron los DataFrames, que definiremos más adelante, y desde Spark 2.0 los propios creadores *recomiendan encarecidamente no utilizar los RDD sino siempre DataFrames con SparkSQL* como veremos en secciones posteriores. Por ese motivo, [todos los ejemplos que veremos en clase utilizan DataFrames desde el lenguaje Python, importando la biblioteca *pyspark*.]{.underline} En el anexo de este tema puedes encontrar un código de ejemplo que utiliza RDDs y un resumen de las operaciones más frecuentes con RDDs. No obstante, y puesto que un DataFrame envuelve dentro a un RDD facilitando su manejo, **todos los conceptos sobre transformaciones y acciones que se tratan en la sección siguiente son válidos cuando manejamos un DataFrame.**

2.3.1 Transformaciones y acciones sobre RDDs

Existen dos tipos de operaciones que podemos llevar a cabo cuando usamos la API de Spark:

- **Transformación**: operación que se ejecuta sobre un RDD y devuelve un nuevo RDD, en el que sus elementos se han modificado de algún modo. Son *lazy* (perezosas): no se ejecuta nada hasta que Spark encuentra una acción. Mientras tanto, Spark simplemente añade la transformación al grafo de ejecución (el **DAG**, *Directed Acyclic Graph*) que mantiene la trazabilidad y permite la resiliency.
  - El DAG tiene anotada la secuencia de transformaciones que se realizaron para obtener cada RDD concreto que se va creando en nuestro código.
- Si la transformación no implica shuffle (movimiento de datos entre nodos) se denomina *narrow*, y cada partición da lugar a otra en el mismo nodo. En caso contrario, cuando no es posible completarla en cada nodo sin acceder a los datos que hay en otros nodos, se denomina *wide*.
- **Acción**: operación que se aplica sobre un RDD y calcula un resultado (generalmente un tipo simple del lenguaje de programación, como enteros, doubles, listas, etc) que es enviado al driver (el cual corre en una máquina).
  - IMPORTANTE: el resultado de la acción debe caber en la memoria de la máquina donde se está ejecutando el proceso driver.
  - Una acción desencadena instantáneamente el cálculo de toda la secuencia de transformaciones intermedias, y la materialización de los RDDs involucrados.
  - Una vez materializado un RDD, se aplica la transformación que toque según indica el DAG para generar el siguiente RDD, y el anterior se libera (no permanece en la memoria RAM, salvo que se indique expresamente mediante el método cache()). Un RDD cacheado permanece materializado en la RAM y no es necesario recalcularlo después de que se haya materializado la primera vez cuando esté involucrado en una secuencia de transformaciones.

Por defecto, el punto de partida del DAG son operaciones de lectura de datos desde una fuente de datos como por ejemplo HDFS, o bien el sistema Amazon S3, o alguna base de datos (distribuida o no) o similar. Si ningún RDD intermedio ha sido cacheado, cualquier operación que haga referencia a un RDD exigirá reconstruir toda la secuencia de transformaciones previas a dicho RDD, empezando en la lectura de los datos, excepto que alguno de los RDD intermedios haya sido cacheado. En ese caso, las operaciones empiezan desde el RDD cacheado y no es necesario remontarse al origen de datos.

Existen ciertas operaciones de la API de Spark que **no son transformaciones ni acciones**, como por ejemplo *cache()*, sino que sirven para configurar, habilitar o deshabilitar ciertos comportamientos, o para obtener características relativas a la distribución física de un RDD (consultar el número de particiones que tiene, consultar si está cacheado, consultar el esquema –nombres y tipos de las columnas– de un DataFrame, etc). Dichas operaciones se ejecutan en el driver y no requieren intervención de las máquinas del cluster de Spark.

2.4 DataFrames y Spark SQL

Manejar RDDs resulta tedioso cuando los tipos de datos que contienen empiezan a complicarse, ya que en todo momento necesitamos saber exactamente la estructura del dato incluso cuando son tuplas jerárquicas (tuplas en las que algún campo es a su vez una tupla o lista). Sería más conveniente poder manejar los RDDs como si fuesen tablas de datos, estructuradas en filas y columnas, lo cual aportaría un mayor nivel de abstracción y más facilidad de uso. Esto es lo que nos proporciona Spark SQL y la estructura de datos fundamental, los DataFrames de Spark.

Note: DataFrame: tabla de datos distribuida en la RAM de los nodos, formada por filas y columnas con nombre y tipo (incluidos tipos complejos), similar a una tabla en una base de datos relacional

---

Internamente, un DataFrame no es más que un RDD de objetos de tipo *Row* de Spark, que representan a una fila como un vector cuyos campos tienen nombre y tipo predefinido. Cada DataFrame envuelve a un RDD de *Row* al cual podemos acceder mediante *.rdd*, por ejemplo: myDF.rdd.

A partir de la versión 2 de Spark, los creadores aconsejan utilizar exclusivamente DataFrames, y no usar los RDD sin prácticamente ninguna excepción. Aparte de la facilidad de uso, el motivo fundamental es que los DataFrames están sujetos a importantes optimizaciones automáticas de código por parte del analizador de Spark, llamado Catalyst, que **genera código al vuelo de forma automática cuando va a ejecutar el DAG con nuestras operaciones**. Catalyst optimiza el plan lógico del DAG, es decir, el orden en que se efectúan las operaciones, para obtener el mejor rendimiento, como veremos más adelante. Los RDD no están sujetos a estas optimizaciones y por tanto, la ejecución es sensiblemente más lenta.

Es importante tener en cuenta que los DataFrame de Spark están distribuidos en la memoria RAM de los nodos, pese a que su manejo parezca ser una tabla de una base de datos. Por otro lado, el nombre DataFrame es el mismo que el definido en otras librerías de lenguajes como Python (Dataframes del paquete Pandas) o R (data.frame). Si bien el **concepto** es el mismo (tabla de datos cuyas columnas tienen nombre y tipo), la implementación y el manejo no tienen nada que ver, más allá de que los autores eligieron el mismo nombre. Los DataFrames de Spark son un tipo de dato definido por Spark, están **distribuidos** en los nodos y se manejan mediante la API de Spark. En cambio, los Dataframes de Pandas existen solo en Python al importar específicamente esa biblioteca, no están distribuidos (toda la estructura está en la máquina única en la cual se esté ejecutando el intérprete de Python) y se utilizan mediante la API específica de Pandas. De la misma forma, la estructura data.frame de R está definida solo en el lenguaje R en el paquete base, no es distribuida y para utilizarla hay que acudir a la API de R.

En el caso de Spark, existe un método especial de los DataFrames que permiten traerse todo el contenido al programa driver como un dataframe de la biblioteca Pandas. Es el método **`toPandas()`**, pero debe usarse con cuidado ya que, de nuevo, requiere que todo el contenido distribuido en los nodos quepa en la memoria RAM del proceso driver. En caso contrario puede provocar una excepción *OutOfMemory*.

2.4.1 Operaciones con DataFrames: la API de Spark SQL

Distinguimos dos grupos de operaciones:

- Transformaciones y acciones sobre el DataFrame
- Funciones que transforman columnas

Transformaciones y acciones sobre el DataFrame

Son métodos de la clase `DataFrame` que devuelven otro DataFrame (transformaciones) o un tipo de dato primitivo. Se aplican con la sintaxis general: *resultDF = variableDF.nombreDelMétodo(argumentos)*. Es frecuente encadenar transformaciones, ya que como devuelven un DF, podemos aplicar directamente otra transformación al DF devuelvo por la transformación anterior.

La mayoría son operaciones sobre *columnas* (clase Column). Spark implementa muchas operaciones entre columnas de manera distribuida (operaciones aritméticas entre columnas, o con constantes, manipulación de columnas tipo string, comparaciones…) que debemos usar siempre que sea posible. Las operaciones con DataFrames son sometidas a optimizaciones por parte de Catalyst.

Todas ellas devuelven como resultado un nuevo DataFrame. Por eso se suelen encadenar transformaciones: *df.método1(args1).método2(args2)*

Supongamos que hemos hecho

```
from pyspark.sql import SparkSession
```

(continues on next page)

(continued from previous page)

```
spark = SparkSession.builder.getOrCreate()
flightsDF = spark.read.csv("flights.csv", inferSchema=True, header=True).cache()
```

Ahora podemos hacer:

- Transformación **`select`**: Recibe los nombres de las columnas que queremos seleccionar y devuelve otro DF que solo contiene esas columnas. Todas las columnas deben existir.

  Se pueden utilizar nombres de columna como string, o utilizar *objetos Column* como argumentos de `select`. Para crear un objeto Column utilizamos la función `col` del sub-paquete `pyspark.sql.functions`. Como dicho subpaquete contiene algunas funciones que colisionan con funciones del paquete base de Python como `round`, `max`, `min`, es muy frecuente importar el subpaquete con el alias `F` y utilizar sus funciones como `F.nombrefuncion`. Las funciones de dicho paquete operan con un objeto columna y devuelven otro objeto columna, y son todas operaciones distribuidas, que se efectúan en los executors, y por tanto, sobre las filas de cada partición que exista en ese executor (es decir, cada executor aplica la operación a los trozos de columna que él tiene).

  La clase Column **re-implementa** los operadores aritméticos `+`, `-`, `*`, `/` y por tanto, el producto de un objeto columna por otro objeto column, o el producto de una columna por un número real, se llevan a cabo de manera distribuida utilizando el operador producto de la clase Column de pyspark, y no el operador de multiplicación habitual de Python, ya que éste último no sabe cómo multiplicar entre sí dos objetos Column o un Column con un número real. A esto se le llama *aritmética de columnas*.

```
from pyspark.sql import functions as F

transformed_df = flightsDF.select("Month", "DayofMonth", "ArrTime", "DepDelay")

nuevo_df = flightsDF.select(
    F.col("Month"),
    F.col("Distance"),
    (1.6 * F.col("Distance")).alias("DistKm") # creamos nueva columna
)
```

- Transformación **`sample`**: recibe el porcentaje de filas que queremos mantener y devuelve otro DF que tiene solo ese porcentaje de forma aleatoria (el porcentaje **no es exacto sino aproximado**).

```
random_sample_df = flightsDF.sample(0.1)  # aproximadamente un 10% aleatorio de las␣
↪filas
```

Note: A veces necesitamos unas pocas filas que reflejen el comportamiento general, con lo cual obtenemos una muestra aleatoria. El objetivo suele ser operar en el driver para realizar, por ejemlpo, **representaciones gráficas** con Python (no Spark) con paquetes de visualización como Seaborn, Matplotlib. El DF de Spark completo probablemente no cabría en memoria.

- Transformación **`filter`** / **`where`** (equivalentes): devuelve un nuevo DF que contiene solo las filas que cumplen cierta condición booleana definida sobre una o más columnas. Las funciones `filter()` y `where` son exactamente equivalentes. El argumento puede ser un string con una condición en SQL puro, o también una condición booleana utilizando la API de SparkSQL API sobre objetos columna, siempre que el resultado de la expresión sea un objeto columna de booleanos.

```
transformed_df = (
    # Como rodeamos estas líneas entre paréntesis, no necesitamos la barra \ al final␣
↪de cada línea
```

(continues on next page)

(continued from previous page)

```
    # y además podemos usar comentarios en cada línea al no haber \
    flightsDF
        .filter("DayOfMonth < 20 or Origin = 'LAX'")     # SQL puro
        .filter("CarrierDelay is not null")              # SQL puro
        .filter(~(F.col("Distance").isNull()))           # operaciones con objetos␣
 ↪Column que devuelven un Column de booleanos
        .filter(1.6 * F.col("Distance") > 5000)          # operaciones con objetos␣
 ↪Column
        .filter(F.col("ArrDelay") > F.col("DepDelay") + F.col("CarrierDelay"))
        .where("ArrDelay > 15")                          # SQL puro
)

# Recordemos que en esta celda no se lleva a cabo ningún cálculo porque
# no hemos invocado ninguna acción sobre el DF resultante transformed_df
```

- Transformación **`withColumn`**: Devuelve un nuevo DF en el que o bien se ha creado una nueva columna o bien se ha reemplazado una columna existente, en ambos casos a partir de otras existentes. El DF devuelto contiene **todas las columnas ya existentes más la nueva por la derecha**.

  Para crear la nueva columna se puede utilizar combinado con cualquiera de las funciones del paquete *pyspark.sql.functions* que veremos en el siguiente apartado, puesto que todas son ya funciones distribuidas y paralelas. Si el nombre corresponde a una columna **ya existente**, estaremos **reemplazando** (*creándola de nuevo*) en el nuevo DF en su misma posición.

```
transformed_df = (flightsDF
        .withColumn("DistKm", 1.6 * F.col("Distance")) # crear nueva columna
        .withColumn("TotalDelay", F.col("DepDelay") + F.col("ArrDelay"))
        .withColumn("Year", F.lit(2018)) # reemplazar columna existente por una␣
 ↪columna constante con valor 2018 en todas las filas
)
```

Se suelen encadenar múltiples operaciones *withColumn* una tras otra. Las nuevas columnas se van creando por la derecha en el mismo orden en el que vamos encadenando las operaciones

- Transformación **`drop`**. Devuelve un nuevo DF en el que se ha(n) eliminado la(s) columna(s) indicada(s), pero no existe, no da ningún error.
- Transformación **`withColumnRenamed`**. Devuelve un nuevo DF en el que se ha renombrado una columna, en la misma posición que ocupaba. En este caso la columna que se está renombrando sí debe existir, o Spark dará un error. Ambas transformaciones pueden encadenarse al igual que las anteriores:

```
transformed_df = flightsDF\
    .withColumnRenamed("ArrDelay", "MinutesLate")\
    .drop("CarrierDelay", "DepDelay", "inventedName")  # esto es correcto (no␣
 ↪obtenemos error)
```

- Transformación **`summary`**. Devuelve un nuevo DF con un **un resumen estadístico** de las columnas numéricas. Media, desv. típica, min, max y cuartiles (Q1, Q2, Q3).
- Transformación **`distinct`** Devuelve un nuevo DF en el que se han eliminado las filas duplicadas, es decir, aquellas que coinciden en todos los campos. uniqueDF =

```
flights_sin_repetidos = flightsDF.distinct()

# distinct() también sirve para contar el número de valores distintos que tiene una
```

(continues on next page)

(continued from previous page)

```
# columna categórica:

n_origenes = flightsDF.select("Origin").distinct().count()
```

- Transformación **`dropDuplicates`**. Si queremos indicar a Spark que considere la fila como duplicada si coincide solo en ciertas columas que le pasamos como argumento, ignorando lo que ocurra con el resto de columnas.

```
one_row_per_route_df = flightsDF.dropDuplicates(["Origin", "Dest"]) # Basta que␣
↪coincidan en Origin y Dest para considerarse duplicadas
```

- Transformaciones **`union`**, **`intersect`** y **`exceptAll`**: devuelven un nuevo DF que es el resultado de la unión (o de la intersección, o de la diferencia, respectivamente) de las filas del DF sobre el que se aplica y el DF que se le pasa como argumento:

```
otro_df = flightsDF.where("Distance > 10000")

union_df = flightsDF.union(otro_df)
intersection_df = flightsDF.intersect(otro_df)
difference_df = flightsDF.exceptAll(otro_df)
```

- Transformación **`sort / orderBy`** (equivalentes). Devuelven un nuevo DF que es el resultado de ordenar las filas del original en base a una o varias columnas que le indiquemos. Cuando se indican varias columnas, se ordena en base a la primera, y en caso de empate, se desempata en base a la segunda columna, y así sucesivamente.

  Como en cualquier transformación, el DF original jamás se modifica. El resultado es un nuevo DF que está ordenado y que, a pesar de que no contiene números de fila, nos garantiza que las acciones que efectuemos sobre él y que actúan sobre las "n" primeras filas (tales como `show`, `take`, `collect`), actuarán sobre las filas en el orden en el que han quedado como resultado de la ordenación.

```
sorted_df = flightsDF.sort("DepTime")  # orden ascendente por defecto
sorted_df = flightsDF.orderBy("Origin", "ArrDelay", ascending = False)  # orden␣
↪descendente para todas las columnas
sorted_df = flightsDF.sort(F.col("Origin"), F.col("DepTime").desc())  # ordenar␣
↪ascendentemente en base a Origin, y en caso de empate,
                                                                      # desempatar␣
↪ordenando descendemente en base a DepTime las filas empatadas
```

- Acción **`collect`**: devuelven al driver el contenido completo del DataFrame como una lista de Python de objetos *Row*. Debe usarse con cuidado porque si es superior al tamaño de memoria del driver, causará una excepción *OutOfMemory*. Como norma general no debería utilizarse en procesos en producción.
- Acción **`head`**: devuelve al driver la *primera* fila del DataFrame como objeto *Row*. El concepto de *primera* solo está garantizado en caso de que el DF esté ordenado, es decir, que sea el DF devuelto por la transformación *sort /orderBy * sobre otro DF previo. Si no, simplemente devuelve una fila, sin garantizar cuál será.
- Acción **`take(n)`**: devuelve al driver una lista de Python de objetos `Row` con las *n* primeras fila del DataFrame, siendo *n* un argumento de tipo entero que debemos pasarle. Mismo razonamiento explicado antes acerca de la ordenación.

```
listaRows = flightsDF.limit(100000).collect()   # ponemos limit en el ejemplo para␣
↪evitar que el collect desborde la memoria del driver

print(f"La longitud de la lista devuelta es de {len(listaRows)} elementos")
print("--------")
```

(continues on next page)

(continued from previous page)

```
algunasRows = flightsDF.take(3)
firstFlight = flightsDF.head()

print(firstFlight)
print("----------")
print(f"El valor del atributo Origin es {firstFlight.Origin}")  # podemos acceder a␣
↪cada campo de un objeto Row con el "."
```

```
La longitud de la lista devuelta es de 100000 elementos
--------
Row(Month=1, DayofMonth=14, DayOfWeek=7, FlightDate=datetime.date(2018, 1, 14),␣
↪Origin='SYR', OriginCity='Syracuse, NY', Dest='DTW', DestCity='Detroit, MI',␣
↪DepTime=None, DepDelay=None, ArrTime=None, ArrDelay=None, Cancelled=1.0,␣
↪CancellationCode='B', Diverted=0.0, ActualElapsedTime=None, AirTime=None,␣
↪Distance=374.0, CarrierDelay=None, WeatherDelay=None, NASDelay=None,␣
↪SecurityDelay=None, LateAircraftDelay=None)
----------
El valor del atributo Origin es SYR
```

- Acción **`show(n=20)`**: envía al driver las *n* primeras filas (20 si no indicamos nada) para que éste las imprima por pantalla (mismo razonamiento acerca de la ordenación). **La función devuelve *`None`* como resultado**, es decir, las filas se imprimen pero no se devuelven como resultado de la función, ya que esta devuelve el objeto nulo de Python.
- Acción **`count()`**: envía al driver un entero con el número total de filas del DF sobre el que la aplicamos.

Funciones que transforman columnas

Son funciones pertenecientes al paquete `pyspark.sql.functions` (existen 420 funciones en dicho subpaquete, según la documentación de pyspark versión 3.5.0) y todas reciben como entrada uno o varios objetos Column de un DataFrame, y devuelven generalmente otro objeto Column transformado. Todas las operaciones están implementadas de forma distribuida, por lo que cada executor las aplica sobre las filas que posee. No se aplican al objeto DataFrame sino que se invocan directamente como funciones. Es frecuente importar el paquete *functions* completo con algún alias, por ejemplo F, para no interferir esas funciones con respecto a otras que se llamen igual en Python (por ejemplo min, max), y usarlas siempre con ese prefijo: F.col(…), F.when(…), etc. La mayoría de estas operaciones reciben como argumento un string con el nombre de una o varias columnas a las que se aplica, aunque algunas requieren pasarles el objeto columna con F.col("nombre") en vez de directamente el string "nombre".

Existe una gran variedad de funciones para transformar columnas de un DataFrame. Solo comentaremos las más frecuentes.

- Función **`F.col("nombreCol")`**: sirve simplemente para obtener un objeto Column a partir del nombre de la columna. Se suele utilizar dentro de otras funciones para referenciar a columnas de un DataFrame con las que queremos operar.
- Función **`F.lit(constante)`**: sirve para crear una columna constante, es decir, con todas sus filas igual al valor que le pasemos como argumento, que puede ser un número, un string, un booleano, …
- Función **`F.when`**: devuelve un nuevo objeto columna en base a casuísticas: `F.when(cond1, valor1).when(cond2, valor2).otherwise(valor3)`

  Los valores *valor1, valor2, valor3* pueden ser constantes con F.lit(…) o pueden ser objetos Column, es decir, objetos columna existentes u operaciones con ellos. Se utiliza generalmente dentro de la transformación *withColumn*. La creación del nuevo objeto columna se lleva a cabo en los executors, fila a fila, de manera que en cada fila se

comprobará la condición y se elegirá el valor que debe colocarse en esa fila de la nueva columna. Dicho valor puede ser una constante o puede ser el valor que ya hubiera en otra columna de esa misma fila.

```
flights_no_negativos_df = flightsDF.withColumn("ArrDelay",
    F.when(F.col("ArrDelay") < 0, F.lit(0))  # Modificamos el valor si era negativo.␣
↪Si no, lo dejamos como estaba
    .otherwise(F.col("ArrDelay"))            # Tomará el valor que hubiera en esta␣
↪misma fila en la columna ArrDelay
)
```

El valor devuelto por *when* es un objeto Column, que es lo que le pasamos a *withColumn* como segundo argumento (el primero es el nombre de la nueva columna).

- Funciones matemáticas y estadísticas que devuelven objetos columna:
  - Columna de números aleatorios: **`F.rand`** (uniforme en [0,1]) o **`F.randn`** (normal estándar). Ejemplo: `df.withColumn("aleat", F.rand())`
  - Correlaciones entre columnas (devuelve un número): **`F.corr("col1", "col")`**, tablas de contigencia.
  - Funciones de aritmética entre columnas: `F.col("ArrDelay") + F.col("DepDelay")` devuelve otro objeto columna con la suma.
  - Operaciones matemáticas sobre una columna: **`F.sqrt("colName")`**, **`F.sin("colName")`**, **`F.log("colName")`**…
  - Suelen utilizarse también dentro de *withColumn* aunque pueden usarse también dentro de *select* para crear al vuelo una nueva columna.

Métodos de la clase Column

La clase Column posee varios métodos pero hay dos muy frecuentes. Transforman el objeto Columna al que se aplican para **renombrarlo** (**alias**, usado en columnas creadas al vuelo) o para **cambiar su tipo de dato (cast)**. Importante: el tipo indicado debe ser un *tipo de dato de Spark* de los existentes en `pyspark.sql.types`. El tipo de dato se puede indicar con un string, como `"int"`, `"double"`, `"boolean"` o como `IntegerType()`, `BooleanType()`, etc.

```
from pyspark.sql.types import IntegerType

flightsTransformadoDF = flightsDF.select(
    F.col("ArrDelay").cast(IntegerType()),   # convertimos a entero
    (1.6 * F.col("Distance")).alias("DistKm")    # damos nombre a la nueva col
)
```

Ejemplos con funciones matemáticas y estadísticas

Generar con select dos columnas de distribuciones uniforme y normal

```
flightsDF.select(
    F.rand(seed=10).alias("uniforme"),  # aleatorio siguiendo una distribución␣
↪uniforme en el intervalo [0 ,1]
    F.randn(seed=27).alias("normal")   # aleatorio siguiendo una Normal estándar␣
↪(media 0 y varianza 1)
).show()
```

```
+-------------------+--------------------+
|           uniforme|              normal|
+-------------------+--------------------+
| 0.1709497137955568| -0.8664700627108758|
| 0.8051143958005459| -0.5970491018333267|
| 0.5775925576589018| 0.18267161219540898|
| 0.9476047869880925| -1.8497305679917546|
|    0.2093704977577|  0.9410417279045351|
|0.36664222617947817| -0.6516475674670159|
| 0.8078688178371882|  0.5901002135239671|
| 0.7135143433452461|  -1.850241871360443|
| 0.7195325566306053| 0.09176896733073023|
|0.31335292311175456|-0.38605118617831075|
| 0.8062503712025726|  1.2134544166783332|
|0.10814914646176654| -1.0757702531630617|
| 0.3362232980701172| 0.04961226872064977|
| 0.8133304803837667|  -0.768259602441542|
|0.47649428738170896|  0.2911293146907403|
|  0.524728096293865|-0.33406080411047484|
| 0.9701253460019921|  1.3607097640771781|
| 0.6232167713919952|  0.5986772981082732|
| 0.5089687568245219|-0.35158579838711623|
| 0.5467504094508642| -0.9115825072509143|
+-------------------+--------------------+
only showing top 20 rows
```

Podemos usar *describe* para calcular estadísticos descriptivos simples de cada columna que le pasemos como argumento, o de todas las columnas si no le pasamos ninguna

```
df = flightsDF.limit(1000)\
              .withColumn("rand1", F.rand(seed=10))\
              .withColumn("rand2", F.rand(seed=27))

cov = df.stat.cov("rand1", "rand2")
corr = df.stat.corr("rand1", "rand2")

print(f"La covarianza es {cov} y la correlación lineal de Pearson es {corr}")
```

```
La covarianza es -0.0002591695400945357 y la correlación lineal de Pearson es -0.
↪0029839468526783536
```

Tablas de contingencia

Creamos un DataFrame con dos columnas (name, item)

```
names = ["Alice", "Bob", "Mike"]

items = ["milk", "bread", "butter", "apples", "oranges"]

df = spark.createDataFrame([(names[i % 3], items[i % 5]) for i in range(100)],
                           ["name", "item"]).cache()

# Vamos a mostra 10 filas por pantalla
```

(continues on next page)

(continued from previous page)

```
df.show(10)

# Lo siguiente equivale a df.groupBy("name").pivot("item").count()
df.stat.crosstab("name", "item").show()
```

```
+-----+-------+
| name|   item|
+-----+-------+
|Alice|   milk|
|  Bob|  bread|
| Mike| butter|
|Alice| apples|
|  Bob|oranges|
| Mike|   milk|
|Alice|  bread|
|  Bob| butter|
| Mike| apples|
|Alice|oranges|
+-----+-------+
only showing top 10 rows
```

```
+---------+------+-----+------+----+-------+
|name_item|apples|bread|butter|milk|oranges|
+---------+------+-----+------+----+-------+
|      Bob|     6|    7|     7|   6|      7|
|    Alice|     7|    7|     6|   7|      7|
|     Mike|     7|    6|     7|   7|      6|
+---------+------+-----+------+----+-------+
```

Operaciones con columnas de tipo vector

Funciones que se aplican a un objeto Column de tipo array y devuelven un nuevo objeto Column donde la función se ha aplicado a cada celda de la columna original. Existen funciones que devuelven el elemento de alguna posición concreta de cada vector, o bien la longitud de cada vector, el mínimo, el máximo, el propio vector ordenado, el array con los elementos comunes a dos columnas de tipo vector, su unión, el subvector de elementos que cumplen una condición (filtrado del vector), la explosión entendida como desenrollar los elementos repitiendo las filas para que cada elemento del vector se asocie a una fila completa diferente (manteniendo constante los elementos de las demás columnas), … Podemos consultar en la documentación ya que la mayoría empiezan por "array_". La tabla de la página siguiente contiene un resumen de algunas.

Ejemplos:

```
todas_fechas_df = flightsDF\
    .groupBy("Origin", "Dest")\
    .agg(F.collect_list("FlightDate").alias("todas_fechas"),
         F.collect_set("FlightDate").alias("fechas_sin_repetidos"))\
    .withColumn("longitud_todas_fechas", F.size("todas_fechas"))\
    .withColumn("longitud_fechas_sin_rep", F.size("fechas_sin_repetidos"))\
    .withColumn("primer_elemento", F.element_at("todas_fechas", 1))    # primer␣
↪elemento de cada array
```

(continues on next page)

(continued from previous page)

```
todas_fechas_df.select("Origin", "Dest", "todas_fechas", "longitud_todas_fechas",
 ↪"fechas_sin_repetidos", "longitud_fechas_sin_rep", "primer_elemento")\
               .show(3)
```

```
+------+----+--------------------+--------------------+--------------------+------
 ↪-----------------+---------------+
|Origin|Dest|        todas_fechas|longitud_todas_fechas|fechas_sin_
 ↪repetidos|longitud_fechas_sin_rep|primer_elemento|
+------+----+--------------------+--------------------+--------------------+------
 ↪-----------------+---------------+
|   ABE| CLT|[2018-01-01, 2018...|                 264|[2018-01-15, 2018...|     ␣
 ↪              120|     2018-01-01|
|   ABE| FLL|[2018-01-03, 2018...|                  34|[2018-03-07, 2018...|     ␣
 ↪               34|     2018-01-03|
|   ABI| DFW|[2018-01-01, 2018...|                 629|[2018-01-15, 2018...|     ␣
 ↪              120|     2018-01-01|
+------+----+--------------------+--------------------+--------------------+------
 ↪-----------------+---------------+
only showing top 3 rows
```

Nombre de la función que se aplica a columnas de tipo vector	Descripción
array_contains(column: Column, value: Any)	Comprueba si un valor se presenta en una columna tipo vecto
array_distinct(e: Column)	Devuelve una columna de tipo vector tras eliminar duplicados
array_except(col1: Column, col2: Column)	Devuelve todos los elementos que están en el vector col1 perc
array_intersect(col1: Column, col2: Column)	Devuelve todos los elementos que están presentes en los vecto
array_join(column: Column, delimiter: String, nullReplacement: String)	Concatena entre sí todos los elementos del vector utilizando e
array_join(column: Column, delimiter: String)	
array_max(e: Column)	Devuelve el valor máximo del vector
array_min(e: Column)	Devuelve el valor mínimo del vector
array_position(column: Column, value: Any)	Devuelve la posición/índice de la primera aparición del "valor
array_remove(column: Column, element: Any)	Devuelve una columna tipo vector tras eliminar el elemento d
array_repeat(e: Column, count: Int)	Crea un vector que contiene el primer argumento repetido el
array_repeat(left: Column, right: Column)	Crea un vector que contiene el primer argumento repetido el
array_sort(e: Column)	Devuelve una versión ordenada del vector. Los valores nulos
array_union(col1: Column, col2: Column)	Devuelve un vector de elementos que están presentes en ambo
arrays_overlap(a1: Column, a2: Column)	True: si `a1` y `a2` tienen al menos un elemento no nulo en cor
arrays_zip(e: Column*)	Devuelve una columna vector de tuplas de dimensión igual al
concat(exprs: Column*)	Concatena todos los elementos de una columna determinada.
element_at(column: Column, value: Any)	Devuelve el elemento del vector situado en la posición de entr
exists(column: Column, f: Column => Column)	Comprueba si el vector contiene un elemento que cumpla la c
explode(e: Column)	Crea una fila para cada elemento en la columna del vector
explode_outer (e : Column)	Crea una fila para cada elemento en la columna del vector. A
filter(column: Column, f: Column => Column)	Devuelve un vector de elementos para los que se cumple un p
filter(column: Column, f: (Column, Column) => Column)	
flatten(e: Column)	Crea un vector a partir de una columna de vector de vectores
forall(column: Column, f: Column => Column)	Devuelve True si un predicado se cumple para cada elemento
posexplode(e: Column)	Crea una fila para cada elemento del vector y crea dos colum
posexplode_outer(e: Column)	Ídem al anterior, pero a diferencia de posexplode, si el vector
reverse(e: Column)	Devuelve el vector de elementos en orden inverso.
shuffle(e: Column)	Desordena aleatoriamente los elementos del vector ("barajar"

Table 2.1

Nombre de la función que se aplica a columnas de tipo vector	Descripción
size(e: Column)	Devuelve la longitud de un vector.
slice(x: Column, start: Int, length: Int)	Devuelve el vector de elementos empezando en la posición "i
sort_array(e: Column)	Ordena el vector en orden ascendente. Los valores nulos se c
sort_array(e: Column, asc: Boolean)	Ordena el vector en orden ascendente o descendente según el
transform(column: Column, f: Column => Column)	Devuelve un vector de elementos después de aplicar a cada el
transform(column: Column, f: (Column, Column) => Column)	

Columnas de tipo estructura (tupla)

Una columna de tipo estructura representa un tipo de dato creado por el usuario, que puede ser "plano", es decir, ser simplemente una tupla, o puede ser "jerárquico" porque dentro de la estructura existan campos que son, a su vez, otras estructuras. El caso más simple es crear una tupla con los valores de varias columnas, fila a fila. Es posible "ordenar" tuplas, porque ordena en base al primer elemento de cada tupla (el más a la izquierda), y si empatan entonces se fija en el segundo, y así sucesivamente. Esto es útil cuando queremos ordenar dentro de un grupo en base a cierta columna, y después queremos ver el valor concreto de otra columna diferente que estaba en la misma fila, emparejado con el valor máximo o mínimo del grupo (es decir, cuando buscamos máximo o mínimo, pero no queremos perder la correspondencia de ese valor con otros valores de su fila).

Se puede acceder a cada uno de los campos de una tupla con el operador . (punto)

Ejemplo: para cada par de aeropuertos origen y destino, encontrar la fecha en la que tuvo lugar el vuelo con más retraso a la llegada. Como estamos creando una columna de tipo estructura con dos campos, de los cuales el primero es el ArrDelay, al buscar dentro de cada grupo el máximo de dicha columna de tipo estructura, se fijará justamente en el ArrDelay.

```
from pyspark.sql import Window

fechaMaxDelayDF = flightsDF\
                    .withColumn("estructura", F.struct("ArrDelay", "FlightDate"))\
                    .groupBy("Origin", "Dest")\
                    .agg(F.max("estructura").alias("max_struct"))\
                    .withColumn("fecha_max_retraso", F.col("max_struct.FlightDate"))

fechaMaxDelayDF.show(5, truncate=False)
```

```
+------+----+------------------+-----------------+
|Origin|Dest|max_struct        |fecha_max_retraso|
+------+----+------------------+-----------------+
|ABE   |ATL |{459.0, 2018-02-20}|2018-02-20       |
|ABE   |CLT |{347.0, 2018-02-04}|2018-02-04       |
|ABE   |DTW |{674.0, 2018-02-09}|2018-02-09       |
|ABE   |FLL |{340.0, 2018-03-07}|2018-03-07       |
|ABE   |MDT |{NULL, 2018-01-12} |2018-01-12       |
+------+----+------------------+-----------------+
only showing top 5 rows
```

Transformaciones para agrupar y agregar

La transformación *groupBy("nombreCol1", "nombreCol2", ...)* sobre un DataFrame devuelve una estructura de datos llamada *RelationalGroupedDataset*, que **no** es un DataFrame y sobre la que apenas se pueden aplicar operaciones. Las más frecuentes que se suelen aplicar son *count*(), que efectúa un conteo del número de elementos por cada grupo, o la función *agg*() que es la más habitual, y realiza para cada grupo las agregaciones que le indiquemos en los argumentos de agg(). El resultado tendrá tantas filas como combinaciones de valores posibles haya en las variables de grupo, y solamente contendrá aquellas columnas que se incluyeron como argumentos en el *groupBy* (que definen los grupos) más una columna por cada función de agregación que incluimos como argumento de *agg()*. La función *pivot*() también sirve para agupar, pero desplegando como columnas cada una de las combinaciones de categorías de las variables que le pasamos como argumento.

```
import pyspark.sql.functions as F

summaries_df = flightsDF\
    .groupBy("Origin", "Dest")\
    .agg(
         F.max("Distance").alias("maxDistance"),
         F.mean("ArrDelay").alias("meanArrDelay"),
         F.mean("DepDelay").alias("meanDepDelay"),
         F.stddev("ArrDelay").alias("stddevArrDelay"),
         F.count("*").alias("groupSize"),
         F.countDistinct("Dest").alias("distinctDestinations")
    )

summaries_df.show()
```

```
+------+----+-----------+-------------------+-------------------+------------------
 ↪+---------+--------------------+
|Origin|Dest|maxDistance|       meanArrDelay|       meanDepDelay|   ␣
 ↪stddevArrDelay|groupSize|distinctDestinations|
+------+----+-----------+-------------------+-------------------+------------------
 ↪+---------+--------------------+
|   BMI| ATL|      533.0| 17.115727002967358|  20.11111111111111| 74.
 ↪55372157534977|      343|                   1|
|   FLL| LGB|     2327.0| 18.266666666666666|             28.575|59.
 ↪455898963590045|      120|                   1|
|   FLL| CVG|      932.0|  5.612903225806452| 11.330645161290322|44.
 ↪525334584674795|      248|                   1|
|   BIS| AZA|     1092.0|  7.526315789473684| 11.355263157894736|  59.
 ↪3011464047052|       77|                   1|
|   ATL| FLL|      581.0|-0.0657752078062884|  6.979429808733309|25.
 ↪828695002011504|     2782|                   1|
|   CLT| BNA|      328.0|              4.721| 7.1365902293120635| 36.
 ↪29730917924432|     1013|                   1|
|   SBA| PDX|      784.0| -4.275862068965517| -4.379310344827586|37.
 ↪764392294029186|      120|                   1|
|   DTW| LEX|      296.0|  2.216589861751152|  9.554022988505746|55.
 ↪302352654559996|      440|                   1|
|   LIT| DEN|      771.0| 10.050314465408805| 11.081761006289309|57.
 ↪802808347844525|      319|                   1|
|   BUF| PHX|     1912.0|  11.98360655737705| 19.360655737704917| 35.
 ↪51067435992755|      125|                   1|
|   DSM| STL|      259.0|-0.2850678733031674| 3.3755656108597285|19.
 ↪548429456094873|      223|                   1|
|   GEG| OAK|      723.0| -4.531120331950207|  2.954356846473029| 19.
 ↪69517883026943|      241|                   1|
```

(continues on next page)

(continued from previous page)

```
|   DEN| LGB|      854.0|-3.0294117647058822| 4.05882352941176 45|15.
↪196660905402767|       35|                   1|
|   SIT| JNU|       95.0|-1.2258064516129032|-2.4566210045662102|36.
↪296868518884395|      223|                   1|
|   MSP| BIS|      386.0|  4.032786885245901|  9.153970826580228| 41.
↪25493142164648|      636|                  1|
|   CVG| CLT|      335.0| 11.320866141732283| 13.566666666666666| 45.
↪82282647594028|     1038|                  1|
|   SJC| JFK|     2569.0|  4.731182795698925|  20.91578947368421| 30.
↪54367499633009|       99|                  1|
|   PIE| CID|     1090.0| 2.5714285714285716|   6.73469387755102| 50.
↪41411839289995|       49|                  1|
|   CVG| LAX|     1900.0|             11.692|  8.466135458167331| 34.
↪99508192640995|      251|                  1|
|   PIE| GSO|      587.0|-3.1714285714285713| 6.0285714285714285|34.
↪055737803062264|       35|                   1|
+------+----+-----------+-------------------+-------------------+-----------------
↪+---------+--------------------+
only showing top 20 rows
```

Cuando ejecutamos funciones de agregación sobre el DF completo, sin haber llevado previamente una agrupación con *groupBy*, se devuelve un DataFrame con una sola fila con el resultado de la agregación. El fragmento de código siguiente muestra cómo se utilizan *groupBy* y *agg*.

Agregación con funciones de ventana

Son similares a una agregación por grupos, pero se usan para añadir la agregación en una nueva columna del DF original. **No modifican el número de filas del DF original** sino que calculan la agregación en cada grupo y la añaden en las filas que pertenecen a ese grupo. Suele utilizarse dentro de *withColumn*. Los grupos se definen en el objeto *Window* que hay que crear y que se utiliza para agregar. A continuación comparamos el comportamiento de groupBy y el de una agregación con ventana.

En el ejemplo siguiente, vemos que a cada vuelo se le ha "pegado" el retraso medio del grupo al cual pertenece, es decir, el retraso medio de los vuelos que tienen su mismo aeropuerto de origen. Por eso, vuelos que tienen mismo origen aunque tengan distinto destino muestran el mismo valor en `MeanDelay`.

```
from pyspark.sql import Window

w = Window().partitionBy("Origin")

# tantas filas como aeropuertos de origen \# tantas filas como tenga flightsDF
flightsDF.withColumn("MeanDelay", F.mean("ArrDelay").over(w))\
         .select("Month", "DayofMonth", "DayOfWeek", "Origin", "Dest", "ArrDelay",
↪"MeanDelay")\
         .dropDuplicates(["Origin", "Dest"])\
         .show()
```

```
+-----+----------+---------+------+----+--------+------------------+
|Month|DayofMonth|DayOfWeek|Origin|Dest|ArrDelay|         MeanDelay|
+-----+----------+---------+------+----+--------+------------------+
|    1|         2|        2|   BGM| DTW|    -2.0|16.874538745387454|
|    1|         2|        2|   BTM| SLC|     0.0| 3.009009009009009|
|    1|         1|        1|   BUR| JFK|    -2.0|4.9543734953571015|
```

(continues on next page)

(continued from previous page)

```
|    1|          2|        2|   BUR| SFO|    33.0|4.9543734953571015|
|    1|          2|        2|   BUR| DEN|     1.0|4.9543734953571015|
|    1|          3|        3|   BUR| SJC|   -18.0|4.9543734953571015|
|    1|          3|        3|   BUR| SLC|   -28.0|4.9543734953571015|
|    1|          3|        3|   BUR| DAL|   -13.0|4.9543734953571015|
|    1|          3|        3|   BUR| LAS|    -6.0|4.9543734953571015|
|    1|          3|        3|   BUR| OAK|   -20.0|4.9543734953571015|
|    1|          3|        3|   BUR| PDX|   -12.0|4.9543734953571015|
|    1|          3|        3|   BUR| PHX|    10.0|4.9543734953571015|
|    1|          3|        3|   BUR| SMF|   -13.0|4.9543734953571015|
|    1|          2|        2|   BUR| SEA|     3.0|4.9543734953571015|
|    1|          2|        2|   COD| SLC|     3.0|3.2696245733788394|
|    3|          4|        7|   COD| DEN|   245.0|3.2696245733788394|
|    1|          1|        1|   DCA| BOS|   -21.0|  4.12925289757857|
|    1|          1|        1|   DCA| PBI|     5.0|  4.12925289757857|
|    1|          1|        1|   DCA| RSW|    -4.0|  4.12925289757857|
|    1|          1|        1|   DCA| TPA|    34.0|  4.12925289757857|
+-----+-----------+---------+------+----+--------+------------------+
only showing top 20 rows
```

En un objeto Window pueden definirse:

- Grupos, mediante **`partitionBy("col1", "col2")`**. Las funciones de agregación se calcularán para cada grupo de la ventana, y el valor del grupo adecuado (en base a los valores que toman en esa fila las columnas que definen el grupo) será añadido en la nueva columna que hemos creado con *withColumn*. Las funciones de agregación que se suelen aplicar son las mismas que con *groupBy*.
- Ordenaciones, mediante **`orderBy*("col1", "col2")`**. Sirven para casos en los que queremos aplicar a cada fila una función de agregación que depende del orden, como por ejemplo
  - `F.first("col1")` o `F.last("col1")`, que devuelven el menor o el mayor valor de cierta columna por la que previamente hemos ordenado la ventana.
  - La función `F.rank()`, que crea una columna de enteros con la *posición* que ocupa cada fila dentro de un grupo que hemos ordenado previamente.
  - En una ventana ordenada también se puede aplicar la función `F.lag("colName")` para calcular la diferencia entre el valor de "colName" de una fila y el de la fila siguiente en la ordenación, o el de *count* filas posteriores si le pasamos un segundo argumento entero. Se usa con frecuencia en problemas de series temporales.

Ejemplo:

```
from pyspark.sql import Window

w = Window().partitionBy("Origin").orderBy("FlightDate")

flightsDF.withColumn("MeanDelay", F.mean("ArrDelay").over(w))\
         .withColumn("rankByDate", F.rank().over(w))\
         .select("Month", "DayofMonth", "FlightDate", "Origin", "Dest", "MeanDelay",
 ↪"rankByDate")\
         .show()
```

```
+-----+----------+----------+------+----+------------------+----------+
|Month|DayofMonth|FlightDate|Origin|Dest|         MeanDelay|rankByDate|
+-----+----------+----------+------+----+------------------+----------+
```

(continues on next page)

(continued from previous page)

```
|    1|          1|2018-01-01|   BGM| DTW|               -3.0|          1|
|    1|          2|2018-01-02|   BGM| DTW|             -8.75|          2|
|    1|          2|2018-01-02|   BGM| DTW|             -8.75|          2|
|    1|          2|2018-01-02|   BGM| DTW|             -8.75|          2|
|    1|          3|2018-01-03|   BGM| DTW|-4.833333333333333|          5|
|    1|          3|2018-01-03|   BGM| DTW|-4.833333333333333|          5|
|    1|          4|2018-01-04|   BGM| DTW|            -1.375|          7|
|    1|          4|2018-01-04|   BGM| DTW|            -1.375|          7|
|    1|          5|2018-01-05|   BGM| DTW|               2.2|          9|
|    1|          5|2018-01-05|   BGM| DTW|               2.2|          9|
|    1|          6|2018-01-06|   BGM| DTW| 6.909090909090909|         11|
|    1|          7|2018-01-07|   BGM| DTW|3.5384615384615383|         12|
|    1|          7|2018-01-07|   BGM| DTW|3.5384615384615383|         12|
|    1|          8|2018-01-08|   BGM| DTW| 4.866666666666666|         14|
|    1|          8|2018-01-08|   BGM| DTW| 4.866666666666666|         14|
|    1|          9|2018-01-09|   BGM| DTW| 3.235294117647059|         16|
|    1|          9|2018-01-09|   BGM| DTW| 3.235294117647059|         16|
|    1|         10|2018-01-10|   BGM| DTW|4.2631578947368425|         18|
|    1|         10|2018-01-10|   BGM| DTW|4.2631578947368425|         18|
|    1|         11|2018-01-11|   BGM| DTW|1.6666666666666667|         20|
+-----+-----------+----------+------+----+------------------+-----------+
only showing top 20 rows
```

Transformación JOIN

Método *join*: misma operación que en SQL con misma semántica.

- Si el nombre de columna es el mismo en ambos DF, se indica como string. Si no, se especifica una condición entre columnas que devuelva una columna de bool.
- Provoca movimiento de datos entre nodos y es la operación más pesada. Antes de usarla, debemos pensar si no hay más remedio. Muchas veces se puede sustituir por agregaciones con una ventana (sección siguiente).

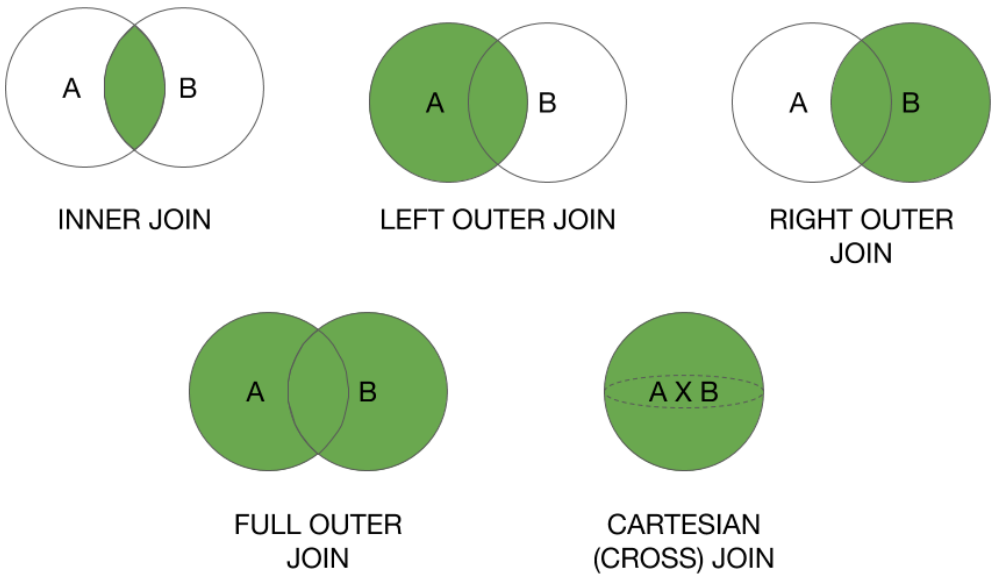


Fig. 6. Tipos de join.

Existen los mismos tipos de JOIN que en SQL: inner, cross, outer, full, full_outer, left, left_outer, right, right_outer, left_semi, left_anti, incluso por condición arbitraria. Si hay matching de una fila con varias filas del otro DF, se incrementa el número de filas del resultado respecto al número de filas original.

Ejemplos:

- Cuando las columnas por las que se hace el matching tienen el mismo nombre en ambos DF, se pueden especificar dichas columnas como una lista de strings.

```
tableA = spark.createDataFrame(
    [("a", 1, "010"), ("b", 4, "011"), ("r", 10, "012"), ("f", 13, "013")],
    ["a1", "a2", "f"]
)
tableA.show()

tableB = spark.createDataFrame(
    [("blue", 3, "023"), ("red", 10, "012"), ("orange", 3, "009"), ("yellow", 13, "013
 ↪")],
    ["b1", "b2", "f"]
)
tableB.show()

# el resultado de un left_outer tiene, como mínimo, tantas filas como el DF de la␣
 ↪izquierda, aunque puede tener más
# si hay filas de un DF que tienen más de una coincidencia con el otro DF
joined_df = tableA.join(tableB, on = ["f"], how = "left_outer")
joined_df.show()
```

```
+---+---+---+
| a1| a2|  f|
+---+---+---+
|  a|  1|010|
|  b|  4|011|
|  r| 10|012|
|  f| 13|013|
+---+---+---+
```

```
+------+---+---+
|    b1| b2|  f|
+------+---+---+
|  blue|  3|023|
|   red| 10|012|
|orange|  3|009|
|yellow| 13|013|
+------+---+---+
```

```
+---+---+---+------+----+
|  f| a1| a2|    b1|  b2|
+---+---+---+------+----+
|010|  a|  1|  NULL|NULL|
|011|  b|  4|  NULL|NULL|
|012|  r| 10|   red|  10|
|013|  f| 13|yellow|  13|
+---+---+---+------+----+
```

El tipo de JOIN por defecto es **inner** cuando no especificamos el argumento `how`: el resultado solo contiene filas que cumplen la condición en ambos DF. Si las columnas que se corresponden **no** tienen el mismo nombre:

```
tabla_izq = tableA.withColumnRenamed("f", "f1")
tabla_der = tableB.withColumnRenamed("f", "f2")

joined_df = tabla_izq.join(tabla_der, on = tabla_izq.f1 == tabla_der.f2) # inner por␣
 ↪defecto
joined_df.show()
```

```
+---+---+---+------+---+---+
| a1| a2| f1|    b1| b2| f2|
+---+---+---+------+---+---+
|  r| 10|012|   red| 10|012|
|  f| 13|013|yellow| 13|013|
+---+---+---+------+---+---+
```

Note: Cuando ya existen columnas con el mismo nombre en ambos DF y no van a ser utilizadas en la condición de join, o bien cuando sí van a utilizarse pero la condición incluye al mismo tiempo otras columnas con distintos nombres (y eso debemos expresar la condición mediante una operación con objetos columna), el DF resultante acabará teniendo columnas repetidas con el mismo nombre. En dicho caso, las columnas con el mismo nombre no podrán utilizarse después, ya que obtendremos un error de "referencia ambigua". Para ese caso, se suele utilizar la operación `alias` de los DF (no de los objetos columna). Con esto, podemos referirnos a las columnas de un DF o del otro aunque tengan el mismo nombre, simplemente utilizando `nombre_alias.nombre_columna` como si nombre de la columna.

```
error_df = tableA.join(tableB, on = (tableA.f == tableB.f) & (tableA.a2 == tableB.b2),
 ↪ how = "left_outer")
# error_df.select("f")  # error! AMBIGUOUS_REFERENCE: Reference 'f' is ambiguous

# Para evitar el error anterior, ponemos un alias a los DF
joined_df = tableA.alias("A").join(tableB.alias("B"), on = ["f"], how = "left_outer")
joined_df.select("A.*", "b1", "b2").show()
```

```
+---+---+---+------+----+
|  f| a1| a2|    b1|  b2|
+---+---+---+------+----+
|010|  a|  1|  NULL|NULL|
|011|  b|  4|  NULL|NULL|
|012|  r| 10|   red|  10|
|013|  f| 13|yellow|  13|
+---+---+---+------+----+
```

Un tipo especial no contemplado en la figura es el *left_semi*: es una especie de *filter* sobre el DataFrame A donde la condición de filtrado es "tener matching con alguien de B". El resultado contiene solo las columnas de A (por eso se puede considerar como un mero filtrado). No se añade ninguna columna de B al resultado.

```
# el resultado contiene sólo las columnas de tableA, y solo las filas que tienen al␣
 ↪menos
# un matching con filas de B
tableA.join(tableB, "f", "left_semi").show()
```

```
+---+---+---+
|  f| a1| a2|
+---+---+---+
```

(continues on next page)

(continued from previous page)

```
|012|  r| 10|
|013|  f| 13|
+---+---+---+
```

Existe un tipo análogo de *join* llamado *left_anti*, cuyo comportamiento es retener aquellas filas del DF izquierdo (**sin añadirles columnas adicionales**) que no tienen correspondencia con ninguna fila del DF derecho. *Left_anti* sería la operación complementaria de *left_semi* ya que el resultado contendría justamente las filas del DF de la izquierda que serían excluidas por *left_semi*.

```
# el resultado contiene sólo las columnas de tableA, y solo las filas
# que no tienen ningún matching con filas de B
tableA.join(tableB, ["f"], "left_anti").show()
```

```
+---+---+---+
|  f| a1| a2|
+---+---+---+
|010|  a|  1|
|011|  b|  4|
+---+---+---+
```

2.4.2 User-Defined Functions (UDF)

Se trata de un mecanismo para convertir una función convencional escrita en Python, R, Java o Scala en una función que puede ser llamada por los executors sobre cada fila de un DF de Spark, procesando los valores de una o varias columnas concretas.

Primero escribimos la función en Python que procesa un solo elemento, y luego la envolvemos en un objeto UDF que permite invocarla sobre una columna completa. Una vez hecho esto, cuando la utilicemos Spark automáticamente enviará el código de nuestra función a los executors para que la invoquen una vez por cada fila. Cada executor invoca a nuestra función sobre las porciones (particiones) de datos presentes en el executor. Sería el análogo a la función *map* que se suele usar con RDDs para transformar cada fila, pero aplicado a DataFrames.

Se utiliza habitualmente dentro de **withColumn()** para *añadir una nueva columna a un DF existente,* con el resultado de aplicarle a cada fila nuestra función, en la que intervienen solo ciertas columnas de la fila. A la hora de implementar la función de Python que después será envuelta por el objeto *udf*(), debemos pensar en una función que procesa valores individuales pasados como argumento, sin tener en cuenta que después esos valores pertenecerán a una misma fila de un DataFrame. Hecho esto, la envolveremos con el objeto UDF mediante una llamada a la función *udf*() de pyspark.sql.functions, y a partir de ese momento, podremos invocar dicho objeto sobre columnas completas (objetos Column) en lugar de sobre valores individuales.

Ejemplo:

```
from pyspark.sql import functions as F
from pyspark.sql import types as T

# Definimos una función de Python que multiplica un número por 1.6
def milesToKm(miles):
    """
    Función que traduce a km el número de millas que pasamos como argumento

    :param miles: distancia en millas
    :returns: distancia traducida a km
    """
```

(continues on next page)

(continued from previous page)

```
    return(1.6 * miles) # multiplica un número por 1.6

# Ahora la envolvemos en un objeto UDF, especificando el tipo devuelto

toKmUDF = F.udf(milesToKm, T.DoubleType())  # indicamos el tipo de dato del valor␣
↪devuelto por la función

# Finalmente, la utilizamos, pasándole una columna completa

result_df = flightsDF.withColumn("DistKm", toKmUDF(F.col("Distance")))
result_df.select("FlightDate", "Origin", "Dest", "Distance", "DistKm").distinct().
↪show(10)
```

```
+----------+------+----+--------+------+
|FlightDate|Origin|Dest|Distance|DistKm|
+----------+------+----+--------+------+
|2018-01-18|   DTW| BTV|   537.0| 859.2|
|2018-01-17|   DTW| SAT|  1214.0|1942.4|
|2018-01-04|   BQK| ATL|   238.0| 380.8|
|2018-01-05|   SYR| LGA|   198.0| 316.8|
|2018-01-31|   STL| CVG|   308.0| 492.8|
|2018-01-07|   CLT| JFK|   541.0| 865.6|
|2018-01-01|   FLL| PHL|   992.0|1587.2|
|2018-01-01|   MCO| PVD|  1072.0|1715.2|
|2018-01-01|   SAN| FLL|  2269.0|3630.4|
|2018-01-02|   BOS| MSY|  1368.0|2188.8|
+----------+------+----+--------+------+
only showing top 10 rows
```

Al invocar a la UDF (recordemos que *withColumn* es una transformación, por lo que sólo se ejecutará cuando encontremos una acción más adelante), Spark se ocupará de:

- Serializar el código Python de la función *milesToKm*.
- Enviarlo por la red a los workers en los que existan particiones del DF *flightsDF*.
- Para cada fila de la partición, transformar el tipo de dato de la columna Distance, que será Integer de Java (Scala), al tipo entero de Python, y entonces invocar a *milesToKm* pasándole los valores convertidos a Python. Esta operación implica un coste computacional al convertir tipos de un lenguaje (Java) a otro (Python) pero es inevitable en las UDF de Python, salvo en la versión 2.4 en los casos que puedan solucionarlo las PandasUDF[^7].

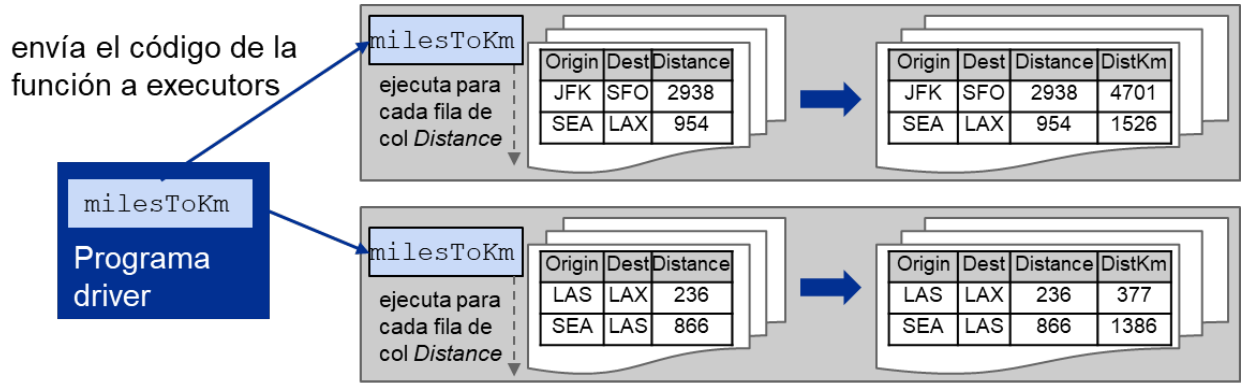


Fig. 7. Ejecución en los executors de la UDF DistKm que envuelve a milesToKm

En el ejemplo anterior, la UDF es muy sencilla y se limita a multiplicar todos los valores por 1.6. En casos donde

la transformación sea simple, debemos primero tratar de implementarla combinando **funciones ya existentes de pyspark.sql.functions**. En este caso podríamos simplemente haber hecho

```
flightsDF.withColumn("DistKm", 1.6 * F.col("Distance"))
```

puesto que se puede hacer aritmética de columnas, y el operador producto existe para columnas y una constante. La mayoría de las veces, incluso en transformaciones complejas, es posible expresarlas utilizando combinaciones de funciones de Spark, y esa implementación **siempre será más eficiente que una UDF**, puesto que una UDF es una caja negra a efectos del optimizador Catalyst y no puede ser optimizada, mientras que las funciones de Spark sí están sujetas a optimizaciones incluso combinadas entre sí.

2.4.3 Ejecución de código SQL puro

Además de la API estructurada de Spark SQL, es posible expresar transformaciones sobre DataFrames utilizando lenguaje SQL puro. Para ello, primero hay que registrar el DataFrame como tabla de Hive. Generalmente se registra como vista temporal volátil para esta sesión, lo cual no crea ningún archivo en almacenamiento físico sino que solo añade metadatos temporales al metastore de Hive, pero no se replican los datos del DF existente. Tras esto, se invoca el método **.sql("códigoSQL")** sobre el objeto SparkSession, pasando como argumento un string con la consulta SQL completa, que puede ser tan compleja como necesitemos. Spark automáticamente es capaz de analizarla y traducirla a trabajos de Spark que se ejecutan de forma distribuida y paralela en el cluster sobre nuestro DF.

```
# Registramos nuestro DF como una table temporal llamada flights

flightsDF.createOrReplaceTempView("flights")
result_df = spark.sql("select max(ArrDelay) from flights group by Origin")
result_df.show(10)
```

```
+-------------+
|max(ArrDelay)|
+-------------+
|       1209.0|
|        292.0|
|       1004.0|
|       1197.0|
|        607.0|
|       1432.0|
|       1205.0|
|       1091.0|
|       1249.0|
|         59.0|
+-------------+
only showing top 10 rows
```

Podemos salvar un DataFrame como una tabla persistente de Hive con el siguiente comando. En ese caso se guardará en el almacenamiento que Hive tenga configurado (suele ser HDFS por defecto) y en la ruta que tenga configurado (alguna ubicación concreta de HDFS, generalmente colgante de /hive).

```
flightsDF.write.saveAsTable("flightsTable")
```

2.4.4 API de lectura y escritura de DataFrames

La lectura y escritura de DataFrames de todo tipo de fuentes de datos se realiza mediante una API uniforme: es otra de las **grandes fortalezas** de Spark, ya que existen **conectores** para casi cualquier fuente de datos:

- Ficheros almacenados en Amazon S3, Google Cloud Storage, HDFS …
- Formatos de archivo como TXT, CSV, JSON, XML, libsvm
- Bases de datos relacionales, para las que se usa el protocolo JDBC habitual (MySQL, PostgreSQL, Oracle, Microsoft SQL server…)
- Bases de datps No-SQL como MongoDB, Cassandra, HBase
- Conector para el índice inverso de ElasticSearch, …
- Fuentes de datos en streaming, por ejemplo una cola de Kafka

El objeto `spark.read` es un objeto de la clase `DataFrameReader` y realiza las lecturas. Podemos configurar varias opciones mediante `option("key", "value")`. Existe una opción genérica `format("formato")` donde "formato" es "csv", "jdbc", "mongo", "parquet", "json" … y termina con el método `load("/path/to/file")`. Los formatos frecuentes tienen un atajo para no usar load, ej: `csv("/path/to/file.csv")`.

La lectura **siempre devuelve un DataFrame**, independientemente de la fuente de datos, lo cual facilita mucho su uso. Entre otras opciones, podemos pasarle el esquema explícito en el momento de hacer la lectura, con la opción .schema(myschema). Para ficheros CSV podemos indicar qué hacer con las filas mal formadas (descartarlas, fallar, o devolverlas a null), y también cuál es el carácter delimitador que separa los campos mediante `.option("delimiter", "\|")` donde estamos indicando en este caso que es `|`.

En archivos en formato Parquet y ORC, el tipo de dato de cada columna se almacena en el propio fichero, por lo que los tipos leídos siempre son correctos y coinciden exactamente con los que se guardaron. En otros casos, Spark puede intentar inferir el tipo de dato de cada columna, aunque se le puede indicar que no lo haga (y en ese caso leerá todo como string), o darle el esquema. Los CSV son más problemáticos. Spark no admite separador de más de un carácter, y además no permite guardar tipos compuestos (columnas cuyo tipo de dato es una tupla, o es una lista), mientras que Parquet sí.

Formato general:

```
myDF = spark.read.format("format")\
            .option("key","value")\
            .schema(mySchema)\
            .load("/path/to/file")
```

Ejemplos:

```
flightsDF = spark.read.format("csv")\
                 .option("mode", "FAILFAST")\
                 .option("header", "true")\
                 .option("inferSchema", "true")\
                 .option("path","flights.csv")\
                 .load()
```

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

flightsSchema = StructType([ \# creamos un esquema
                           StructField("Origin", StringType(), nullable = True),
                           StructField("ArrDelay", IntegerType(), nullable = True),
                           #  resto de campos del esquema
                          ])
```

(continues on next page)

(continued from previous page)

```
flightsDF = spark.read.schema(flightsSchema)\
                      .option("mode", "DROPMALFORMED")\\
                      .csv("flights.csv")
```

Otro ejemplo: lectura de una base de datos MongoDB (ver detales en https://www.mongodb.com/docs/spark-connector/current/batch-mode/batch-read-config)

```
collection_df = spark.read.format("mongodb")\
                          .option("connection.uri", "mongodb://localhost:27017")\
                          .option("collection", "collection_name")
```

La escritura de un DF la realiza el objeto **myDF.write**, de la clase DataFrameWriter.

Se le pasan opciones mediante .option("key", "value"), y la opción .format("formato") donde casi todos los formatos frecuentes tienen ya un atajo para no tener que usarlo. Termina con el método **save**("/path/to/file") que, cuando es HDFS, **crea un nuevo directorio**. En ese directorio **se crean tantos ficheros como particiones tenga el DataFrame**. Algunas opciones interesantes:

- **mode**("modo"): modo es "overwrite" ó "append" ó "error" (por defecto), ó "ignore"
- **partitionBy**() cuando guardamos en formato Parquet, crea una estructura de directorios en base a los valores de una variable categórica que acelera operaciones de filtrado cuando leamos de nuevo el fichero.
- **saveAsTable**() en lugar de save() al final, para guardar el DF físicamente en el formato y ubicación que tenga configurado Hive (en el caso de Databricks se guarda en formato Delta sobre DBFS), y añadir los metadatos correspondientes al metastore de Hive.

Ejemplos:

```
flightsWeekendDF.write.partitionBy("Origin")\
                      .format("parquet")\
                      .save("/path/new/folder")
```

La parte .format().save() se puede sustituir por .parquet("/path/folder"). En ese directorio Spark creará subcarpetas llamadas "Origin=JFK", "Origin=LAX", etc y dentro de cada subcarpeta se guardarán los datos de esa partición, creando dentro tantos ficheros como particiones físicas tenga el DF que estamos salvando.

Otra opción es salvar físicamente los datos en una tabla del metastore de Hive, en el formato que venga configurado por defecto y en la ubicación que venga configurada por defecto en Spark. Habitualmente el formato de fichero es Parquet, aunque si ejecutamos en la plataforma de Databricks, el formato será Delta. La ruta depende de lo que indique la propiedad `spark.sql.warehouse.dir` y suele tener el valor `/user/hive/warehouse` o `/hive/warehouse/` o algo similar.

```
# Salvamos físicamente los datos en una tabla Hive en Parquet
flightsWeekendDF.write.**saveAsTable**("flightsWeekends")
```

Lectura y escritura de BBDD relacionales

Spark puede leer de cualquier BBDD relacional, como MySQL, PostgreSQL, Oracle, SQL Server…mediante el protocolo habitual **JDBC.** Spark hace **predicate pushdown**: una operación de filtrado de Spark puede traducirla automáticamente en un filtrado nativo del datasource. Hay reglas de cuándo será capaz y cuándo no. También es posible hacerlo con archivos Parquet.

Cuando leemos con Spark desde una BBDD relacional, los executors de Spark se conectan en paralelo a la base de datos para leer simultáneamente porciones distintas de la misma tabla al mismo tiempo.

Federación de datos con Spark

Al igual que se puede construir una base de datos federada como una base de datos virtual formada por BBDD heterogéneas, de forma transparente al usuario, es posible usar Spark para construir DataFrames a partir de fuentes de datos diversas, y después manipularlos y cruzarlos mediante código Spark, sin importar su origen. Spark actúa como una **capa de abstracción común** y lleva a cabo las operaciones que no puedan ser traducidas a consultas nativas



2.4.5 Jobs, stages y tasks (trabajos, etapas y tareas)

- Un **job** (trabajo) de Spark es todo el procesamiento necesario *para llevar a cabo una acción del usuario*

Por ejemplo: df.count(), df.take(4), df.show(), df.read(…), df.write(…), etc). Cada *job* se divide en una serie de *stages* (etapas).

- Un **stage** (etapa) es todo el procesamiento que puede llevarse a cabo *sin mover datos entre nodos*. Cada nodo hace exactamente el mismo procesamiento aplicado a diferentes particiones del mismo DataFrame que están procesando todos.

Cuando hay una operación que implica movimiento de datos (*shuffle*), finaliza un stage y se crea otro nuevo. Ej: df.join(…), df.groupBy(…).agg(…)

- Una **task** (tarea) de Spark es el procesamiento aplicado por un core físico (CPU) a una partición de un RDD. La tarea es la unidad mínima de trabajo de Spark.

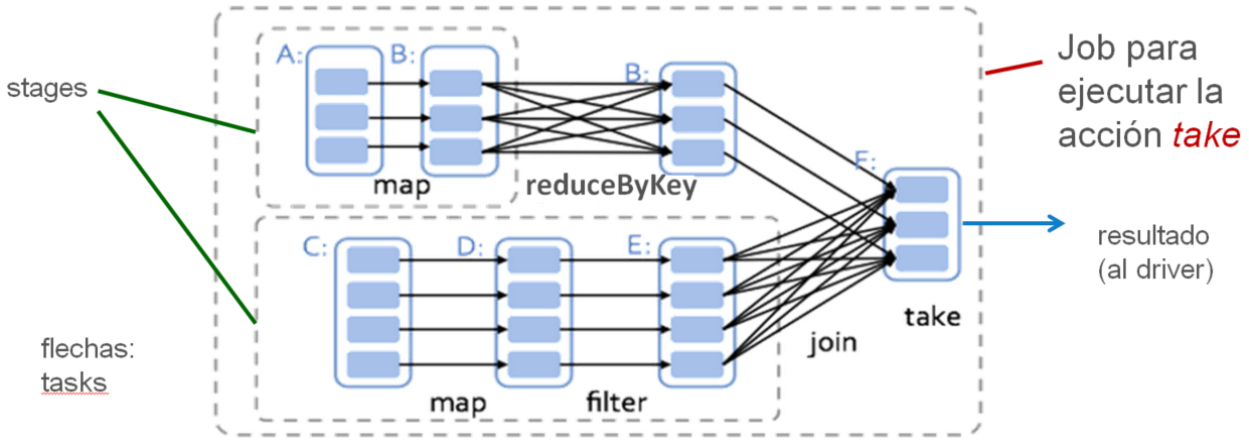


Fig. 9. Representación de Jobs, stages y tasks en Spark

2.5 Bibliografía recomendada

- Bill Chambers y Matei Zaharia. *Spark: The Definitive Guide* O'Reilly, 2018.
- La web oficial de Spark, https://spark.apache.org/docs/latest/ contiene una extensa documentación.
- Más detalles sobre jobs, staes y tasks en la doc. oficial: https://spark.apache.org/docs/latest/job-scheduling.html
- RDD programming guide: http://spark.apache.org/docs/latest/rdd-programming-guide.html
- Spark SQL programming guide: http://spark.apache.org/docs/latest/sql-programming-guide.html
- Fuente: Apache Spark Analytics Made Simple – Highlights from the Databricks Blog
- Pandas-on-Spark (pyspark-pandas): http://spark.apache.org/docs/latest/sql-pyspark-pandas-with-arrow.html

CHAPTER

THREE

CAPÍTULO 3: CATÁLOGO DE METADATOS DE HIVE

3.1 Antecedentes: Apache Hive

La tecnología Apache Hive fue desarrollada inicialmente por Facebook en al año 2007, y donado a la Apache Software Foundation en 2010 para convertirlo en un proyecto open-source. Podemos definir Apache Hive como:

Note: Sistema de Data Warehouse para manejar (leer, escribir, ETL, realizar reporting y análisis) datos almacenados de manera distribuida utilizando lenguaje SQL.

Hive proporciona un armazón SQL para construir tablas sobre ficheros de datos ya existentes, probablemente masivos, estructurados, que pueden estar en varios formatos como Parquet -formato más frecuente-, CSV, RCFile, ORC, …) almacenados en diversas fuentes (la más frecuente era HDFS aunque posteriormente se añadieron otros sistemas de almacenamiento distribuido soportados por Hive, como Amazon S3). Las tablas son en realidad metadatos sobre estructura y tipos de datos, y localización física de los datos a los cuales apunta la tabla. Se introdujo cuando aún no existía Spark, como una manera de ganar abstracción y facilitar el uso de MapReduce de una manera menos árida y complicada de aprender como es SQL. La creación de tablas se efectuaba también mediante sentencias SQL de creación de tablas que incluían opciones indicando la ruta de los ficheros de datos que soportan a esa tabla.

Note: Entre 2016 y 2019 muchas empresas montaron su lago de datos como tablas Hive definidas sobre ficheros Parquet en HDFS, seleccionaron Spark como motor y conectaron sus sistemas de BI a Hive. Es una arquitectura muy frecuente en sistemas legacy

---

Se trataba de una herramienta para acceder fácilmente a los datos usando exclusivamente lenguaje SQL, para uso principalmente por parte de analistas de Business Intelligence que conocen bien sus datos y el negocio, pero no dominan la programación, y para tareas como ETL, reporting y análisis de datos.

El lenguaje de consulta que utiliza se denomina HiveQL, que permite ejecutar prácticamente cualquier sentencia de SQL estándar y además tiene soporte para funciones a nivel de fila definidas por el usuario (User Defined Functions, UDF), y tipos de datos complejos.

Note: Con Spark es posible crear una UDF en cualquier lenguaje, registrarla permanentemente en el catálogo de Hive y utilizarla desde lenguaje SQL

Se utilizaba principalmente en modo OLAP (Online Analytical Processing), es decir, para realizar análisis en batch del informacional (histórico) de la empresa. Las consultas SQL no necesariamente se esperaba que fuesen interactivas: se programaban incluso ETL's con Hive en SQL. No es adecuado para OLTP (Online Transactional Processing) ya que no está pensado para manejo interactivo con respuestas rápidas ni para transaccionalidad. La ejecución de la consulta SQL la llevaba a cabo un motor de ejecución externo, intercambiable, que originalmente fue MapReduce pero después se podía

configurar para apuntar a Spark o Apache Tez. Hive tiene un analizador SQL que traduce la consulta a jobs del motor de ejecución seleccionado.

Varias características de Hive lo diferencian de una Base de Datos pura, como la transaccionalidad (añadida después pero que penaliza mucho) o la imposibilidad de modificar datos, así como su filosofía de procesar la tabla como un todo, sin buscar la eficiencia en acceder a un registro concreto.

3.1.1 Relación entre Hive y Spark

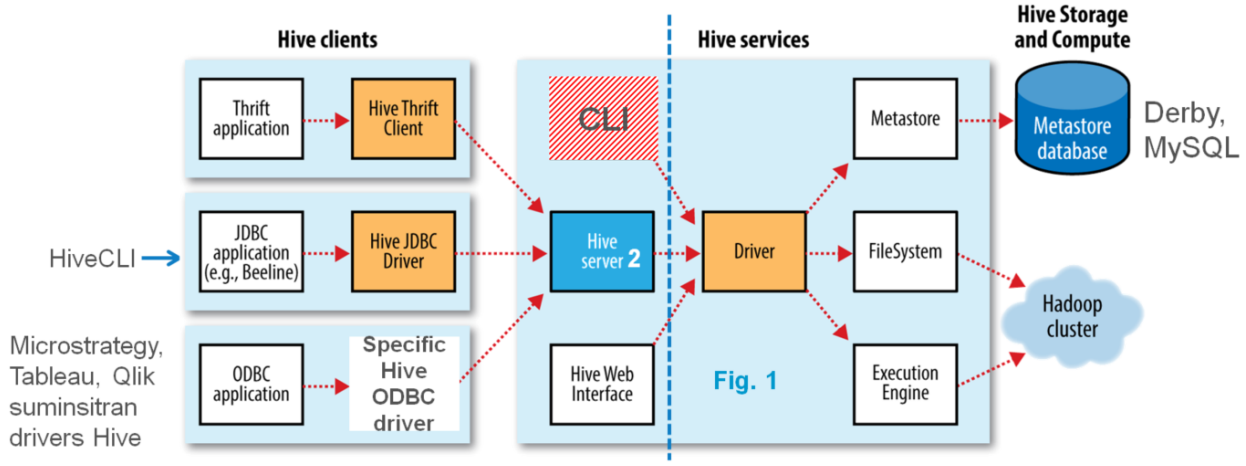
La estructura de tabla se define creando metadatos en un formato concreto y almacenados en el catálogo de Hive, que habitualmente se almacena en una base de datos relacional convencional (por ejemplo MySQL para sistemas en producción, o Apache Derby como base de datos extremadamente ligera y compacta). A pesar de que no utilizaremos Apache Hive, el catálogo de metadatos creado por Spark se sigue llamando catálogo de Hive, aunque hay quien lo denomina "catálogo de Spark".

Note: Apache Spark interactúa con el catálogo de metadatos de Hive (Hive metastore), ya sea con uno existente o creado por Spark, ya que entienden el mismo formato de metadatos.

Note: Spark incluye un analizador SQL para traducir SQL a jobs, y puede crear tablas a partir de ficheros de datos existentes, pero además permite definir vistas temporales sobre DataFrames en memoria. Por tanto, es posible definir un Data Warehouse utilizando Spark

---

Aunque ya no se utiliza, la manera en la que habitualmente se enviaban a Hive consultas SQL estaba basada en la arquitectura mostrada a continuación.



Arriba: maneras de utilizar Apache Hive (legacy)

Thrift Server 2 es un proceso demonio que está siempre levantado y escuchando en un puerto, a la espera de consultas. En la actualidad, es frecuente hacer algo similar pero sin Hive, utilizando exclusivamente Spark, ya que es posible conectarse mediante ODBC al catálogo de metadatos de Hive creado por Spark "como si fuese una base de datos" en la que las consultas son ejecutadas siempre por Spark. Esto es posible gracias al Thrift Server que incorpora Spark, y que se detalla en https://spark.apache.org/docs/latest/sql-distributed-sql-engine.html

De hecho, es habitual ver Hive y también Spark como fuentes de datos (data sources) disponibles en las aplicaciones de Business Intelligence más populares, como Microstrategy, QlikSense, PowerBI, etc. En la actualidad, la conexión al Catálogo de Hive incluido en Databricks se lleva a cabo de forma similar, aunque Databricks ha anunciado conectores

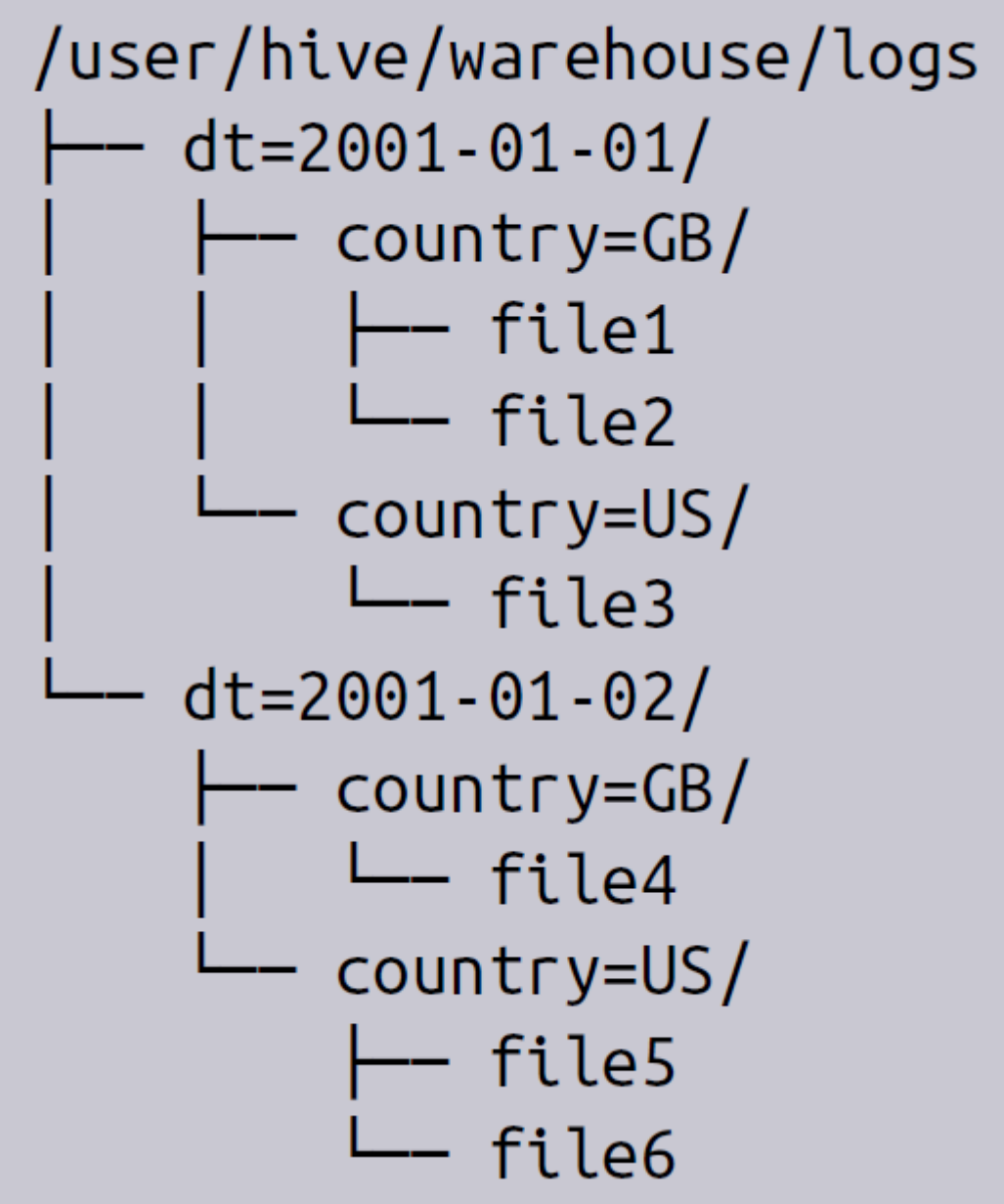
específicos para las herramientas más habituales de BI a través de su funcionalidad Partner Connect. Pueden encontrarse detalles en https://www.databricks.com/partnerconnect.

3.2 Unidades de información en el metastore de Hive

Estas son las distintas unidades de información que maneja Hive:

- Tabla: físicamente es un directorio de HDFS que almacena los datos de una tabla
- Partición: corresponde a un subdirectorio de HDFS dedicado al subconjunto de filas que tienen un cierto valor en una columna (la cual en realidad no forma parte de los metadatos)
- Bucket (compartimentos): grupos de filas de una tabla que no necesariamente comparten el valor de una columna, pero los valores de dicha columna han sido agrupados en unos pocos compartimentos (menos que los valores existentes). Hace más eficientes las consultas, incluyendo los JOIN si los atributos de JOIN están bucketizados en ambas tablas.

La siguiente figura muestra una tabla llamada logs particionada en base a dos de sus columnas.



Jerarquía de directorios donde se almacena la tabla logs particionada por las columnas dt y country

3.2.1 Tablas externas y tablas manejadas

Spark soporta los conceptos de tabla externa y tabla manejada que introdujo Hive:

- Tabla manejada (managed): Spark es totalmente dueño de ella. El borrado de la tabla implica borrado físico de los datos contenidos en la tabla. La ubicación de física de los datos para las tablas manejadas se configura en un parámetro de Spark y por eso no es necesario especificar ninguna ruta cuando queremos guardar un DataFrame de Spark como tabla manejada.
- Tabla externa (external): Spark sólo genera metadatos para ficheros de datos ya existentes. El borrado de la tabla solo borra metadatos del metastore de Hive, pero no borra los datos físicos, puesto que estos ya existían desde antes de la creación de la tabla en sí, así que Spark considera que la tabla no le pertenece.

Puedes comprobar el funcionamiento de las tablas manejadas y externas en el notebook en el que manipulamos el catálogo de Hive. En él, primero creamos un DataFrame de Spark en memoria a partir de un fichero CSV, y procedemos a guardar el DataFrame:

- Primero, como como tabla (vista) temporal de Spark, que no guarda datos físicos sino que solamente crea en el metastore de Hive los metadatos necesarios para que la tabla pueda ser consultada mediante lenguaje SQL (incluso desde aplicaciones externas) durante esta sesión. Cuando cerremos el notebook y muera el proceso driver, la tabla desaparecerá y sus metadatos se borrarán del catálogo de Hive. Las vistas temporales son tablas no manejadas. Si hacemos un borrado de la tabla, no estamobs borrando datos ya que los datos ni siquiera han llegado a guardarse (a escribirse) en ningún almacenamiento persistente (ni HDFS ni ningún otro).

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.getOrCreate()
flightsDF = spark.read.option("inferSchema", "true").option("header", "true").csv(
 ↪"flights.csv")

flightsDF.createOrReplaceTempView("flights")  # esta vista desaparece cuando finalice␣
 ↪el driver (y por tanto nuestra aplicación)
                                              # La vista son solamente metadatos, que␣
 ↪son volátiles
consulta_df = spark.sql("select * from flights where Distance > 5000")  # el método␣
 ↪sql es lazy ya que hace una transformación
```

- Después, como tabla manejada, que guarda por un lado los datos físicamente en una ubicación física ya pre-configurada en una variable de entorno de Spark (que hace referencia a la ubicación donde se guardan los datos físicos desde Spark), y además crea los metadatos correspondientes en el catálogo de Hive. La tabla en este caso pertenece a Hive y se denomina manejada por Hive. Si hacemos un borrado de la tabla, estaremos borrando tanto los metadatos como los datos. En la plataforma de Databricks, el formato por defecto es Delta, y los datos se guardan dentro del contenedor de ADLS de la Storage Account privada que ha creado Databricks. En cambio en una instalación local en nuestro portátil, el formato habitualmente será Parquet y la ubicación por defecto será dentro de la carpeta spark-warehouse que Spark ha creado automáticamente.

```
# Guarda el DF como tabla manejada en una ubicación y formato ya configurados por␣
 ↪defecto para Spark. Le indicamos que la tabla debe crearse
# en la database "default" que es la única que existe por defecto. El concepto de␣
 ↪database es equivalente a "esquema", entendido como un conjunto de tablas que␣
 ↪guardan
# relación entre sí por la temática (por ejemplo, la database "financiero" donde se␣
 ↪encuentran todas las tablas con información de aspectos económicos de una empresa).
# Es posible crear una nueva database con el comando spark.sql("CREATE DATABASE IF␣
 ↪NOT EXISTS nombre_database").

flightsDF.write.option("mode", "overwrite").saveAsTable("default.vuelos")
```

- Por último, crearemos una tabla externa a partir de un fichero de datos CSV que ya existía en HDFS (nótese que pueden crearse tablas externas también a partir de otros sistemas de almacenamiento distintos a HDFS). Puesto que los datos que forman parte de la tabla ya existían con anterioridad a la creación de la tabla, Hive considera que dicha tabla no le pertenece y se denomina externa. Se crearán los metadatos y se escribirán en el catálogo de Hive, y esto será permanente incluso tras cerrar la sesión actual. Entre esos metadatos se encontrará almacenada la ubicación de los datos contenidos en la tabla. El borrado de la tabla no borrará los datos (el fichero CSV) sino que solamente borrará los metadatos que se crearon en el catálogo en el momento de crear la tabla.

```
# Opción 1: creamos tabla externa a partir de un fichero existente. La opción path le␣
 ↪indica que ya existe y por eso la crea como externa.
```

(continues on next page)

(continued from previous page)

```
# Si el fichero es CSV, hay que indicar manualmente el esquema al crearla. Si es un␣
 ↪Parquet, puede inferir el esquema automáticamente.
# Para que no sea tedioso, vamos a guardar el CSV como Parquet y a crear la tabla␣
 ↪desde ahí

flightsDF.write.mode("overwrite").parquet("C:/flights")
spark.sql("create table if not exists default.vuelos_externa using Parquet location
 ↪'C:/flights'")
result_df = spark.sql("select * from default.vuelos_externa where Distance > 5000")
```

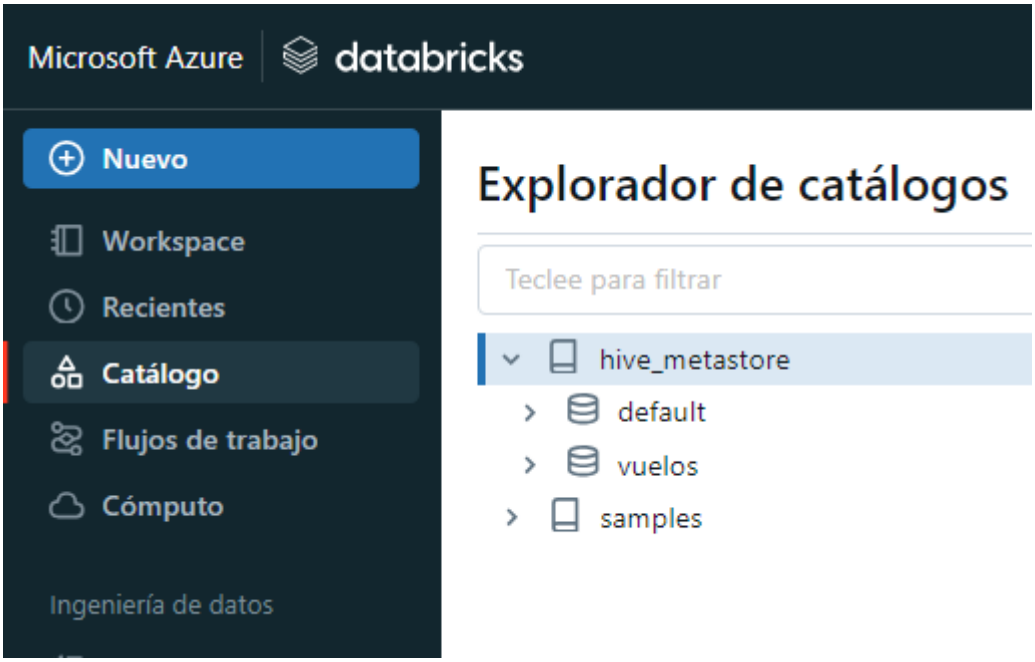
```
# Opción 2: salvamos el DataFrame en la ubicación que queremos, y sobre ella creamos␣
 ↪una tabla externa sobre la marcha
# Todo en una única operación. Spark entiende que es tabla externa porque hemos␣
 ↪puesto option path

flightsDF.write.mode("overwrite").option("path", "C:/flights_folder_ext").saveAsTable(
 ↪"default.vuelos_ext2")
consulta_df = spark.sql("select * from default.vuelos_ext2 where Distance > 5000")
```

3.3 Databricks y el catálogo de Hive

Tradicionalmente, Databricks ofrecía simplemente una interfaz gráfica al catálogo de metadatos de Hive para poder ver las tablas existentes y los metadatos de las tablas de manera gráfica, como vemos en la parte izquierda. Además, en 2022 introdujo la funcionalidad llamada Unity Catalog, que añade un nivel de abstracción al metastore, en el que se pueden compartir tablas y databases entre diferentes workspaces de Databricks (y por tanto, incluso entre diferentes empresas). Por ejemplo, ciertos usuarios pueden tener acceso solamente a un workspace con sus peculiaridades, su propio DBFS, etc, y a la vez pueden acceder a distintos catálogos que a su vez también pueden ser accesibles desde otros workspaces donde existen otros usuarios. El concepto de catálogo, como conjunto de databases, es transversal. Generalmente se asocia un catálogo a un ámbito de datos determinado, por ejemplo datos productivos, datos para pruebas, datos generados depurando otros datos, datos que son todos predicciones de modelos, etc. O incluso datos de una empresa concreta perteneciente a un holding de empresas que, siendo distintas, comparten solamente ciertos datos o modelos.

Una tabla se nombra con 3 partes, nombreCatálogo.database.tabla. Además, Unity introduce muchas funciones adicionales, como una gestión totalmente centralizada del permiso de acceso a los datos, que antes se delegaba en Azure y se aplicaba a cada workspace por separado, mientras que ahora se centraliza la configuración deseada de cada workspace. También la posibilidad de ver el linaje del dato: para cada columna de cada tabla, Unity Catalog muestra la columna o columnas de la(s) tabla(s) a partir de las cuales está construida. Aprovechando el DAG de Spark, que intrínsecamente mantiene traza de todos los DataFrames (en realidad, de los RDD) de la aplicación, es posible traducir su estructura en un linaje del dato que se queda guardado.


Microsoft Azure
databricks
Nuevo
Workspace
Recientes
Catálogo
Flujos de trabajo
Cómputo
Ingeniería de datos
Explorador de catálogos
Teclee para filtrar
hive_metastore
default
vuelos
samples

CHAPTER

FOUR

CAPÍTULO 4: SPARK MLLIB

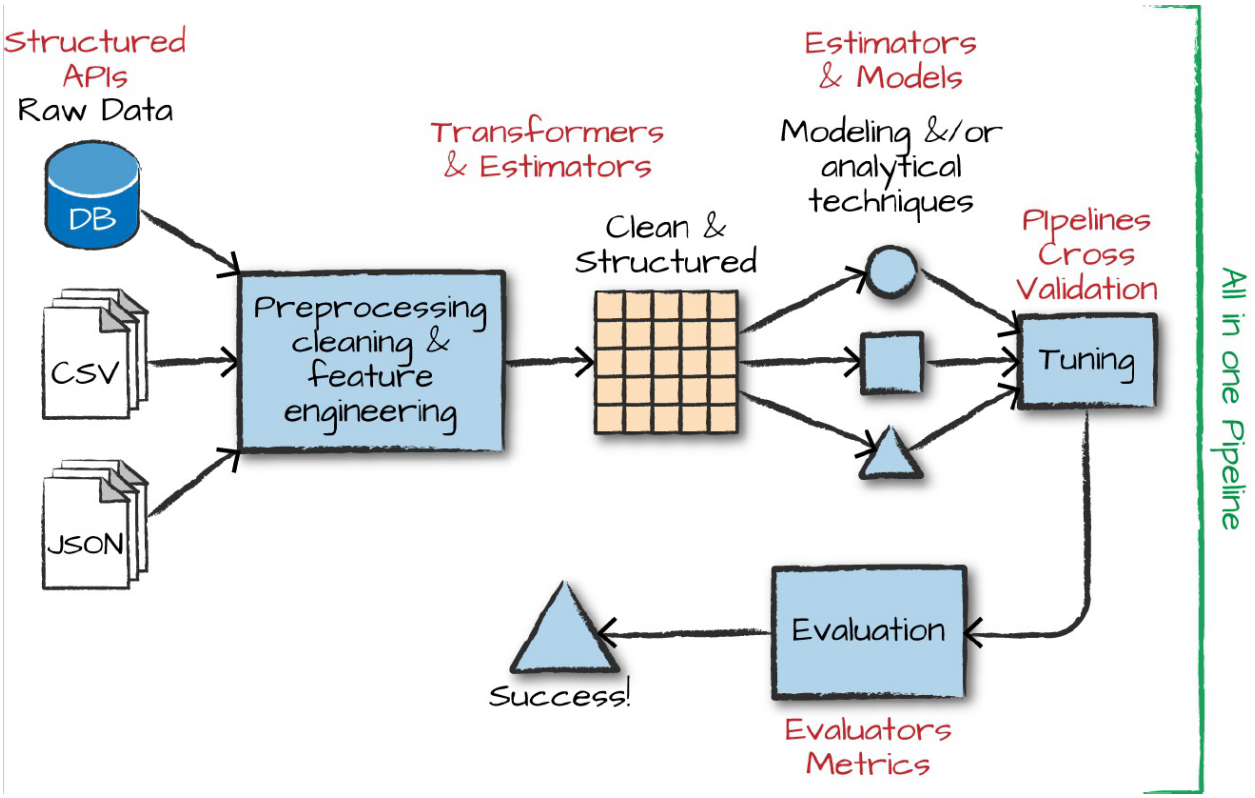
Spark MLlib es el módulo de Spark para tareas de

- Limpieza de datos
- Ingeniería de variables (creación de variables desde datos en crudo)
- Aprendizaje de modelos sobre datasets muy grandes (distribuidos)
- Ajuste de parámetros y evaluación de modelos

No proporciona métodos para despliegue en producción de modelos entrenados. En la actualidad, es muy frecuente desplegar modelos como microservicios que usan el modelo entrenado para realizar predicciones sobre un nuevo ejemplo que han recibido por medio de una llamada HTTP de una aplicación externa que les ha enviado el dato para predecir. Comentaremos más detalles en la próxima sección.

La esencia de MLlib es la implementación de modelos de manera distribuida utilizando Spark, y capaces de aprender sobre datasets muy grandes almacenados de manera distribuida. Desde el punto de vista conceptual, los modelos predictivos que ofrece MLlib son los habituales de una biblioteca estándar de Machine Learning, tanto de aprendizaje supervisado como no supervisado. La diferencia con respecto a otros paquetes que también implementan estos métodos, como por ejemplo scikit-learn en Python o caret en R, es que **la implementación se ha realizado desde 0 utilizando operaciones distribuidas** (en especial, operaciones de manipulación de un DataFrame de Spark, las cuales son siempre distribuidas). Como resultado, obtenemos una implementación distribuida de ese algoritmo abstracto. Por supuesto, una implementación que se hizo no-distribuida, como por ejemplo la regresión logística de scikit-learn que en cada paso de la implementación utiliza operaciones no distribuidas, no podrá nunca convertirse automáticamente en distribuida a pesar de ejecutarla en un cluster, puesto que seguirá siendo ejecutada por el intérprete de python que corre en una única máquina, no importa si esa máquina es nuestro PC portátil o es uno de los nodos de un cluster.

No obstante, también es muy frecuente utilizar solo ciertas funcionalidades de MLlib para pre-procesar variables en datasets masivos, limpiar, normalizar, etc y finalmente filtrar y pasar el dataset resultante (asumiendo que ya no es masivo) al driver para guardarlo en el sistema de archivos local. Posteriormente se puede utilizar una biblioteca de Machine Learning no distribuida, como por ejemplo Scikit-learn de Python o paquetes específicos de R como caret o e1071. Tanto Python como R son capaces de leer ficheros no distribuidos tanto en formato texto como CSV entre otros, y usarlos como entrada para un algoritmo de aprendizaje automático.



La figura anterior muestra una idea general del ciclo de ajuste de un modelo, junto a las herramientas que proporciona Spark para cada etapa (en color rojo). Además, Spark tiene una herramienta adicional llamada pipelines para encapsular todas estas etapas en un solo objeto indivisible, y asegurarnos de que los nuevos datos recibidos y con los que queremos realizar predicciones pasen por exactamente las mismas etapas de pre-procesamiento que el dataset estático con el que se entrenó el modelo. Por eso la figura se refiere a estos pasos como un all-in-one Pipeline.

La etapa de pre-procesamiento incluye, además de las operaciones típicas de un proyecto de Data Science, cierto procesamiento específico de Spark para pasar los datos al formato adecuado de columnas que esperan recibir las implementaciones de cada algoritmo en la API de Spark ML. MLlib incorpora herramientas para esto también.

4.1 Despliegue de modelos en producción con Spark

Spark no fue concebido para explotación online de modelos entrenados, es decir, dar respuestas rápidas (predicciones) a un ejemplo nuevo que se recibe, por ejemplo, desde un sitio web. Por el contrario, la fortaleza de Spark está en entrenar modelos en modo batch (offline) con datasets de gran cantidad de datos.

Aun así, hay varias formas de aprovechar el modelo entrenado obtenido por Spark:

- Entrenar en batch con datos offline almacenados en un sistema de almacenamiento de ficheros como ADLS (probablemente el proceso de creación de variables + entrenamiento llevará un tiempo), y usar el modelo entrenado para predecir (también en modo batch) sobre otro conjunto de datos nuevos preparados para la predicción (esta etapa es mucho más rápida).
  - Es un enfoque muy frecuente en casos de uso reales. Es posible cuando los datos que hay que predecir se conocen al completo porque constituyen una cartera más o menos fija (de clientes, productos, etc) que se actualiza en momentos concretos y a partir de ese momento, debemos calcular una nueva predicción para todos ellos.

  - Las predicciones pre-calculadas se almacenan en ADLS u otro sistema de ficheros, o incluso en una Base de Datos para que sea muy rápido servirlas desde un microservicio si fuese necesario servirlas de esa manera.

- Entrenar, guardar el modelo entrenado, y usarlo desde Spark para hacer predicciones. Poco recomendable: lanzar un job nuevo para cada ejemplo que predecir implica sobrecarga. Balanceo de carga con réplicas del modelo.
- Entrenar y exportar el modelo a un formato de intercambio, ej. PMML, para leerlo y explotarlo con otra herramienta no distribuida (Python en especial aunque también R soporta PMML).
- Entrenar en modo online usando Structured Streaming para recoger datos. Siempre re-entrenar desde 0 salvo si el modelo no está preparado para esto (modelos que soporten online learning, que en la actualidad son una minoría).

4.2 Estimadores y transformadores

Antes de describir en detalle la API del módulo Spark MLlib, hay que tener en cuenta que en la API de Spark (Java/Scala/Python) se distinguen:

- Paquete org.apache.spark.mllib (en Python: pyspark.mllib): API antigua basada en RDDs de una estructura llamada LabeledPoint: `LabeledPoint(etiqueta, [vector de atributos])`. Obsoleta, no debe usarse.
- Paquete org.apache.spark.ml (en Python: pyspark.ml): API actual, sobre DataFrames. Se debe utilizar siempre.

Para la creación de variables y el preprocesamiento, en general se utiliza la API de Spark SQL que vimos en el tema anterior. No obstante, Spark ML ofrece una transformación en la que puede escribirse código SQL arbitrario e incluir dicha transformación en un pipeline. Además, ciertas transformaciones relacionadas con el pre-procesamiento estadístico de datos (normalización, estandarización, codificación one-hot, etc) también están disponibles en Spark ML. Por último, existen varias transformaciones cuyo propósito no es realmente modificar los datos sino prepararlos (dar a las columnas del DataFrame los tipos adecuados) para la entrada de los algoritmos de Machine Learning de Spark. En concreto, Spark requiere estos formatos:

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()

classification_df = spark.createDataFrame([    # Ejemplo de DF para problemas de␣
 ↪clasificación. La clase se codifica
    ([-32.2, 4.5, 1.0, 6.7], 1.0),             # con números reales con la parte␣
 ↪decimal a 0 y empezando en 0.0.
    ([-40.8, 2.25, 4.0, 2.3], 0.0)             # Ejemplos: 0.0, 1.0, 2.0 ...
], ["features", "class"])

regression_df = spark.createDataFrame([        # Ejemplo de DF para problemas de␣
 ↪regresión
    ([-32.2, 4.5, 1.0, 6.7], 2.87),
    ([-40.8, 2.25, 4.0, 2.3], -5.41)
], ["features", "target"])

clustering_df = spark.createDataFrame([        # Ejemplo de DF para problemas de␣
 ↪clustering
    ([-32.2, 4.5, 1.0, 6.7],),
    ([-40.8, 2.25, 4.0, 2.3],)
], ["features"])

recommendation_df = spark.createDataFrame([        # Ejemplo de DF para problemas de␣
 ↪recomendación
    (2, 10, 4.5),                                  # con evaluaciones explícitas
```

(continues on next page)

(continued from previous page)

```
    (3, 8, 3.2)
], ["user", "item", "rating"])

classification_df.show(truncate=False)
regression_df.show(truncate=False)
clustering_df.show(truncate=False)
recommendation_df.show(truncate=False)
```

```
+------------------------+-----+
|features                |class|
+------------------------+-----+
|[-32.2, 4.5, 1.0, 6.7] |1.0  |
|[-40.8, 2.25, 4.0, 2.3]|0.0  |
+------------------------+-----+
```

```
+------------------------+------+
|features                |target|
+------------------------+------+
|[-32.2, 4.5, 1.0, 6.7] |2.87  |
|[-40.8, 2.25, 4.0, 2.3]|-5.41 |
+------------------------+------+
```

```
+------------------------+
|features                |
+------------------------+
|[-32.2, 4.5, 1.0, 6.7] |
|[-40.8, 2.25, 4.0, 2.3]|
+------------------------+
```

```
+----+----+------+
|user|item|rating|
+----+----+------+
|2   |10  |4.5   |
|3   |8   |3.2   |
+----+----+------+
```

Como vemos, los valores de las variables deben presentarse en una sola columna de tipo vector. Por otro lado, si se trata de un problema de aprendizaje supervisado (clasificación o regresión), la columna target debe ser siempre de tipo real (Double). En el caso de los problemas de clasificación, cada clase se indica con un número real empezando en 0.0 y con la parte decimal del número siempre a 0 (es decir, si tenemos un problema con 5 clases, se tienen que codificar como 0.0, 1.0, 2.0, 3.0 y 4.0). En el caso de los problemas de regresión, la columna target puede ser cualquier número real. Los nombres de las columnas no tienen por qué ser features y label como en los ejemplos de arriba, sino que pueden ser cualesquiera, siempre que le indiquemos al algoritmo cómo se llama la columna (de tipo vector) que contiene las features en el DataFrame que le pasamos para entrenar, y cómo se llama la columna target, si la hay.

En el caso de algoritmos de recomendación, Spark solo permite filtrado colaborativo . En ese caso hay que pasarle un DataFrame que contenga, al menos, tres columnas con los identificadores de un usuario, un ítem y el rating que ha dado ese usuario a ese ítem, ya sea implícito o explícito. Los identificadores de usuarios y de ítems tienen que ser códigos de tipo entero, mientras que el rating puede ser un número real en cualquier rango.

Según la documentación oficial de Spark, actualmente nos ofrece las siguientes posibilidades para preprocesamiento, divididas en varios grupos:

- Extracción: extraer variables a partir de datos en crudo

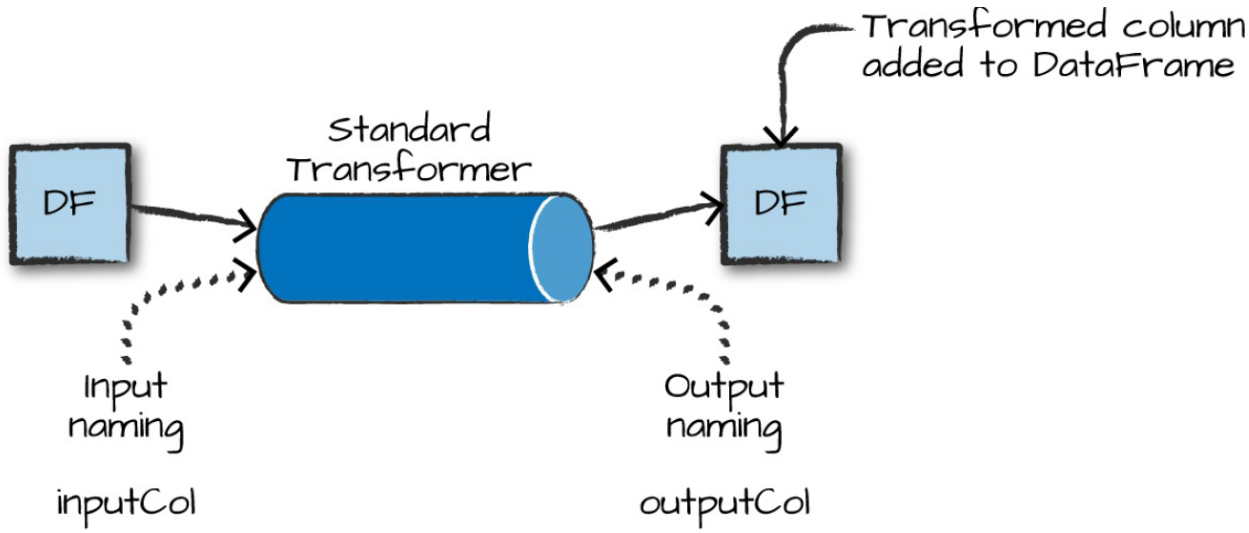
- Transformación: re-escalar, convertir o modificar variables
- Seleccionar: seleccionar un buen subconjunto de variables de otro más grande
- Locality Sensitive Hashing (LSH): combinación de trasnformaciones de variables con otros algoritmos.

Para más detalles, puede consultarse la página de la documentación de Spark sobre piezas de preprocesamiento en https://spark.apache.org/docs/latest/ml-features.html

4.2.1 Transformadores de Spark ML

Un *transformer* es un objeto que recibe como entrada un DataFrame de Spark y uno o varios nombres de columna existentes (por ejemplo inputCol), y las transforma de alguna manera. Su salida es el mismo DataFrame con una nueva columna añadida a por la derecha, con el nombre que hayamos indicado en el parámetro correspondiente (generalmente outputCol pero a veces puede ser predictionCol).

La interfaz Transformer tiene un único método: transform(df: DataFrame) que recibe un DataFrame y devuelve otro DataFrame. Los transformadores no necesitan aprender ningún parámetro del DataFrame de entrada. Desde el momento de ser creados, ya están preparados (tienen toda la información) para transformar un DataFrame, y esto es lo que hacen cuando llamamos a `transform(df)` tal como muestra la siguiente figura. La única condición es que el DataFrame que le pasamos como argumento contenga la(s) columna(s) que ese transformador tiene configuradas para transformar. No importa qué otras columnas adicionales existan en el DataFrame - eso sí, no puede contener columnas que tengan el mismo nombre que las que el transformador va a añadir nuevas al DataFrame resultante de la transformación.



Vemos a continuación algunos transformadores frecuentes, aunque la lista de todos los disponibles es mucho más larga, tal como puede consultarse en la documentación de Spark mencionada más arriba para las piezas de preprocesamiento.

VectorAssembler

Recibe varias columnas y las concatena en una sola de tipo vector, de longitud igual al número de columnas que se quieran ensamblar. Necesario para calcular la columna (única) de features en los algoritmos de aprendizaje supervisado. IMPORTANTE: sólo puede ensamblar columnas numéricas o de vectores numéricos, por lo que cualquier columna categórica debe haberse convertido previamente en numérica con `StringIndexer`. Ejemplo:

```
from pyspark.ml.linalg import Vectors
from pyspark.ml.feature import VectorAssembler
```

(continues on next page)

(continued from previous page)

```
dataset = spark.createDataFrame(
    [(0, 18, 1.0, Vectors.dense([0.0, 10.0, 0.5]), 1.0)],
    ["id", "hour", "mobile", "userFeatures", "clicked"])

assembler = VectorAssembler(inputCols=["hour", "mobile", "userFeatures"],
                            outputCol="features")

output = assembler.transform(dataset)

print("DataFrame original:")
dataset.show()

print("DataFrame resultado de ensamblar hour, mobile, userFeatures en la columna␣
 ↪features:")
output.show(truncate=False)
```

```
DataFrame original:
```

```
+---+----+------+--------------+-------+
| id|hour|mobile|  userFeatures|clicked|
+---+----+------+--------------+-------+
|  0|  18|   1.0|[0.0,10.0,0.5]|    1.0|
+---+----+------+--------------+-------+

DataFrame resultado de ensamblar hour, mobile, userFeatures en la columna features:
```

```
+---+----+------+--------------+-------+-----------------------+
|id |hour|mobile|userFeatures  |clicked|features               |
+---+----+------+--------------+-------+-----------------------+
|0  |18  |1.0   |[0.0,10.0,0.5]|1.0    |[18.0,1.0,0.0,10.0,0.5]|
+---+----+------+--------------+-------+-----------------------+
```

SQLTransformer

Recibe una sentencia SQL de tipo SELECT-FROM-WHERE, expresada string, y la aplica al DF que se le pase como argumento a transform(). Útil para dejar fijadas series de transformaciones arbitrarias que luego sean incluidas en un pipeline. Hay que recordar que, en general, en un proceso de ML, el código que escribimos para pre-procesar, limpiar, generar nuevas features, etc, no suele estar incluido como tal dentro de una pieza de tipo `Pipeline` (en el sentido que veremos luego, pipeline de Spark ML). Lo que sucede es que el desarrollador es responsable de asegurarse de que el mismo código que hace todo este proceso durante la etapa de entrenamiento (para convertir datos en crudo en features utilizadas por el modelo para entrenar) será también ejecutado en la etapa de predicción: cuando el modelo se despliega y recibe nuevos datos para predecir, los cuales también deben ser pre-procesados y convertidos en las features que el modelo requiere para poder predecir sobre ellas. Dicho código (que debería ser "único", y no estar duplicado para las etapas de entrenamiento y de predicción), generalmente no suele/puede convertirse en una "etapa de pipeline", puesto que ese tipo de etapas suelen tener funcionalidad muy específica y limitada. Aun así, podemos expresar ciertas transformaciones arbitrarias dentro de una sentencia SQL, que será aplicada sobre el DataFrame recibido como argumento por el método `transform()` de dicha pieza.

```
from pyspark.ml.feature import SQLTransformer

df = spark.createDataFrame([
```

(continues on next page)

(continued from previous page)

```
    (0, 1.0, 3.0),
    (2, 2.0, 5.0)
], ["id", "v1", "v2"])

sql_transformer = SQLTransformer(
    statement = "SELECT *, (v1 + v2) AS v3, (v1 * v2) AS v4 FROM __THIS__"
)
result_df = sql_transformer.transform(df)  # añadirá las nuevas columnas v3 y v4
result_df.show()
```

```
+---+---+---+---+----+
| id| v1| v2| v3|  v4|
+---+---+---+---+----+
|  0|1.0|3.0|4.0| 3.0|
|  2|2.0|5.0|7.0|10.0|
+---+---+---+---+----+
```

Bucketizer

Discretizador que transforma una columna continua en una columna con identificadores de intervalo (bucket). Recibe como parámetro el vector con los límites de cada intervalo. La columna resultante codifica las categorías como números reales con la parte decimal a 0 y empezando en 0.0, es decir, tal como Spark necesita representar columnas categóricas. Además, dicha columna contiene **metadatos** que, aunque no se visualizan al mostrar por pantalla el contenido del DataFrame, son utilizados por los algoritmos para interpretar correctamente esa columna como categórica y actuar en consecuencia al entrenar un modelo. Los puntos de corte indicados se interpretan por parejas como intervalos. Por ejemplo, la lista de puntos de corte [-3.2, 0.6, 3.5, 8] corresponde a los intervalos [-3.2, 0.6), [0.6, 3.5) y [3.5, 8), donde cada intervalo siempre es cerrado por la izquierda (incluye al extremo inferior) y abierto por la derecha (no incluye al extremo superior).

```
from pyspark.ml.feature import Bucketizer

splits = [-float("inf"), -0.5, 0.0, 0.5, float("inf")]
data = [(-999.9,), (-0.5,), (-0.3,), (0.0,), (0.2,), (999.9,)]

dataFrame = spark.createDataFrame(data, ["features"])

bucketizer = Bucketizer(splits = splits, inputCol = "features", outputCol =
 ↪"bucketedFeatures")

# Transformamos los datos originales en el índice de su intervalo
bucketedData = bucketizer.transform(dataFrame)

# Obtenemos la longitud del vector de límites de intervalos
n_intervalos = len(bucketizer.getSplits())-1
print(f"Salida del bucketizer con {n_intervalos} intervalos")

bucketedData.show()
```

```
Salida del bucketizer con 4 intervalos
```

```
+--------+----------------+
|features|bucketedFeatures|
+--------+----------------+
|  -999.9|             0.0|
|    -0.5|             1.0|
|    -0.3|             1.0|
|     0.0|             2.0|
|     0.2|             2.0|
|   999.9|             3.0|
+--------+----------------+
```

Cualquier modelo entrenado es un transformador

El resultado de entrenar un modelo sobre un DF es un objeto *Model de la subclase específica del modelo que hayamos ajustado (por ejemplo, `LinearRegressionModel`, `RandomForestClassifierModel` etc), que además es también un `Transformer`, por lo que es capaz de transformar (hacer predicciones) un DF de ejemplos, siempre que tenga el mismo formato (mismos nombres de columnas y tipos de datos) que el DF que se utilizó para entrenar. El motivo es que todas las clases *Model son subclases de la clase `Transformer`, que actúa como una interfaz común a todas ellas, y por tanto, se comportan como transformadores.

- En un modelo entrenado, *transformar* quiere decir añadir una o varias columnas con predicciones al DF resultante.
  - En el caso de los modelos de regresión, añaden una nueva columna, mientras que los de clasificación añaden tres columnas.
- Las predicciones se añaden para cada ejemplo en una nueva columna
- Para facilitar que se mantenga el mismo formato, se suele entrenar un pipeline completo y utilizar su salida (pipeline entrenado) como transformador, como veremos después.

En el ejemplo siguiente, leemos un fichero con un formato peculiar llamado *libvsm*, que es el formato de fichero de datos propuesto por los autores de la primera implementación abierta de las Support Vector Machines (SVM), en lenguaje C++, llamada también **libsvm** y descrita en este enlace. El formato "libsvm" especifica en la primera columna el valor de la target. Después, cada línea del fichero es un ejemplo, y sus features se indican en formato índice:valor donde el índice es un entero con el número de columna, y valor es el valor de esa feature. Sólo se codifican las features con valores distintos de 0, por lo que podemos tener filas como `2.42 1:0.65 4:5.21 7:-4.55`, que indica que las columnas 0, 2, 3, 5 y 6 tienen valor 0 para este ejemplo, y el valor 2.42 es la target.

```
1  +1 1:0.06992427 2:0.043791715 3:0.010570114 4:0.041858662 5:0.0016906267 6:0.043480627 7
2  +1 3:0.0028312805 5:0.0012679701 7:0.010755781
3  +1 1:0.021851335 2:0.008553068 3:0.0040109805 4:0.032702077 5:0.0016906267 6:0.007764398
4  +1 2:0.011729923 3:0.009707248 4:0.11212142 5:0.0016906267 7:0.0046096207 9:0.019167876
5  +1 3:0.005662561 4:0.017441109 5:0.0016906267 6:0.031057592 7:0.006572977 9:0.029816695
6  +1 1:0.09711705 2:0.018246546 3:0.008808428 4:0.043602776 5:0.0016906267 6:0.015528796 7
7  +1 2:0.008210946 3:0.0033975367 4:0.031393997 5:0.0016906267 6:0.018634556 7:0.006453465
8  +1 2:0.037322484 3:0.0061773392 4:0.028539997 5:0.0016906267 6:0.008470252 7:0.005866785
9  +1 1:0.06992427 3:0.004530049 5:0.0016906267 7:0.008604625 9:0.08050507 13:0.13395478
10 +1 2:0.03518977 3:0.008089373 4:0.033636425 5:0.0016906267 6:0.019965595 7:0.0046096207
11 +1 1:0.037459433 2:0.038122248 3:0.006875967 4:0.022424284 5:0.0016906267 6:0.009982797
12 +1 1:0.058270227 3:0.009437601 5:0.0016906267 6:0.015528796 7:0.0071705207 9:0.02236252
13 +1 2:0.02526445 3:0.008711632 5:0.0016906267 6:0.028668547 7:0.0066189426 8:0.12320319 9
```

```
from pyspark.ml.regression import LinearRegression
training = spark.read.format("libsvm")\
                     .load("sample_linear_regression_data.txt")
```

(continues on next page)

(continued from previous page)

```
training.show(5, truncate=False)

lr = LinearRegression(maxIter=10, regParam=0.3, elasticNetParam=0.8)  # El objeto␣
↪almacenado en lr es un Estimator
lrModel = lr.fit(training)   # El método fit de un Estimator devuelve un Transformer␣
↪(modelo entrenado)b

print(f"Clase del objeto almacenado en lrModel: {lrModel.__class__}")

from pyspark.ml import Transformer

print(f"¿Pertenece también a la clase Transformer? {isinstance(lrModel, Transformer)}
↪") # Comprobamos que el modelo ajustado

pred = lrModel.transform(training)
pred.show(5, truncate=False)
```

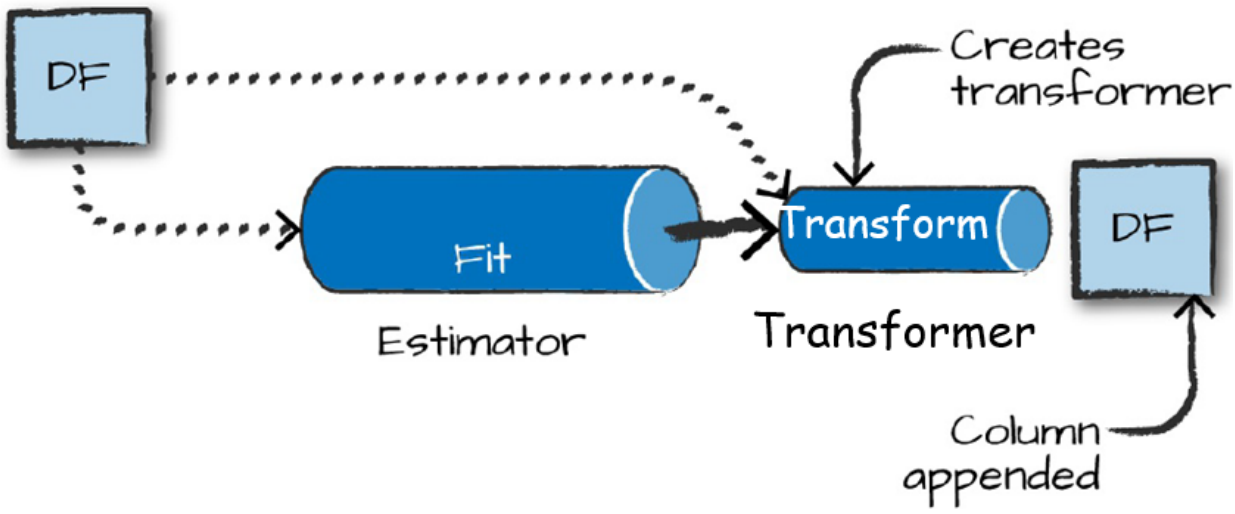
```
+-------------------+------------------------------------------------------------------
↪------------------------------------------------------------------------------------
↪------------------------------------------------------------------------------------
↪-----+
|label              |features                                                          ␣
↪                                                                                    ␣
↪                                                                                    ␣
↪     |
+-------------------+------------------------------------------------------------------
↪------------------------------------------------------------------------------------
↪------------------------------------------------------------------------------------
↪-----+
|-9.490009878824548 |(10,[0,1,2,3,4,5,6,7,8,9],[0.4551273600657362,0.
↪36644694351969087,-0.38256108933468047,-0.4458430198517267,0.33109790358914726,0.
↪8067445293443565,-0.2624341731773887,-0.44850386111659524,-0.07269284838169332,0.
↪5658035575800715]) |
|0.2577820163584905 |(10,[0,1,2,3,4,5,6,7,8,9],[0.8386555657374337,-0.
↪1270180511534269,0.499812362510895,-0.22686625128130267,-0.6452430441812433,0.
↪18869982177936828,-0.5804648622673358,0.651931743775642,-0.6555641246242951,0.
↪17485476357259122])      |
|-4.438869807456516 |(10,[0,1,2,3,4,5,6,7,8,9],[0.5025608135349202,0.
↪14208069682973434,0.16004976900412138,0.505019897181302,-0.9371635223468384,-0.
↪2841601610457427,0.6355938616712786,-0.1646249064941625,0.9480713629917628,0.
↪42681251564645817])       |
|-19.782762789614537|(10,[0,1,2,3,4,5,6,7,8,9],[-0.0388509668871313,-0.
↪4166870051763918,0.8997202693189332,0.6409836467726933,0.273289095712564,-0.
↪26175701211620517,-0.2794902492677298,-0.1306778297187794,-0.08536581111046115,-
↪0.05462315824828923])  |
|-7.966593841555266 |(10,[0,1,2,3,4,5,6,7,8,9],[-0.06195495876886281,0.
↪6546448480299902,-0.6979368909424835,0.6677324708883314,-0.07938725467767771,-0.
↪43885601665437957,-0.608071585153688,-0.6414531182501653,0.7313735926547045,-0.
↪026818676347611925])|
+-------------------+------------------------------------------------------------------
↪------------------------------------------------------------------------------------
↪------------------------------------------------------------------------------------
↪-----+
only showing top 5 rows
```

```
Clase del objeto almacenado en lrModel: <class 'pyspark.ml.regression.
↪LinearRegressionModel'>
¿Pertenece también a la clase Transformer? True
+-------------------+-------------------------------------------------------------
↪-------------------------------------------------------------------------------
↪-------------------------------------------------------------------------------
↪-----+-------------------+
|label              |features                                                     ␣
↪                                                                              ␣
↪                                                                              ␣
↪     |prediction         |
+-------------------+-------------------------------------------------------------
↪-------------------------------------------------------------------------------
↪-------------------------------------------------------------------------------
↪-----+-------------------+
|-9.490009878824548 |(10,[0,1,2,3,4,5,6,7,8,9],[0.4551273600657362,0.
↪36644694351969087,-0.38256108933468047,-0.4458430198517267,0.33109790358914726,0.
↪8067445293443565,-0.2624341731773887,-0.44850386111659524,-0.07269284838169332,0.
↪5658035575800715]) |0.3992228042786488 |
|0.2577820163584905 |(10,[0,1,2,3,4,5,6,7,8,9],[0.8386555657374337,-0.
↪1270180511534269,0.499812362510895,-0.22686625128130267,-0.6452430441812433,0.
↪18869982177936828,-0.5804648622673358,0.651931743775642,-0.6555641246242951,0.
↪17485476357259122])      |-0.2955974176468648|
|-4.438869807456516 |(10,[0,1,2,3,4,5,6,7,8,9],[0.5025608135349202,0.
↪14208069682973434,0.16004976900412138,0.505019897181302,-0.9371635223468384,-0.
↪2841601610457427,0.6355938616712786,-0.1646249064941625,0.9480713629917628,0.
↪42681251564645817])      |0.7651496483023065 |
|-19.782762789614537|(10,[0,1,2,3,4,5,6,7,8,9],[-0.0388509668871313,-0.
↪4166870051763918,0.8997202693189332,0.6409836467726933,0.273289095712564,-0.
↪26175701211620517,-0.2794902492677298,-0.1306778297187794,-0.08536581111046115,-
↪0.05462315824828923])  |0.783923925892972  |
|-7.966593841555266 |(10,[0,1,2,3,4,5,6,7,8,9],[-0.06195495876886281,0.
↪6546448480299902,-0.6979368909424835,0.6677324708883314,-0.07938725467767771,-0.
↪43885601665437957,-0.608071585153688,-0.6414531182501653,0.7313735926547045,-0.
↪026818676347611925])|1.4831466765011345 |
+-------------------+-------------------------------------------------------------
↪-------------------------------------------------------------------------------
↪-------------------------------------------------------------------------------
↪-----+-------------------+
only showing top 5 rows
```

4.2.2 Estimadores de Spark ML

Un *Estimator* es un objeto de Spark capaz de realizar transformaciones que primero requieren que ciertos parámetros de la transformación se ajusten (o se aprendan) a partir de los datos. Normalmente requieren una pasada previa (o varias) sobre la columna de datos que se desea transformar.

La interfaz Estimator tiene un único método: `fit(df: DataFrame)` que recibe un DataFrame y devuelve un objeto de tipo *Model, el modelo entrenado, que es además un Transformer tal como explicamos anteriormente (*cualquier modelo entrenado es un Transformer*). Una vez entrenado, el modelo resultante puede aplicarse para predecir cualquier conjunto de datos, no solamente los mismos datos con los que fue entrenado.



Es importante notar que Spark llama *modelo* a cualquier cosa que requiera un fit previo, no sólo a los algoritmos de aprendizaje automático que conocemos como modelos propiamente. El motivo es que existen varias piezas de pre-procesamiento que, sin ser algoritmos predictivos, también son **entrenables**. Esto mismo ya ocurría en la biblioteca de `scikit-learn`: por ejemplo la pieza `StandardScaler`, que también existe en la biblioteca de Spark ML puesto que su diseño está muy inspirado por scikit-learn, es una pieza *entrenable*: para estandarizar una variable, primero tenemos que aprender (calcular y memorizar) la media y varianza de la columna que vamos a estandarizar. La estandarización se lleva a cabo restando la media y dividiendo por la varianza, de manera que para la variable X, sus valores estandarizados serán $z_i = \frac{x_i - \mu}{\sigma}$ (a esto se denomina transformación Z-score). A continuación, podemos estandarizar esa columna, pero lo que es más importante, la pieza StandardScaler habrá sido entrenada y tendrá almacenados los valores de μ y σ. Si hemos usado esa pieza para estandarizar los datos con los que después hemos entrenado un modelo, entonces cuando queramos hacer predicciones con ese modelo usando datos nuevos, habrá que estandarizarlos de la misma manera que se estandarizaron los datos de entrenamiento, es decir, restando *la misma media* que habíamos calculado sobre los datos de entrenamiento, y dividiendo por *la misma varianza* de los datos de entrenamiento.

Vamos a ver ejemplos de algunos estimadores comunes.

StringIndexer

Estimador para pre-procesar variables categóricas. Es el más utilizado. Convierte una columna categórica (da igual el tipo ya que los valores serán entendidos como categorías) en números reales (Double) con la parte decimal a 0 y empezando en 0.0, donde las categorías se representan mediante 0.0, 1.0, 2.0… Entrenar el StringIndexer consiste en generar una pieza, que es el StringIndexer entrenado (clase `StringIndexerModel`), que memoriza internamente la correspondencia entre cada categoría original y su valor codificado. Esa correspondencia ya creada (con los datos de entrenamiento) será aplicada cuando se utilice el `StringIndexerModel` para codificar una variable en datos nuevos para predicción.

La correspondencia creada no es realmente relevante, aunque por defecto Spark asigna un 0.0 a la categoría que más veces se repite, un 1.0 a la siguiente que más veces aparece, y así sucesivamente.

- Además, añade metadatos al DataFrame transformado (resultante de transform) indicando que esa columna es categórica (no es como cualquier otra columna de números reales)
- Los algoritmos que sí soportan variables categóricas (ej: DecisionTree, RandomForest, GradientBoostedTrees) requieren que se las pasemos indexadas
- Los algoritmos que no soportan variables categóricas (LinearRegression, LogisticRegression) requieren el uso de OneHotEncoder (siguiente slide)

IMPORTANTE: al usar un modelo de machine learning para predecir ejemplos nuevos, primero hay que codificar sus variables categóricas siguiendo exactamente la misma codificación que se utilizó con los datos de entrenamiento con los

que se entrenó el modelo. Si al crear el dataset de train, se había codificado la categoría "alto" como 1.0, entonces cuando queramos predecir un nuevo dato, antes de pasárselo al modelo, tendremos que codificar "alto" también como 1.0, porque de lo contrario estaríamos falseando ("inventando") el valor de esa feature del ejemplo que queremos predecir. Por eso cobra utilidad la estructura de Pipeline que comentaremos más adelante.

Es posible utilizar como argumentos inputCols y outputCols, que deben ser dos listas de string de la misma longitud, para codificar un conjunto de columnas (de manera independiente) en un solo artefacto. De esa forma evitamos tener que crear tantos objetos StringIndexer como columnas categóricas queramos codificar: es posible codificarlas todas usando un único StringIndexer. El argumento `handleInvalid` indica qué debe hacer el StringIndexerModel (es decir, ya entrenado) cuando tenga que codificar una categoría que no ha visto nunca antes cuando fue entrenado. Los posibles valores son `error` (por defecto, saltará una excepción), `skip` (las filas donde ocurra esto no irán al resultado, y por tanto podríamos tener un DF más pequeño que el original, lo cual es peligroso), o `keep` (valor recomendado, que hace que se codifique con el primer valor que esté libre, por ejemplo 4.0 si ya teníamos ocupados 0.0, 1.0, 2.0 y 3.0).

```
from pyspark.ml.feature import StringIndexer, StringIndexerModel

df = spark.createDataFrame([(0,"a"), (1,"b"), (2,"c"), (3,"a"), (4,"a"), (5,"c")], [
 ↪"id", "category"])

indexer = StringIndexer(inputCol = "category",
                        outputCol = "categoryIndexed",
                        handleInvalid = "keep")

indexerModel = indexer.fit(df)
indexed = indexerModel.transform(df)

# resto de código

# lo uso otra vez
nuevo_df = spark.createDataFrame([("pepe", "c"), ("hola", "c"), ("paco", "b"), ("mari
 ↪", "c"), ("ana", "a")], ["name", "category"])
indexedNuevo = indexerModel.transform(nuevo_df)

print("Resultado de la codificación por defecto (a->0, b->2, c->1):")
indexedNuevo.show()

# otra opción: puedo indicar la correspondencia que deseamos si creamos directamente␣
 ↪el StringIndexerModel
customIndexerModel = StringIndexerModel.from_labels(inputCol = "category",
                                                    outputCol = "categoryIndexed",
                                                    labels = ["b", "a", "c"]) #␣
 ↪códigos asignados en el orden de las labels: 0 para "b", 1 para "a", 2 para "c"

print("Resultado de la codificación personalizada (a->1, b->0, c->2):")
customIndexerModel.transform(nuevo_df).show()

print("Entrenamos un único StringIndexer para codificar por separado de una vez ambas␣
 ↪columnas:")
otro_indexer = StringIndexer(inputCols = ["name", "category"],
                             outputCols = ["nameIndexed", "categoryIndexed"])

otro_indexer_model = otro_indexer.fit(nuevo_df)
result_df = otro_indexer_model.transform(nuevo_df)
result_df.show()
```

```
Resultado de la codificación por defecto (a->0, b->2, c->1):
```

```
+----+--------+---------------+
|name|category|categoryIndexed|
+----+--------+---------------+
|pepe|       c|            1.0|
|hola|       c|            1.0|
|paco|       b|            2.0|
|mari|       c|            1.0|
| ana|       a|            0.0|
+----+--------+---------------+

Resultado de la codificación personalizada (a->1, b->0, c->2):
```

```
+----+--------+---------------+
|name|category|categoryIndexed|
+----+--------+---------------+
|pepe|       c|            2.0|
|hola|       c|            2.0|
|paco|       b|            0.0|
|mari|       c|            2.0|
| ana|       a|            1.0|
+----+--------+---------------+

Entrenamos un único StringIndexer para codificar por separado de una vez ambas␣
↪columnas:
```

```
+----+--------+-----------+---------------+
|name|category|nameIndexed|categoryIndexed|
+----+--------+-----------+---------------+
|pepe|       c|        4.0|            0.0|
|hola|       c|        1.0|            0.0|
|paco|       b|        3.0|            2.0|
|mari|       c|        2.0|            0.0|
| ana|       a|        0.0|            1.0|
+----+--------+-----------+---------------+
```

OneHotEncoder

Recibe un conjunto de columnas y convierte cada una (de manera independiente) a un conjunto de variables dummy con codificación one-hot

- Cada variable (con n categorías posibles) da lugar a n-1 columnas (en una sola columna de tipo vector) donde en cada ejemplo, sólo una de ellas tiene valor 1 y el resto son 0 indicando cuál es el valor de la categoría presente en ese ejemplo
- Spark siempre asume que los valores provienen de una indexación previa con StringIndexer: obligatoriamente números reales con la parte decimal a 0
- Entrenar consiste en crear y memorizar la correspondencia entre la categoría original, expresada como número real con la parte decimal a 0, y su vector de 1 y 0. Esa asociación será luego aplicada cuando utilicemos el `OneHotEncoderModel` resultante (one-hot ya entrenado) para codificar nuevos datos siguiendo la codificación que fue aprendida sobre los datos de entrenamiento.

```
from pyspark.ml.feature import OneHotEncoder
from pyspark.ml.linalg import Vectors, VectorUDT  # convertir vectores sparse a dense
```

(continues on next page)

(continued from previous page)

```
                                                    # porque el show() de vectores␣
 ↪sparse se ve feo en pantalla
from pyspark.sql import functions as F

df = spark.createDataFrame([
     (0.0, 1.0, 2.0),
     (1.0, 0.0, 3.0),    # Spark asume que la 3ª col tiene 5 categorías
     (2.0, 1.0, 2.0),    # porque el máximo valor que aparece es 4.0
     (0.0, 2.0, 1.0),    # y asume que vienen de una indexación previa
     (0.0, 1.0, 4.0),    # con StringIndexer y por tanto empiezan en 0.0
     (2.0, 0.0, 4.0)
], ["categoryIndex1", "categoryIndex2", "categoryIndex3"])

encoder = OneHotEncoder(
   inputCols = ["categoryIndex1", "categoryIndex2", "categoryIndex3"],
   outputCols = ["categoryVec1", "categoryVec2", "categoryVec3"]
)
model = encoder.fit(df)
encoded = model.transform(df)

# Lo siguiente simplemente convierte la columna de salida, que eran vectores sparse␣
 ↪(dispersos), en densos
# para visualizarlos mejor, aunque lo óptimo es dejarlos como están (como vectores␣
 ↪dispersos) porque ocupan menos espacio
# en la memoria RAM

toDenseUDF = F.udf(lambda r: Vectors.dense(r), VectorUDT())

encoded.withColumn("categoryVec1", toDenseUDF("categoryVec1"))\
       .withColumn("categoryVec2", toDenseUDF("categoryVec2"))\
       .withColumn("categoryVec3", toDenseUDF("categoryVec3"))\
       .show()
```

```
+--------------+--------------+--------------+------------+------------+-----------
 ↪------+
|categoryIndex1|categoryIndex2|categoryIndex3|categoryVec1|categoryVec2|    ␣
 ↪categoryVec3|
+--------------+--------------+--------------+------------+------------+-----------
 ↪------+
|           0.0|           1.0|           2.0|   [1.0,0.0]|   [0.0,1.0]|[0.0,0.0,1.
 ↪0,0.0]|
|           1.0|           0.0|           3.0|   [0.0,1.0]|   [1.0,0.0]|[0.0,0.0,0.
 ↪0,1.0]|
|           2.0|           1.0|           2.0|   [0.0,0.0]|   [0.0,1.0]|[0.0,0.0,1.
 ↪0,0.0]|
|           0.0|           2.0|           1.0|   [1.0,0.0]|   [0.0,0.0]|[0.0,1.0,0.
 ↪0,0.0]|
|           0.0|           1.0|           4.0|   [1.0,0.0]|   [0.0,1.0]|[0.0,0.0,0.
 ↪0,0.0]|
|           2.0|           0.0|           4.0|   [0.0,0.0]|   [1.0,0.0]|[0.0,0.0,0.
 ↪0,0.0]|
+--------------+--------------+--------------+------------+------------+-----------
 ↪------+
```

4.2.3 Pipelines de Spark ML

Es frecuente en Machine Learning extraer características de datos en crudo ("raw") y prepararlas antes de llamar a un algoritmo de aprendizaje. Sin embargo, puede ser difícil tener control de todos los pasos de pre-procesamiento que hemos hecho al entrenar, para luego replicarlos en otros conjuntos de datos o en el momento de hacer predicciones para nuevos datos con el modelo entrenado. Por ejemplo, para procesar un documento:

- División en palabras
- Procesamiento de palabras para obtener un vector de características numéricas
- Preparación de esas características para el formato que requiere el algoritmo elegido en Spark
- Finalmente, entrenamiento de un modelo.

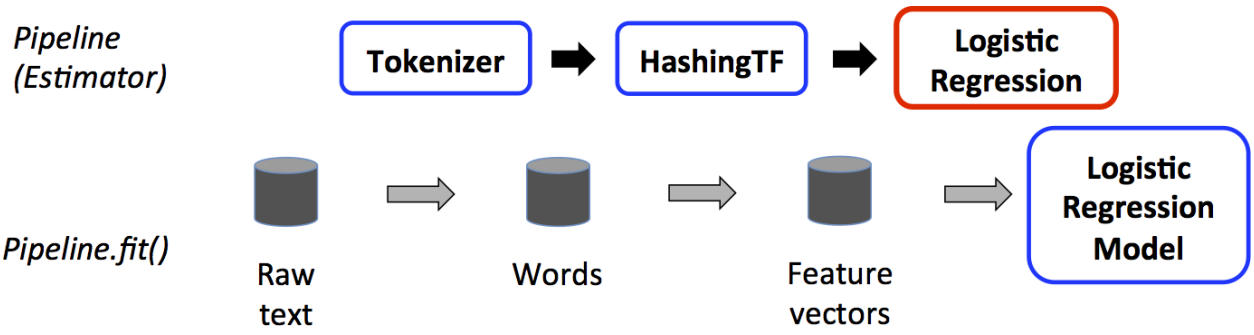
Spark nos proporciona un mecanismo para esto.

Note: Pipeline: secuencia de etapas (PipelineStage, superclase común de Estimator y Transformer) que se ejecutan en un cierto orden. La salida de una etapa es entrada para alguna de las etapas posteriores (no necesariamente la inmediatamente siguiente)

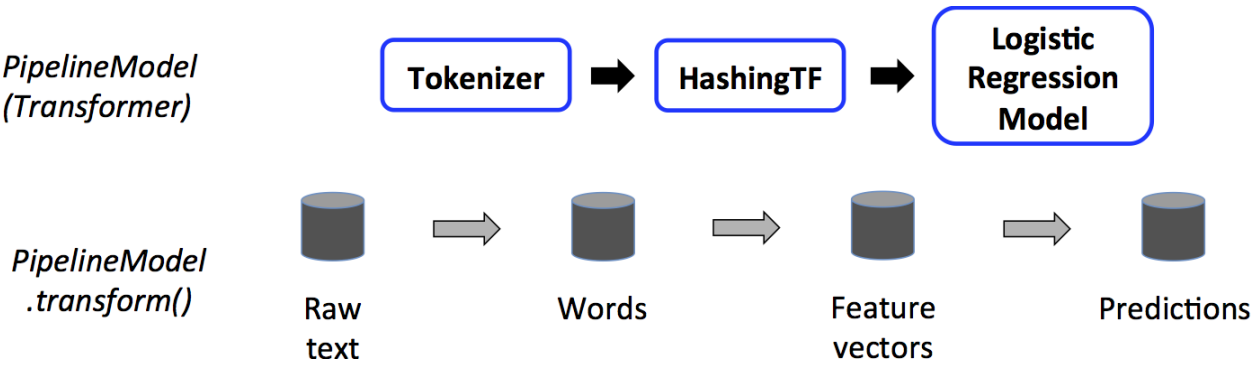
Warning: En general, nunca invocamos directamente a los métodos `fit` o `transform` de ninguna de las etapas que hemos visto. Lo habitual es crear cada etapa para introducirla en un pipeline, y luego invocar al método `fit` del Pipeline, que entrena el pipeline como una sola cosa.

Un Pipeline es un Estimator. El método fit(df) recorre cada etapa, llama a transform() si la etapa es un Transformer, y llama a fit(df) y luego a transform(df) si es un Estimator, pasando siempre el DF tal como esté en ese punto (con las columnas originales más las que le hayan añadido las etapas previas). Es habitual (pero no obligatorio) que la última etapa sea un algoritmo de ML, aunque podría haber varios a lo largo de un pipeline Es importante recordar que un mismo objeto (sea un estimador o un transformador) no puede ser añadido como a dos pipelines diferentes.

Pipeline antes de llamar a fit:



Las etapas en azul son transformadores mientras que las rojas son estimadores. A continuación vemos el objeto PipelineModel (pipeline ajustado), donde las etapas que eran estimadores han pasado a ser transformadores. Si llamamos al método transform() sobre un PipelineModel, éste irá llamando a transform para cada etapa, pasando como argumento el DF resultante del transform de la etapa previa.



Lo habitual es llamar a fit sobre el objeto Pipeline una sola vez con los datos de entrenamiento. Esto devuelve un objeto PipelineModel (modelo ajustado), sobre el que podemos llamar a transform tantas veces como queramos sobre conjuntos de datos nuevos (nunca vistos por el modelo, pero contengan las columnas que esperan cada una las etapas) para realizar predicciones sucesivamente.

IMPORTANTE: el orden de las etapas en el pipeline es crucial, ya que probablemente existirán dependencias entre ellas, causadas porque una etapa creará como salida una columna que puede ser necesitada por alguna etapa posterior.

```
from pyspark.ml.feature import StringIndexer, VectorAssembler, Binarizer,␣
↪VectorSlicer, StandardScaler
from pyspark.ml import Pipeline
from pyspark.ml.classification import LogisticRegression

trainTest = spark.read.option("header", "true")\
                      .option("inferSchema", "true")\
                      .csv("flights.csv")\
                      .dropna("any", subset=["Month", "DayofMonth", "DayOfWeek",
↪"Origin", "DepTime",
                                             "ArrTime", "ActualElapsedTime", "AirTime", "DepDelay",
↪"Distance", "ArrDelay"])\
                      .randomSplit([0.8, 0.2], 12345)

trainingData = trainTest[0].cache()
testingData = trainTest[1].cache()

indexers = StringIndexer(inputCols = ["Origin", "Dest", "Month", "DayOfWeek"],
                         outputCols = ["OriginCat", "DestCat", "MonthCat",
↪"DayOfWeekCat"],
                         handleInvalid = "keep") # salida utilizada por el␣
↪VectorAssembler

assembler = VectorAssembler().setInputCols(["MonthCat", "DayOfWeekCat", "OriginCat",
↪"DestCat", "DepTime",
                                             "ArrTime", "ActualElapsedTime", "AirTime",
↪ "DepDelay", "Distance"])\
                             .setOutputCol("rawFeatures")\
                             .setHandleInvalid("keep") # salida utilizada por el␣
↪StandardScaler

scaler = StandardScaler(inputCol="rawFeatures", outputCol="features")\
                        .setWithStd(True).setWithMean(True)     # salida utilizada␣
↪después por LogisticRegression

binarizerClassifier = Binarizer().setInputCol("ArrDelay")\
                                 .setOutputCol("binaryLabel")\
```

(continues on next page)

(continued from previous page)

```
                                .setThreshold(15.0)  # salida utilizada por␣
↪LogisticRegression

lr = LogisticRegression().setMaxIter(100)\
                         .setRegParam(0.03)\
                         .setElasticNetParam(0.8)\
                         .setLabelCol("binaryLabel")\
                         .setFeaturesCol("features")

pipeline = Pipeline().setStages([indexers, assembler, scaler, binarizerClassifier,␣
↪lr])
pipelineModel = pipeline.fit(trainingData)
lrPredictions = pipelineModel.transform(testingData)

lrPredictions.select("rawPrediction", "probability", "prediction", "binaryLabel",
↪"features").show(5, truncate=False)
```

```
+----------------------------------------+----------------------------------------
↪+----------+-----------+----------------------------------------------------------
↪----------------------------------------------------------------------------------
↪-------------------------------------------------------------+
|rawPrediction                           |probability                             ␣
↪|prediction|binaryLabel|features                                                  ␣
↪                                                                                  ␣
↪                                                                    |
+----------------------------------------+----------------------------------------
↪+----------+-----------+----------------------------------------------------------
↪----------------------------------------------------------------------------------
↪-------------------------------------------------------------+
|[2.435588861275205,-2.435588861275205]  |[0.9195011844305776,0.0804988155694224]␣
↪|0.0       |0.0        |[0.5085589655018775,-1.4466524732666268,0.
↪3695356314042823,-0.33159446369907947,-1.5629587754946674,-1.3783768874295041,-0.
↪2972263152357282,-0.5189235848034222,-0.31648330739903746,-0.4700944750190298]|
|[2.382940484923568,-2.382940484923568]  |[0.9155171456095604,0.
↪08448285439043957]|0.0       |0.0        |[0.5085589655018775,-1.
↪4466524732666268,-0.7308852638648066,1.9061095994934991,1.646229256291597,1.
↪4752698841606116,-1.0614987728347534,-1.0173285679856696,-0.2934807410010932,-1.
↪0337331194592838]     |
|[2.5408856139784795,-2.5408856139784795]|[0.9269588107889988,0.
↪07304118921100122]|0.0       |0.0        |[0.5085589655018775,-1.
↪4466524732666268,-0.7308852638648066,-0.3715534648275184,-1.3665187835427286,-1.
↪033513864568016,0.31419165084349193,-0.020518601621174647,-0.362488440194926,0.
↪30597940319201894] |
|[2.067050226813745,-2.067050226813745]  |[0.8876591451126371,0.1123408548873629]␣
↪|0.0       |0.0        |[0.5085589655018775,-1.4466524732666268,-0.
↪7308852638648066,-0.3715534648275184,1.215263967825609,1.3881067025582574,0.
↪17523302218912373,0.20732367640499566,-0.15546534261342768,0.30597940319201894] ␣
↪ |
|[0.5928956889679053,-0.5928956889679053]|[0.6440292750336633,0.
↪35597072496633675]|0.0       |1.0        |[0.5085589655018775,-1.
↪4466524732666268,-0.7308852638648066,-0.31161496313486003,0.5618003211283434,0.
↪626376289424641,-0.32501804096660186,-0.29108130677725186,0.4886065165290115,-0.
↪3193339865365303]  |
+----------------------------------------+----------------------------------------
↪+----------+-----------+----------------------------------------------------------
↪----------------------------------------------------------------------------------
↪-------------------------------------------------------------+
```

(continues on next page)

(continued from previous page)

```
only showing top 5 rows
```

El significado de las columnas añadidas por un modelo entrenado es el siguiente.

- En problemas de **regresión**, la única columna que se añade es `prediction`, que se puede configurar en el argumento `predictionCol` del estimador. Es una columna de números reales con las predicciones.
- En problemas de **clasificación**, el modelo entrenado añade 3 columnas al predecir:
  - La columna `rawPrediction`, que se puede configurar al crear el estimador original (antes de entrenar) mediante el argumento `rawPredictionCol`, es una columna de tipo vector, donde el contenido de cada celda es un vector de números reales de tantos elementos como clases tenga el problema, siendo cada elemento una puntuación o score que indica la plausibilidad de que la clase del ejemplo sea la de esa posición. Por ejemplo, el vector `[23.05, 10.22, 6.73]` sería un vector posible para un problema de clasificación de 3 clases. Los scores no tienen que estar en ningún rango fijo, y su significado depende totalmente de qué algoritmo hayamos entrenado. Por ejemplo, si han sido generados al predecir con un `RandomForestClassifierModel`, representan una puntuación que es el voto ponderado de los árboles al predecir, y por tanto, la suma de esos scores será igual al número de árboles (los scores del ejemplo anterior suman 40 y sería compatible con un RandomForest de 40 árboles). En cambio, si los scores han sido generados por un `LogisticRegressionModel` (regresión logística ya entrenada), el significado es que son los *log-odds* (logaritmo de la ventaja, es decir, $log\frac{p}{1-p}$) de cada una de las clases de ese ejemplo, y por tanto, son iguales pero de signo contrario en problemas de clasificación binaria que es donde suele emplearse la regresión logísica.
  - La columna `probability`, que puede configurarse al crear el estimador original mediante el argumento `probabilityCol`, es el resultado de normalizar los valores de `rawProbability` en el caso de los modelos que dan un score, como por ejemplo RandomForest. Nótese que normalizar scores **no da como resultado una probabilidad** sino simplemente unos valores que están entre 0 y 1, y que suman 1, pero esto no es suficiente para considerarlos una distribución de probabilidad. Para ver métodos de calcular una verdadera distribución de probabilidad a partir de scores, pueden revisarse los *métodos de calibración* en https://scikit-learn.org/stable/modules/calibration.html. Spark no incorpora métodos de calibración como tal, aunque una regresión logística *adicional* puede bastar si se ajusta bajo las condiciones descritas en esa documentación. Por otro lado, para los métodos que sí ofrecen nativamente como salida una verdadera probabilidad (lo cual no ocurre con casi ninguno, pero sí por construcción con la regresión logística), la columna `probability` es tal cual la probabilidad de pertenecer a la clase positiva.
  - La columna `prediction` es la posición (empezando en 0) del máximo valor de la columna `probability`.

Evaluación del modelo

Creamos un objeto **evaluador** del paquete `pyspark.ml.evaluator` que depende del tipo de problema que tengamos (`BinaryClassificationEvaluator`, `MulticlassClassificationEvaluator`, `RegressionEvaluator`, `RakingEvaluator`). El objeto sirve para calcular, sobre cualquier DataFrame, la métrica que hayamos configurado en el argumento `metric`. Debemos indicar cómo se llama la columna del DF que contiene las predicciones (en el caso del evaluador multiclase) o la columna de los scores (en el caso del evaluador de clasificación binaria) y la columna que contiene la target auténtica.

```
# Select (prediction, true label) and compute test error
from pyspark.ml.evaluation import BinaryClassificationEvaluator

# Objeto evaluador, para evaluar las predicciones. Compara una columna de␣
 ↪predicciones con la columna target del verdadero valor
# Hay varias métricas posibles, pero nosotros hemos elegido la más simple: porcentaje␣
 ↪de acierto en clasificación.
```

(continues on next page)

(continued from previous page)

```
evaluator = BinaryClassificationEvaluator(
    labelCol="binaryLabel", rawPredictionCol="rawPrediction", metricName="areaUnderROC
 ↪")

auc = evaluator.evaluate(lrPredictions)

print("Test Error = %g " % (1.0 - auc))
```

```
Test Error = 0.0795499
```

4.2.4 Ajuste de híper-parámetros

Casi todos los algoritmos tienen híper-parámetros cuyos valores debe fijar el usuario antes de iniciar el algoritmo. Un híper-parámetro es un valor que afecta a cómo se lleva a cabo el entrenamiento, pero no varía durante dicho entrenamiento (no es aprendido de los datos, a diferencia de los parámetros del modelo) sino que debe ser fijado al principio. La búsqueda del valor óptimo se hace por ensayo-error de entre un conjunto de valores habituales factibles, aunque existen algunos procedimientos inteligentes, no implementados en Spark, que van guiando la búsqueda.

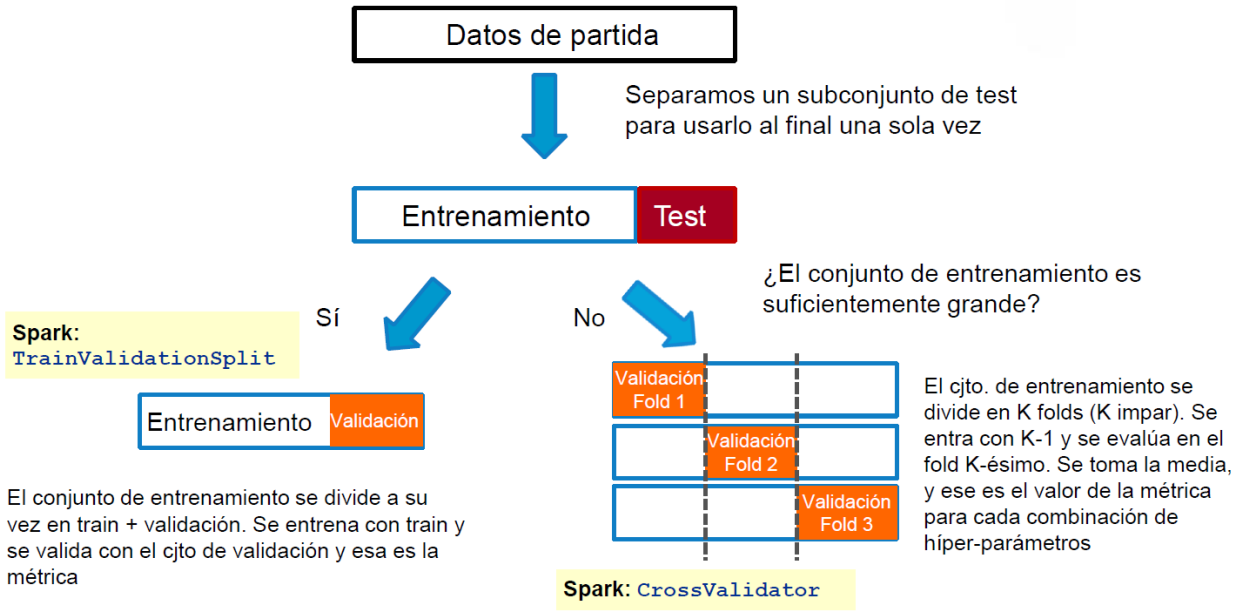
Spark proporciona herramientas para buscar la mejor combinación de valores de los híper-parámetros, indicándole el conjunto de posibles valores que debe probar con cada uno. Se fija una combinación, se entrena el modelo con un subconjunto de los datos (subconjunto de entrenamiento: el mismo subconjunto para todas las combinaciones que se irán probando) y se evalúa el modelo obtenido con esa combinación utilizando otro subconjunto (subconjunto de validación, que también debe ser el mismo para evaluar todas las combinaciones, con lo que se consigue una evaluación justa).

- Si hay suficientes datos, basta con un solo conjunto de entrenamiento y otro de validación. Se utiliza la pieza `TrainValidationSplit` para buscar automáticamente la mejor combinación de híper-parámetros.
- Si no, se lleva a cabo Cross Validation (validación cruzada). Su misión es asegurar que la métrica es estable y fiable, y para ello, para cada combinación de híper-parámetros que estamos evaluando, no entrena un solo modelo sino k modelos independientes. Se particiona el subconjunto de entrenamiento en k partes iguales, disjuntas, llamadas *folds*, y se entrenan k modelos independientes, cada uno de ellos utilizando todos los datos del subconjunto de entrenamiento excepto uno de los folds, que se deja fuera para utilizarlo como fold de validación de ese modelo en concreto. Por tanto, cada fold tendrá que haber actuado exactamente una vez como fold de validación de algún modelo. Esto implica que, para cada combinación que estemos evaluando, habremos entrenado k modelos, habremos aplicado cada uno de ellos a su fold de validación correspondiente y habremos obtenido k valores independientes de la métrica con la que estemos midiendo el modelo (ej., si el problema es de clasificación, podemos usar accuracy, precision, recall … , o si el problema es de regresión, RMSE, MAE, …). El **promedio** de esos k valores de la métrica es el valor final que asignaremos a esa combinación de híper-parámetros. Como los folds son los mismos para todas las combinaciones de híper-parámetros que estamos evaluando, la comparación es justa. El promedio se considera una métrica más estable y fiable para conjuntos de datos relativamente pequeños que si hiciésemos una sola división de los datos en subconjunto de entrenamiento y de validación y con eso calculásemos una sola vez la métrica. Se utiliza la pieza `CrossValidation` para la búsqueda.
- No siempre es necesario utilizar CrossValidation para el ajuste de híper-parámetros. Si las métricas en el fold de validación en cada caso son muy similares entre sí cada vez que entrenamos distintos modelos con la misma combinación de híper-parámetros, probablemente cualquier fold de validación es suficientemente representativo del conjunto original y podríamos hacer una sola división en entrenamiento/validación, y utilizarla para entrenar un solo modelo con cada combinación de híper-parámetros que queramos medir.

Con cualquiera de las dos herramientas, Spark devuelve el modelo entrenado con la mejor combinación de parámetros, usando para ello todo el dataset (sin dividir en folds ni en entrenamiento/validación). Por tanto, en el caso de utilizar el `TrainValidationSplit`, Spark entrena en total tantos modelos como combinaciones queramos probar, más uno

entrenado al final con todo el dataset. En el caso de utilizar el `CrossValidation`, el número de modelos independientes que Spark entrena es igual a *k* por el número de combinaciones de híper-parámetros, más un modelo extra al final, sin división en folds, entrenado con la combinación que ya ha comprobado (gracias a los folds) que funciona mejor.

Antes de empezar todo el proceso, se separa el **subconjunto de test**, que no participará nunca en el proceso. Será utilizado al final, solamente una vez, cuando ya está decidida la mejor combinación de híper-parámetros. El objetivo del subconjunto de test es anticipar cómo se comportará el mejor modelo cuando se aplique datos nunca vistos, lo que permite obtener una métrica que se entregará junto con el modelo entrenado.



El ajuste de híper-parámetros se realiza sobre objetos `Param` que son variables internas de cualquiera de las piezas que hemos añadido a nuestro pipeline. Cada variable interna con la que configuramos cualquier pieza de pre-procesamiento, sea estimador o sea transformador, es un objeto de la clase `Param` (parámetro) de Spark ML. Esto implica que cualquier variable de cualquier etapa del pipeline puede ser sometida a tuning, no solamente los híper-parámetros de los algoritmos predictivos propiamente.

```
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

# Objeto ParamGrid al que le tenemos que indicar cada uno de los parámetros sobre los␣
 ↪que queremos buscar,
# y la lista de valores posibles que queremos probar con cada uno. Deben ser␣
 ↪parámetros tomados directamente
# del objeto estimador que se añadió al pipeline (no pueden ser de ningún otro␣
 ↪modelo). En nuestro caso este objeto
# está almacenado en la variable randomForest que habíamos creado en celdas␣
 ↪anteriores.
paramGrid = ParamGridBuilder()\
    .addGrid(lr.regParam, [0.01, 0.05, 0.1])\
    .addGrid(lr.elasticNetParam, [0.2, 0.8])\
    .build() # como tenemos 2 híper-parámetros con 2 valores posibles cada uno, hay 4␣
 ↪combinaciones posibles

# CrossValidator es un estimador. Cuando invocamos a fit(), probará todas las␣
 ↪combinaciones de valores de los
# parámetros, invocando con cada combinación al método fit() del objeto pipeline que␣
 ↪le hemos pasado como argumento
```

(continues on next page)

(continued from previous page)

```
crossval = CrossValidator(estimator=pipeline,
                          estimatorParamMaps=paramGrid,
                          evaluator=evaluator,
                          numFolds=3)

# Como hemos hecho 3 folds, habrá que entrenar 3 veces cada modelo candidato (cada␣
 ↪combinación de parámetros)
# y obtener su media de accuracy. En total vamos a entrenar 12 modelos + 1 modelo␣
 ↪extra al final
# con todo el dataset de train (sin folds) usando la combinación de híper-parámetros␣
 ↪que ya sabemos que funciona mejor

cvModel = crossval.fit(trainingData.sample(0.1, seed=123))  # intentamos evitar que␣
 ↪tarde mucho en este ejemplo
cvModel.bestModel
```

```
PipelineModel_69b3cb3bb144
```

```
logisticRegressionModel = cvModel.bestModel.stages[-1]
print(f"regParam óptimo: {logisticRegressionModel.getOrDefault('regParam')}")
print(f"elasticNetParam óptimo: {logisticRegressionModel.getOrDefault('regParam')}")
```

```
regParam óptimo: 0.01
elasticNetParam óptimo: 0.01
```

CHAPTER

FIVE

CAPÍTULO 5: STRUCTURED STREAMING Y GRAPHFRAMES

5.1 Structured Streaming

El procesamiento en tiempo real o streaming es el acto de incorporar de manera continua nuevos datos **para ir actualizando el resultado del cálculo**. Lo importante no es simplemente recibir datos en tiempo real sino que, en ese mismo instante, los datos tienen que ser utilizados por la aplicación para hacer un cálculo con ellos, cuyo resultado debe darse también en tiempo real. Los datos de entrada no tienen principio ni fin, y se pueden ver como series de eventos que van llegan a la aplicación y van siendo procesados.

El programa calcula nuevas versiones del resultado según van llegando nuevos datos, de manera que Spark automáticamente actualiza el resultado cuando recibe nuevos datos, sin necesidad del control del usuario.

5.1.1 Casos de uso típicos

Algunos casos de uso típicos en los que se necesita procesar datos en tiempo real son:

- Alertas: un nuevo dato causa que se actualice cierto resultado y a consecuencia, se envíe una alerta a una persona
- Reporting en tiempo real: cuadros de mando construidos sobre agregaciones de datos que se actualizan en tiempo real
- ETL incremental: "mi trabajo batch pero en streaming". Limpiar y procesar datos en crudo según van llegando, para escribirlos a un data warehouse.
- Decisiones en tiempo real: ¿es fraudulenta esta transacción? Denegar sobre la marcha en caso de serlo
  - Usando reglas cableadas, o bien usando un modelo de ML
  - Usando un modelo de ML que se actualiza (sin re-entrenar) según llegan nuevos datos (online learning, solo si el modelo lo permite). Es lo menos frecuente al existir pocos algoritmos así, y Spark no los incorpora.

Spark Streaming es el módulo para procesamiento en tiempo real de flujos de datos con Spark. Existen dos APIs distintas para esto.

- La API original que se utilizaba antes de Spark 2 era la API de DStream. Es una API de bajo nivel basada en RDDs y una estructura llamada DStream que es el realidad una colección de RDDs. Proporciona muy poca ayuda a los problemas adicionales que surgen cuando se tienen que procesar datos que van llegando en el tiempo, tales como mantener un estado, procesar datos desordenados, etc.
- La API actual, que se introdujo a partir de Spark 2, se denomina Structured Streaming y es la que se utiliza actualmente. Se basa en un mecanismo que permite aplicar la API estructurada que ya hemos usado (API de Spark SQL y DataFrames) a datos que van llegando en tiempo real, gracias a una abstracción llamada Streaming DataFrames.

5.1.2 Retos de los sistemas de procesamiento real y fuentes de datos típicas

El diseño de sistemas de procesamiento de datos en tiempo real da lugar a una serie de retos propios de este tipo de situaciones:

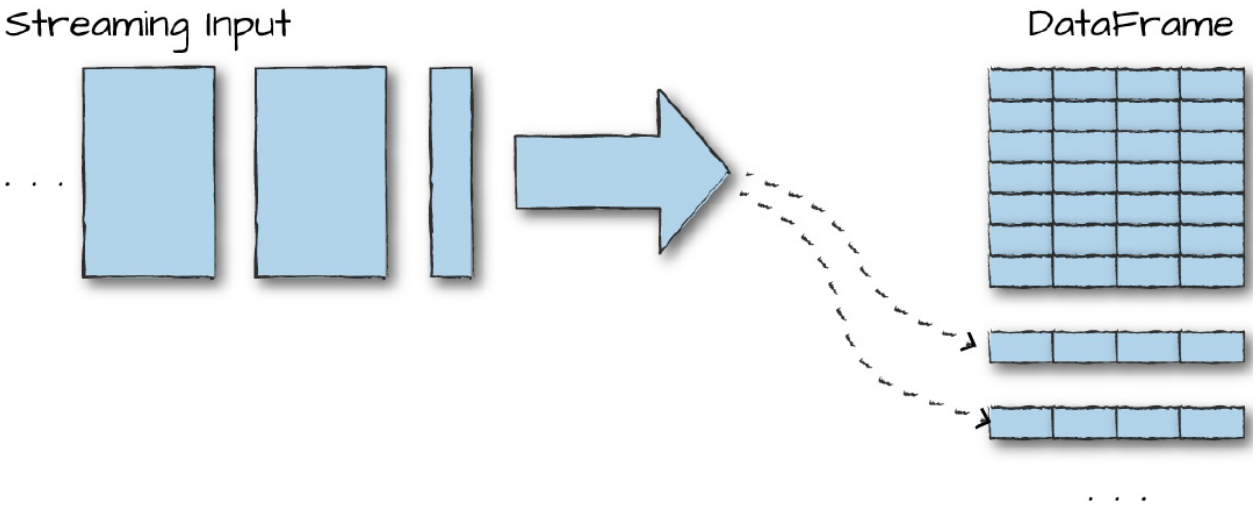
- Ofrecer baja latencia, es decir, respuesta rápida a un solo evento.
- Ofrecer alto throughput, o en otras palabras, ser capaces de procesar rápidamente un volumen alto de entradas que se han recibido en muy poco tiempo. Se mide en salidas (resultados) por unidad de tiempo. Suele estar en contraposición con una baja latencia.
- Ser capaces de procesar datos desordenados (recibidos en orden diferente al que se generaron)
- Mantener un estado interno para determinar qué está pasando
- Ser capaz de unir datos en streaming con datos batch históricos
- Ofrecer robustez frente a pérdida de datos o a datos que llegan duplicados (se recibe más de una vez el mismo dato), pero sin duplicar salidas.

Las fuentes de datos más habituales de las que se suelen recibir datos en tiempo real cuando los vamos a procesar con Spark son :

- Apache Kafka: cola de mensajes distribuida, a la que se le añaden mensajes por un extremo y se consumen por otro. Spark generalmente adopta el rol de consumidor, y asumimos que otros procesos externos son los que están generando en tiempo real nuevos datos y los van escribiendo en Kafka para ser procesados por Spark en algún momento posterior también en tiempo real. Esto es lo más habitual en los sistemas desplegados en producción en la actualidad.
- Sistema de ficheros (DBFS, Azure DataLake Storage, etc): con Structured Streaming es posible monitorizar un directorio de DBFS ó ADLS, en el cual algún otro proceso externo está creando archivos, y del que Spark va leyendo y procesando archivos al mismo tiempo.

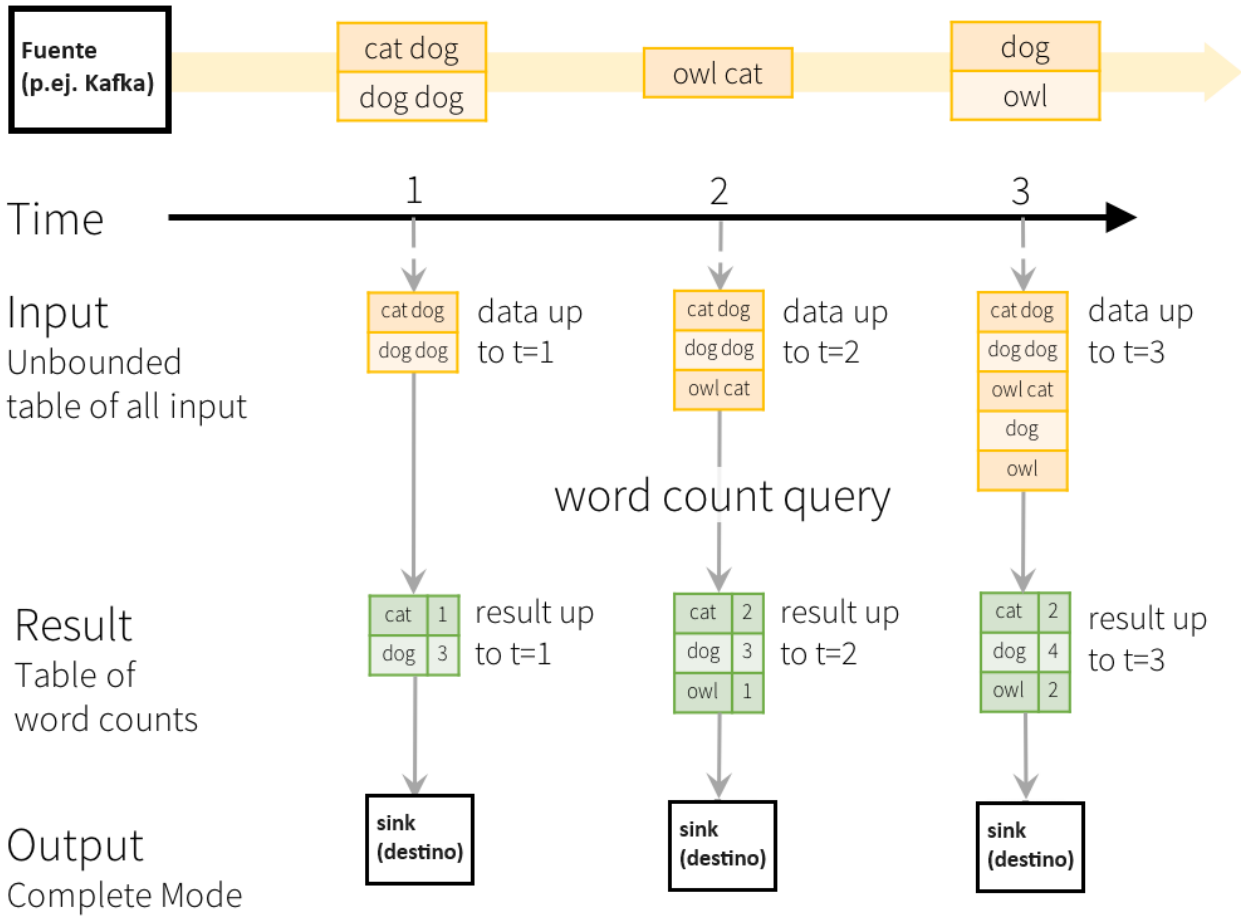
La API original de Spark, previa a la versión 2, utilizaba DStreams, que son una secuencia de RDDs. El procesamiento se basa en microbatches: los datos que van llegando se van acumulando en una pequeña ventana (por ejemplo de 500 ms) para formar un pequeño RDD que se procesa en paralelo con Spark, como si fuese un trabajo batch. Esto no es un verdadero sistema de procesamiento en tiempo real, y además tiene más latencia, pero a cambio también mejor throughput. Spark optimiza el throughput en su procesamiento en streaming. Los sistemas que realmente procesan datos de forma continua tienen baja latencia pero mayor sobrecarga por registro, y se acentúa cuando recibimos muchos datos de entrada casi a la vez (suelen tener bajo throughput). En la nueva API de Structured Streaming, seguimos teniendo microbatches, aunque hay una propuesta para soportar verdadero procesamiento continuo, implementada en modo experimental y que obtiene latencias de 1 milisegundo, frente a los 100 milisegundos que ofrece el procesamiento en micro-batches . La idea de Structured Streaming es poder utilizar la misma API de Spark SQL (la API estructurada, basada en DataFrames) a pesar de manejar un DataFrame de tiempo real:

- ¡El mismo código batch que ya teníamos, transformado a streaming!
- Conceptualmente, un Streaming DataFrame es un DataFrame al que se le van añadiendo filas en tiempo real. La filosofía es que el mismo código que ya funcionaba para un trabajo batch debería servir sin cambios en Structured Streaming.



- Restricción: solo hay una acción disponible sobre un Streaming DataFrame: arrancar un flujo que permanecerá ejecutando continuamente la secuencia de transformaciones que le hayamos indicado, actualizando el resultado continuamente.

La idea para programar con Structured Streaming se muestra en la figura siguiente, mostrando cómo resolveríamos el problema del word count (contar el número de ocurrencias de cada palabra de un DataFrame, como df.groupBy("word").count(), que es un código que podemos ejecutar tal cual sobre un DF batch o sobre un Streaming DF). Habitualmente escribiremos un código que leerá de una fuente (específicamente una que soporte procesamiento en streaming, por ejemplo Kafka aunque existen otras), y obtendremos así un Streaming DataFrame. Le aplicaremos transformaciones mediante la API estructurada, y finalmente el DF resultante, que será también un Streaming DataFrame, lo escribiremos en un destino que soporte también escrituras en tiempo real, conocido genéricamente como "sink". Para ello, tras cada intervalo de ejecución (trigger interval), Spark habrá añadido automáticamente más filas al Streaming DF leído como entrada (un nuevo micro-batch), y estas causarán que se actualice el resultado de la tabla final que estamos guardando. Cada vez que dicha tabla cambia, queremos escribir el resultado actualizado en nuestro almacenamiento de destino. La manera de escribir depende del modo de escritura que hayamos elegido, el cual además depende también del tipo de cálculo que estemos haciendo, ya que no todos los modos de escritura tienen sentido en cualquier cálculo.



Note: Spark nunca materializa la tabla de entrada completa ni tampoco recalcula usando de nuevo todos los datos desde el principio en cada paso, sino que incrementaliza el cálculo: sólo procesa las filas nuevas, las añade al estado interno (intermedio) que mantenía para esta consulta (secuencia de transformaciones) concreta, y lo usa para actualizar el resultado.

En otras tecnologías el usuario es responsable de mantener estos resultados intermedios para agregarlos con los nuevos datos, pero Spark se ocupa de todo esto, y también de qué ocurre si falla algo durante el procesamiento de un mini-batch de datos.

5.1.3 Modos de escritura

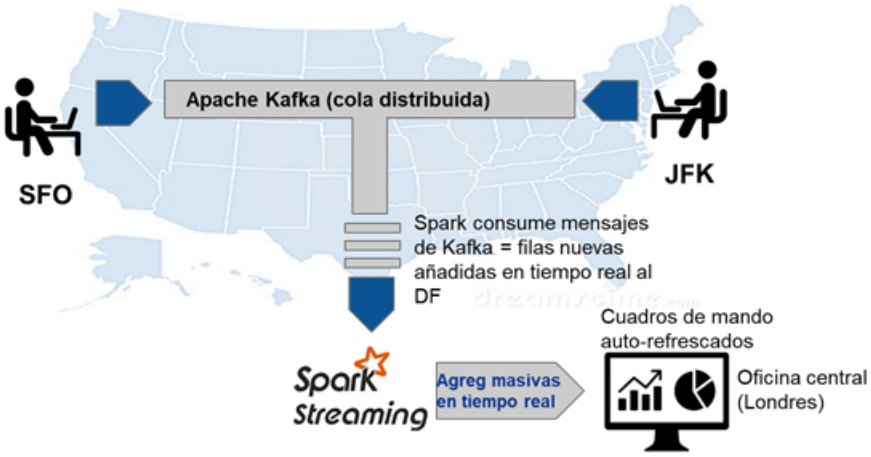
Cuando escribimos resultados en tiempo real, hay varios modos de escritura:

- Añadir ("append"): solo se escriben los nuevos registros al sink de destino. Se debe aplicar cuando las filas ya escritas no van a cambiar, sino que siempre se van a generar nuevas filas que no interfieren con las que ya se habían calculado previamente.
- Actualizar ("update"): sólo se escribirán los registros que han cambiado desde la escritura anterior (último trigger). Si las transformaciones que estamos haciendo no contienen agregaciones, que provoquen que alguna fila ya calculada pueda haber cambiado, este modo de escritura se comporta igual que append.
- Completo ("complete"): se re-escribe la tabla completa, y depende del almacenamiento destino la manera de decidir cómo almacenar el contenido que ya existía. Adecuado para agregaciones. Dependiendo del tipo de transformación

que hagamos, algunos de estos modos no tienen sentido. En nuestro ejemplo de word count, append no tendría sentido porque los resultados de la agregación deben actualizarse, y sigue habiendo el mismo número de registros. En cambio, en una aplicación donde estamos utilizando un modelo predictivo ya entrenado para realizar predicciones sobre nuevos datos que vamos recibiendo en tiempo real, necesitamos append para que cada nueva predicción obtenida sea añadida al resultado, teniendo en cuenta que las predicciones ya realizadas no van a modificarse.

5.1.4 Ejemplo de aplicación al dataset de vuelos

Imaginemos que los aeropuertos envían en tiempo real información a nuestro sistema sobre cada vuelo que aterriza. Se podrían utilizar distintas arquitecturas para esto. La más habitual sería que Spark leyese en tiempo real desde Kafka, una cola de mensajes distribuida y replicada donde operarios que trabajan en los aeropuertos van escribiendo información de vuelos que van aterrizando.



El código para leer un Streaming DataFrame desde Kafka, suscribiéndonos al topic "vuelos", sería similar a:

```
vuelos_streaming_df = (
    spark.readStream
         .format("kafka")
         .option("kafka.bootstrap.servers", "hostname.domain:9092")
         .option("subscribe", "vuelos")     # nos suscribimos a todas las particiones␣
↪del topic
         .load()
)
```

A continuación escribimos una secuencia de transformaciones sobre vuelos_streaming_df, el cual es un Streaming-DataFrame sobre el que podemos aplicar la API estructurada similar a Spark SQL. Por último, escribimos el resultado de dicha transformación, que es otro streaming DF. Como estamos en modo test, lo escribimos a memoria principal, concretamente a una nueva tabla temporal de Hive, e invocamos sobre él la acción start que desencadena la ejecución. Por último, para que el programa driver no finalice sino que se mantenga esperando indefinidamente, invocamos a awaitTermination. No obstante, en el ejemplo que veremos en el notebook no es necesario este último paso porque el programa driver es el propio notebook, y por tanto se mantendrá corriendo hasta que nosotros cerremos manualmente el kernel del notebook. Podemos ver cómo se va actualizando el resultado en memoria haciendo operaciones de consulta de la tabla resultante en Hive con la operación .sql("consulta"). Si la mostramos varias veces, veremos cada vez un contenido distinto que se va actualizando a medida que Spark va leyendo y procesando nuevos datos de entrada de Kafka.

```
# Operaciones de preprocesamiento como haríamos con cualquier DataFrame
countStreamingDF = vuelos_streaming_df \
                    .withColumn("value", F.col("value").cast("string"))\
```

(continues on next page)

(continued from previous page)

```
                    .withColumn("parsed_flight", F.from_json("value", flights_
↪schema))\
                    .groupBy("Origin").count()

# Escribimos periódicamente el resultado. Para desarrollos, podemos escribir a memoria

countQuery = countStreamingDF\
              .writeStream.queryName("countsPerAirport")\
               .format("memory")\
               .outputMode("complete")\    # Como es una agregación, escribimos la␣
↪salida completa cada vez
               .start()                    # arrancamos el flujo de datos

# lo siguiente es obligatorio para evitar que el driver finalice aunque
# cuando es un notebook, no es necesario
countQuery.awaitTermination()
```

5.1.5 Watermarks y event-time

Si tenemos en nuestros datos una columna específica (habitualmente de tipo timestamp) del Streaming DataFrame que indica el instante exacto en el cual fue generado ese dato, podemos utilizarla para indicarle a Spark con cuánto retraso ("intervalo de retardo") esperamos recibir cada dato. Esto le sirve a Spark para poder liberar progresivamente parte de los datos almacenados en su estado interno más antiguo, ya que de otro modo, habría agregaciones (no todas) en las que el estado intermedio que necesita mantener crecería indefinidamente en un proceso que está siempre ejecutándose en producción.

5.2 Procesamiento de grafos con GraphFrames

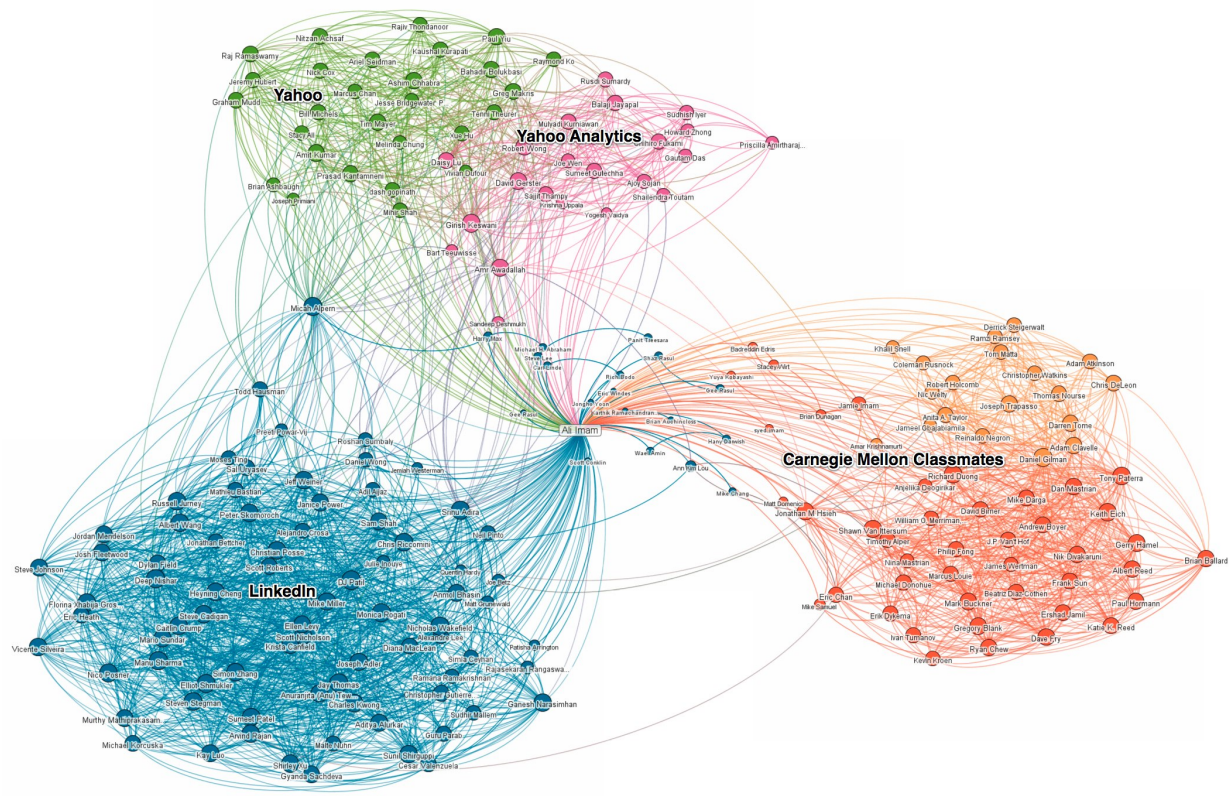
Desde casi el primer momento, Spark posee un módulo para procesar grafos llamado GraphX. Utilizando operaciones distribuidas, implementa algoritmos frecuentes sobre grafos, los cuales están representados mediante RDDs. No obstante, su manejo era tedioso y pronto se vio reemplazado de facto por la biblioteca GraphFrames que estudiamos aquí.

5.2.1 Grafos en la vida real

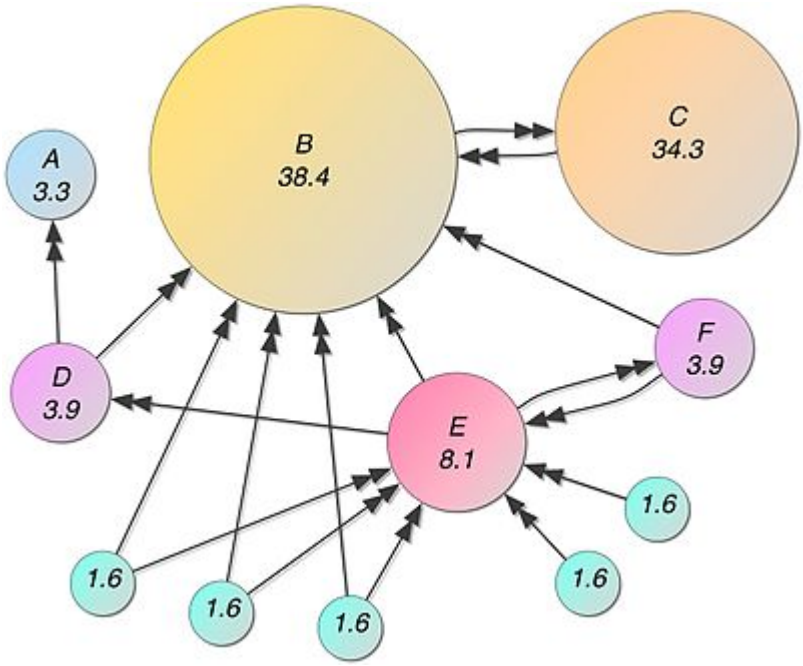
Un grafo modela relaciones entre objetos en una estructura con vértices (nodos) y arcos (aristas). Los vértices son objetos. Los arcos conectan dos vértices si existe relación entre ellos.

Los nodos y los arcos pueden representar cualquier concepto, y ambos pueden tener propiedades asociadas. El ejemplo perfecto es el de una red social, donde los vértices son personas, y las aristas representan la relación de amistad entre dos personas en la red social. Analizando esta representación en grafo podemos obtener información útil, como por ejemplo:

- Detección de comunidades: muchas relaciones entre sus miembros, pocas con miembros de fuera
- Detección de personas clave ("influencers") que participan en muchos caminos entre otras dos personas



Otro ejemplo típico es el de las páginas web en internet. Los vértices son páginas web, y las aristas indican si un sitio web contiene un link que lleva a otro. El algoritmo PageRank de Google mide la importancia de un sitio web como un problema de grafos, en función de cómo está relacionado con los demás, y qué importancia tienen aquellos sitios con los que está relacionado (es decir, sitios que contienen un link a esa web).



Un caso diferente pero muy útil es la identificación (tracking) de dispositivos en la navegación por Internet. Los vértices pueden tener distinta naturaleza: pueden representar una dirección IP, o bien un dispositivo, o bien una cookie.

Recordemos que una cookie es un fichero creado por el sitio web en el ordenador del usuario al entrar la primera vez, para identificar visitas recurrentes. Las aristas representan sesiones (cuando un usuario empieza a navegar por nuestra web). El usuario utiliza cierto dispositivo, con cierta IP y con una cookie, que podía existir ya en su dispositivo, o no (si es la primera vez). Las posibles aplicaciones de esta representación se basan en que la misma cookie puede usarse en varias sesiones con distinta IP (por ejemplo si nuestro proveedor de internet ha cambiado la IP, o nos hemos cambiado de compañía), o que un usuario puede usar distintos dispositivos (ordenador portátil, teléfono móvil, Tablet…) desde una misma IP. Al final, un conjunto de vértices interconectados (independientemente de que algunos sean IPs, cookies, etc) representan a la misma persona.

5.2.2 Concepto de grafo

Un grafo es una dupla G = (V, E) siendo V el conjunto de vértices y E el de aristas. Cada vértice puede representar cualquier concepto. Incluso en un mismo grafo pueden convivir vértices que representen cosas distintas (ejemplo de cookies e IPs). Las aristas pueden ser dirigidas o no dirigidas (y así se denomina al grafo). Las aristas pueden tener pesos asociados (ej: distancia, o cualquier otra métrica del grado de intensidad de la conexión). Definimos:

- Grado de entrada de un vértice: número de aristas que llegan al vértice.
- Grado de salida: número de aristas que parten de él.
- Grado de un vértice: suma de ambos grados.

En el dataset de vuelos:

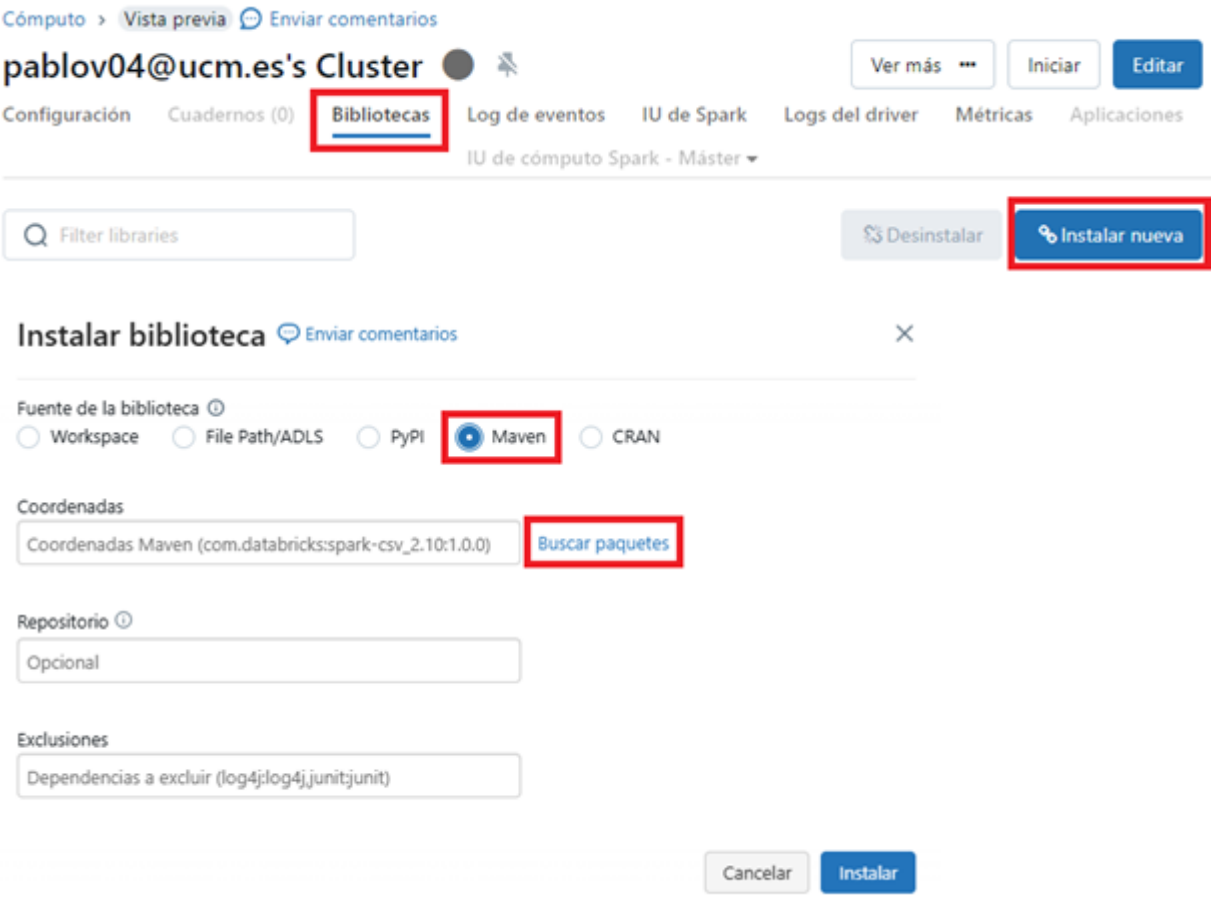
- Vértices: aeropuertos
- Aristas (dirigidas): cuando existe un vuelo directo desde un aeropuerto a otro
- Peso de cada arista: distancia entre los aeropuertos. Podría ser también el número de vuelos diarios entre ellos, o cualquier otra métrica.

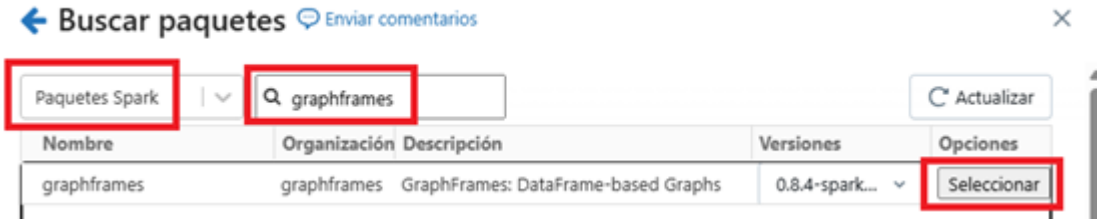
Para calcular, el grafo suele representarse mediante su matriz de adyacencia: una matriz cuadrada de orden igual al número de vértices, donde el valor de la casilla (i, j) puede ser 1 si existe una arista que va desde el vértice i al vértice j o 0 en caso contrario, si el grafo no tiene pesos. Si el grafo tiene pesos, entonces el valor de la casilla (i, j) es igual al peso asociado a esa arista, o infinito si no existe arista.

5.2.3 Grafos con Spark

GraphX es el módulo de Spark para procesamiento y algoritmos sobre grafos con RDDs. Forma parte de Spark pero su API es de bajo nivel, y es incómoda de usar, además de no tener optimizaciones en tiempo de ejecución por utilizar RDDs. Puesto que la mayoría de usuarios preferían usar DataFrames, surgió la biblioteca GraphFrames como una evolución de GraphX con DataFrames. Será lo que veremos en este curso. Es un paquete separado que no forma parte de Spark propiamente, por lo que hay que instalarlo expresamente. Está muy extendida y estaba planificado que pasase a formar parte de la descarga de Spark estándar, pero llevan casi dos años sin fusionarlo definitivamente.

GraphFrames está implementado en lenguaje Scala, y por tanto tendremos que instalar un paquete de Scala que se distribuye como un fichero JAR. Después, podemos invocar a dicho código desde Python gracias a un wrapper que se distribuye como un fichero .py con la propia dependencia de Scala (hace un tiempo era necesario instalar por separado un paquete de Python también, pero actualmente no es necesario). En Databricks podemos instalar un paquete en todos los nodos mediante la interfaz gráfica:





Los paquetes de Java y Scala se pueden instalar de manera automática a partir del repositorio centralizado de paquetes de Maven Central (algo similar al repositorio PyPI que usamos en Python) indicando las coordenadas Maven de la dependencia que queremos instalar. Recordemos que Maven es un software gestor del ciclo de desarrollo y también un repositorio centralizado (Maven Central) de bibliotecas (paquetes en formato JAR) para Java/Scala.

A pesar de haber instalado solamente la dependencia JAR de Scala, podremos invocar al paquete usando las clases de Python porque el wrapper viene incorporado. También es posible utilizar un cluster con DBR (Databricks Runtime) de tipo ML (cualquier versión reciente, por ejemplo DBR 15.4 ML) que trae ya instalada una versión de GraphFrames modificada y optimizada por Databricks.

5.2.4 Grafos con el paquete GraphFrames

En el paquete GraphFrames un grafo se representa mediante dos DataFrames de Spark:

- DataFrame de vértices: debe tener una columna llamada id
- DataFrame de aristas: debe tener columnas llamadas src y dst con los IDs de los vértices que conecten.
- Además de esas columnas obligatorias, pueden existir otras en ambos DF, ya que no intervendrán en los algoritmos que después ejecutaremos sobre el grafo.

Ejemplo:

```
from pyspark.sql import SparkSession, functions as F
from graphframes import GraphFrame

# Opción alternativa para incorporar una dependencia de Java al vuelo, al crear la␣
 ↪sesión de Spark
# Esto no es necesario en el cluster de Databricks porque ya hemos instalado la␣
 ↪dependencia en la
# pestaña de Bibliotecas
spark = SparkSession.builder\
                    .config('spark.jars.packages','graphframes:graphframes:0.8.1-
 ↪spark3.0-s_2.12')\
                    .getOrCreate()

flightsDF = spark.read.option("header", "true")\
                      .option("inferSchema", "true")\
                      .csv("flights.csv")

verticesDF = flightsDF.select(F.col("Origin").alias("id"))\
                      .distinct().cache()

edgesDF = flightsDF.withColumnRenamed("Origin", "src")\
                   .withColumnRenamed("Dest", "dst")\
                   .select("src", "dst", "Distance")\
                   .distinct()\
                   .cache()

graph = GraphFrame(verticesDF, edgesDF)
```

```
C:\Users\Lenovo\anaconda3\envs\pyspark\lib\site-packages\pyspark\sql\dataframe.
 ↪py:168: UserWarning: DataFrame.sql_ctx is an internal property, and will be␣
 ↪removed in future releases. Use DataFrame.sparkSession instead.
  warnings.warn(
```

Una vez construido el grafo podemos aplicar distintos algoritmos. Algunos algoritmos devuelven un DataFrame y otros, un grafo completamente nuevo. Grados de entrada y salida: el grafo calcula tres DataFrames:

inDegrees por vértice

- outDegrees por vértice
- degrees por vértice (suma de los grados de entrada y de salida)

```
graph.inDegrees.orderBy(F.col("inDegree").desc()).show(3)

graph.outDegrees.orderBy(F.col("outDegree").desc()).show(3)
```

(continues on next page)

(continued from previous page)

```
graph.degrees.orderBy(F.col("degree").desc()).show(3)
```

```
C:\Users\Lenovo\anaconda3\envs\pyspark\lib\site-packages\pyspark\sql\dataframe.
↪py:147: UserWarning: DataFrame constructor is internal. Do not directly use it.
  warnings.warn("DataFrame constructor is internal. Do not directly use it.")
```

```
+---+--------+
| id|inDegree|
+---+--------+
|ORD|     165|
|ATL|     161|
|DFW|     157|
+---+--------+
only showing top 3 rows
```

```
+---+---------+
| id|outDegree|
+---+---------+
|ORD|      165|
|ATL|      160|
|DFW|      157|
+---+---------+
only showing top 3 rows
```

```
+---+------+
| id|degree|
+---+------+
|ORD|   330|
|ATL|   321|
|DFW|   314|
+---+------+
only showing top 3 rows
```

5.2.5 Cálculo de componentes conexas:

- Para grafos nos dirigidos: son sub-grafos que están aislados del resto del grafo, es decir, dos vértices cualesquiera están conectados y no es posible viajar fuera de ese subgrafo.
- Para grafos dirigidos, como en el problema de los vuelos, se utiliza el concepto de componentes fuertemente conexas. Son grupos de vértices tales que dos vértices cualesquiera del grupo están conectados. Serían grupos de aeropuertos tales que desde cualquiera del grupo es posible llegar a cualquier otro, directamente o a través de pasos intermedios. En nuestro dataset de vuelos sólo hay una componente conexa (formada por el grafo entero) porque siempre es posible volar de cualquier aeropuerto a cualquier otro, a través de conexiones intermedias con otros aeropuertos.

Este algoritmo devuelve un DataFrame:

```
spark.sparkContext.setCheckpointDir(".")
resultDF = graph.connectedComponents()
```

(continues on next page)

(continued from previous page)

```
resultDF.show(20)    # todos los vértices caen en la misma componente conexa, ya que␣
↪todos los aeropuertos están interconectados

resultDF.groupBy("component").count().show()  # El 0 es un identificador de␣
↪componente conexa
```

```
+---+---------+
| id|component|
+---+---------+
|ABE|        0|
|ABI|        0|
|ABQ|        0|
|ABR|        0|
|ABY|        0|
|ACT|        0|
|ACV|        0|
|ACY|        0|
|ADK|        0|
|ADQ|        0|
|AEX|        0|
|AGS|        0|
|ALB|        0|
|ALO|        0|
|ALW|        0|
|AMA|        0|
|ANC|        0|
|APN|        0|
|ART|        0|
|ASE|        0|
+---+---------+
only showing top 20 rows
```

```
+---------+-----+
|component|count|
+---------+-----+
|        0|  356|
+---------+-----+
```

5.2.6 Algoritmo PageRank

Algoritmo iterativo que utiliza el grado de entrada y de salida de los vértices para determinar la importancia de cada vértice. Es bastante pesado desde el punto de vista computacional. Devuelve un nuevo grafo en el que los vértices tienen una nueva columna de números reales llamada pagerank con la puntuación otorgada por el algoritmo a cada vértice y que indica su importancia. Requiere establecer el checkpointDir de Spark, una carpeta donde cada cierto número de iteraciones se guardan los resultados parciales del algoritmo para casos de fallo.

```
ranksGraph = graph.pageRank(resetProbability=0.15, maxIter=10)
ranksGraph.vertices.orderBy(F.col("pagerank").desc()).show()
```

```
+---+------------------+
| id|          pagerank|
```

(continues on next page)

(continued from previous page)

```
+---+------------------+
|DFW|10.473812993687384|
|ORD|10.260145353782415|
|ATL|  9.43038674195667|
|DEN| 8.853062782143189|
|MSP| 7.554764057993901|
|CLT|   7.2447963343465|
|DTW|6.2044026817052975|
|LAS| 6.105757091251459|
|IAH| 5.961382132435387|
|PHX| 5.365815675762966|
|SEA|  5.31619944641921|
|SLC| 5.044415678853287|
|PHL| 4.882291006095359|
|LAX| 4.771622359389784|
|EWR| 4.533074360459349|
|SFO| 4.503367605949667|
|SFB| 4.292855657463282|
|DCA|4.2826697449293665|
|IAD| 3.987501939440854|
|MCO| 3.838361785555211|
+---+------------------+
only showing top 20 rows
```

5.2.7 Algoritmo Breadth-first search

BFS: sirve para calcular el camino más corto entre dos conjuntos de vértices, definido como el más corto en cuanto al número de aristas (no se tienen en cuenta los pesos asociados a una arista). Los conjuntos de partida y llegada los define una expresión booleana. Devuelve un DataFrame como resultado, con una columna que contiene la sucesión de vértices que hay que recorrer para llegar de uno a otro. Todas las columnas son de tipo estructura (struct). En las columnas donde se representan vértices, la estructura contiene como campos anidados las columnas del DataFrame de vértices, que en nuestro caso era solamente el `id`, mientras que en las columnas donde se representan aristas, contiene los 4 campos del DataFrame de aristas (`src`, `dst`, `Distance`).

```
paths = graph.bfs(fromExpr = "id = 'LAX'", toExpr= "id = 'SFO'")
paths.show(truncate=False)
```

```
+-----+-----------------+-----+
|from |e0               |to   |
+-----+-----------------+-----+
|{LAX}|{LAX, SFO, 337.0}|{SFO}|
+-----+-----------------+-----+
```

El paquete GraphFrames no implementa algoritmos de caminos que utilicen los pesos de las artistas, como por ejemplo Dijkstra, Floyd-Warshall, etc. Algoritmo de búsqueda de motivos (motif finding): búsqueda en base a consultas estructurales. Por ejemplo, encontrar dos aeropuertos que no estén directamente conectados pero que se pueda llegar de uno a otro a través de una conexión intermedia. El filtrado posterior evita que nos devuelva todos los aeropuertos ya que nunca es posible ir desde un aeropuerto a sí mismo pero sí a través de cualquier otro aeropuerto intermedio, por lo que en realidad cualquier aeropuerto individualmente cumple la condición respecto de sí mismo. Evidentemente esto no es lo que buscamos, así que los excluimos del resultado. Devuelve un DataFrame:

```
res = graph.find("(a)-[]->(b); (b)-[]->(c); !(a)-[]->(c)")\
           .filter("c.id != a.id")

res.show(truncate=False)
```

```
+-----+-----+-----+
|a    |b    |c    |
+-----+-----+-----+
|{ACY}|{ATL}|{LAX}|
|{AEX}|{DFW}|{KOA}|
|{AUS}|{DEN}|{MFR}|
|{AUS}|{MDW}|{PBI}|
|{BDL}|{ORD}|{CAK}|
|{BDL}|{RDU}|{BNA}|
|{BHM}|{MDW}|{BOS}|
|{BIL}|{DEN}|{GSO}|
|{BMI}|{ORD}|{AUS}|
|{BOS}|{MIA}|{BDL}|
|{BOS}|{SEA}|{TUS}|
|{BUF}|{EWR}|{CVG}|
|{BUF}|{ORD}|{MOB}|
|{CAE}|{CLT}|{CHS}|
|{CAK}|{TPA}|{BHM}|
|{CDC}|{SLC}|{MTJ}|
|{CHA}|{ATL}|{MLI}|
|{CHA}|{ORD}|{XNA}|
|{CLT}|{DFW}|{SUX}|
|{CMH}|{ATL}|{PIT}|
+-----+-----+-----+
only showing top 20 rows
```